

1 Appendix MS-A: Arithmetic Interface Meta-Theory

MS-A.1. Scope and interface status

This appendix defines the meta-theory by which arithmetic data may be connected to the AK Core. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Arithmetic realization is an interface, not an arithmetic theorem.

Accordingly, this appendix does not prove the Birch–Swinnerton-Dyer conjecture, the ABC conjecture, the Riemann hypothesis, Iwasawa main conjectures, modularity theorems, Selmer finiteness statements, p -adic L -function identities, or any other arithmetic theorem. It only specifies the admissible form of an arithmetic interface into the AK Core.

All Core-relevant persistence data produced by an arithmetic interface are evaluated through the repaired windowed profile:

$$\text{Eval}_{i,W,\tau}(F) := T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Remark 1.1 (Interface status). The material here is interface-level material. Any implication from arithmetic structure to Core-visible persistence, collapse, diagnostic, or certificate behavior is [Spec] by default unless separately promoted and reproved under the Core guard-rails.

Remark 1.2 (Relation to Chapter 8). Chapter 8 specified the general interface protocol. This appendix specializes that protocol to arithmetic data. In particular, every admissible arithmetic interface must provide:

- (i) a purpose-faithful arithmetic realization hypothesis;
- (ii) a declared Core-readable realization target;
- (iii) repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F);$$

- (iv) admissible representative data

$$C_{\tau,W}F$$

whenever filtered or derived clauses are invoked;

- (v) eligibility whenever the Ext^1 -clause is invoked;
- (vi) declared tower comparison artifacts whenever Type IV diagnostics are invoked;
- (vii) strict separation between arithmetic pre-gates and Core gates;
- (viii) ledger entries for arithmetic transfer, extraction, comparison, approximation, or truncation defects;
- (ix) [Spec] status for problem-specific arithmetic implications by default.

Remark 1.3 (No shortcut through arithmetic meaning). Arithmetic meaning may motivate a construction, but it cannot bypass the repaired Core verifier. In particular, no arithmetic invariant, conjectural equality, height computation, L -value, Selmer rank, Iwasawa invariant, or modularity input replaces the required Core objects:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad \Sigma\delta < \text{gap}_{\tau}.$$

MS-A.1bis. Arithmetic interface sovereignty

Definition 1.4 (Arithmetic interface sovereignty). *Arithmetic interface sovereignty* is the rule that arithmetic source data, arithmetic meanings, domain-specific theorems, numerical computations, literature references, and conjectural structures remain outside the AK Core unless they re-enter through declared Core-readable verifier-side data. The only admissible verifier-side re-entry routes are:

- (i) constructible one-parameter persistence objects in $\text{Pers}_k^{\text{cons}}$;
- (ii) finite-type filtered objects in $\text{FiltCh}(()k)$;
- (iii) repaired evaluation records $\text{Eval}_{i,W,\tau}(F)$;
- (iv) admissible representatives $C_{\tau,W}F$;
- (v) eligible realized objects for an invoked Ext^1 -clause;
- (vi) declared tower comparison artifacts $\phi_{i,W,\tau}$;
- (vii) typed Type IV diagnostic records;
- (viii) certified metric-ledger entries;
- (ix) audit-readable certificate records satisfying the Minimal Manifest Axiom.

Proposition 1.5 (No arithmetic sovereignty override). *No arithmetic theorem, conjecture, invariant, computation, or literature artifact overrides the Core verifier-side requirements.*

Proof. The AK Core consumes only typed verifier-side records: repaired evaluations, representatives, eligibility records, tower artifacts, diagnostics, ledgers, gate verdicts, and manifest data. Arithmetic material has external mathematical semantics. Unless it is realized as one of the declared Core-readable records, it is not an input to the Core verdict function. $\square$

Remark 1.6 (Arithmetic meaning is proposer-side by default). Arithmetic meaning may guide realization, window selection, threshold selection, tower construction, or bridge design. By default it is proposer-side information, not verifier-side evidence.

MS-A.2. Arithmetic source data

Definition 1.7 (Arithmetic source datum). An *arithmetic source datum* is a mathematical object or structured collection of objects arising from arithmetic geometry, number theory, or arithmetic representation theory. Typical examples include:

- (i) elliptic curves or abelian varieties over global fields;
- (ii) number fields, local fields, or global function fields;
- (iii) Galois representations;
- (iv) Selmer groups, Selmer complexes, or local Selmer conditions;
- (v) Iwasawa modules and Iwasawa towers;
- (vi) cyclotomic, anticyclotomic, congruence, or level towers;

- (vii) congruence modules and Hecke modules;
- (viii) modular, automorphic, or eigenvariety data;
- (ix) classical, motivic, or p -adic L -functions;
- (x) Newton polygons, valuation profiles, or characteristic ideals;
- (xi) motivic, cohomological, or derived arithmetic complexes;
- (xii) height, conductor, regulator, Tamagawa, or local correction data.

Definition 1.8 (Arithmetic interface family). An *arithmetic interface family* is a declared class of arithmetic source data together with a proposed realization route into filtered objects, constructible persistence objects, eligible realized objects, tower comparison artifacts, or certificate-record components.

Definition 1.9 (Arithmetic source category). An *arithmetic source category* is a declared category, groupoid, indexed family, tower, diagram, or structured collection

$$\mathcal{A}_{\text{arith}}$$

whose objects are arithmetic source data and whose morphisms encode the arithmetic relationships relevant to the intended interface.

Remark 1.10 (Arithmetic data are not automatically Core data). An arithmetic object is not automatically a Core object. It becomes Core-readable only after a declared realization route produces one of:

$$\text{FiltCh}(()k), \quad \text{Pers}_k^{\text{cons}}, \quad D^{[-1,0]}(k\text{-mod}),$$

or an admissible certificate-record component compatible with the repaired Core protocol.

MS-A.3. Purpose-faithful arithmetic realization

Definition 1.11 (Purpose-faithful arithmetic realization hypothesis). A *purpose-faithful arithmetic realization hypothesis* is a declared statement of the form

$$\mathfrak{R}_{\text{arith}} : \mathcal{A}_{\text{arith}} \longrightarrow \text{FiltCh}(()k) \quad \text{or} \quad \mathfrak{R}_{\text{arith}} : \mathcal{A}_{\text{arith}} \longrightarrow \text{Pers}_k^{\text{cons}},$$

together with the claim that $\mathfrak{R}_{\text{arith}}$ preserves the arithmetic features relevant to the intended Core test.

Definition 1.12 (Arithmetic realization target). An arithmetic interface must declare whether its realization target is:

- (i) a filtered chain object in $\text{FiltCh}(()k)$;
- (ii) a constructible persistence object in $\text{Pers}_k^{\text{cons}}$;
- (iii) a repaired windowed collapsed profile

$$\text{Eval}_{i,w,\tau}(F) = T_{\tau}^{\text{bar}}(W_w \mathbf{P}_i(F));$$

- (iv) an admissible representative

$$C_{\tau,w}F;$$

(v) a realized eligible object in

$$D^{[-1,0]}(k\text{-mod});$$

(vi) a tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

(vii) a certificate-record component;

(viii) a non-Core advisory, Search, or explanatory artifact.

Definition 1.13 (Purpose profile). A *purpose profile* of an arithmetic interface records:

- (i) the arithmetic feature intended to be tested;
- (ii) the Core-visible feature intended to represent it;
- (iii) the monitored degrees;
- (iv) the windows and thresholds;
- (v) the expected gate or diagnostic clause;
- (vi) the direction of intended transport.

Remark 1.14 (Purpose-faithful is not categorical faithfulness). The phrase “purpose-faithful” does not mean that

$$\mathfrak{R}_{\text{arith}}$$

is faithful as a functor in the strongest categorical sense. It means that the realization preserves the features required for the declared interface purpose. For example, if the interface is designed to test a Selmer-like obstruction, the realization must preserve the relevant obstruction profile. It need not preserve every arithmetic invariant of the original object.

Proposition 1.15 (No arithmetic interface without declared realization). *No arithmetic source datum may be used in Core certification unless a purpose-faithful arithmetic realization hypothesis and realization target are declared.*

Proof. The Core operates on constructible persistence objects, admissible representatives, eligible realized objects, tower diagnostics, ledgers, and certificate records. An arithmetic object without declared realization has no typed input to the Core. Therefore it cannot be used for Core certification. $\square$

MS-A.3bis. Purpose-faithfulness burden

Definition 1.16 (Purpose-faithfulness burden). The *purpose-faithfulness burden* of an arithmetic realization is the obligation to specify and support:

- (i) the arithmetic feature intended to be preserved;
- (ii) the Core-visible feature intended to represent it;
- (iii) the direction of preservation;
- (iv) the hypotheses under which preservation is asserted;

- (v) the artifacts or proof references supporting the preservation claim;
- (vi) the failure mode when the preservation claim is unavailable.

Definition 1.17 (Purpose-faithfulness status). A purpose-faithfulness claim has exactly one of the following statuses:

proved, conditional, operational, [Spec], rejected, undefined.

Only proved or fully discharged conditional purpose-faithfulness may support an arithmetic transport theorem.

Proposition 1.18 (Declared purpose is not purpose-faithfulness). *Declaring an intended arithmetic purpose does not establish purpose-faithfulness.*

Proof. A purpose declaration identifies what the realization is supposed to preserve. Purpose-faithfulness additionally requires evidence that the declared arithmetic feature is actually represented by the declared Core-visible feature under the stated hypotheses. The declaration alone does not provide that evidence. $\square$

Proposition 1.19 (Unproved purpose-faithfulness blocks arithmetic transport). *If the purpose-faithfulness status of an arithmetic realization is [Spec], undefined, or rejected, then no arithmetic transport theorem may be invoked through that realization.*

Proof. An arithmetic transport theorem requires a proof that the arithmetic and Core-side conditions correspond. Without proved or discharged conditional purpose-faithfulness, the correspondence is missing. Therefore the transport remains a specification target or candidate, not theorem evidence. $\square$

Remark 1.20 (Purpose-faithfulness is feature-relative). A realization may be purpose-faithful for one feature and non-faithful for another. For example, a realization may preserve a finite-layer growth profile while failing to preserve a p -adic analytic identity, Selmer-rank statement, or characteristic-ideal relation.

MS-A.4. Core-readable arithmetic objects

Definition 1.21 (Core-readable arithmetic object). An arithmetic source datum A_{arith} is *Core-readable* if the interface supplies an object

$$F_{\text{arith}} \in \text{FiltCh}((k))$$

or

$$P_{\text{arith}} \in \text{Pers}_k^{\text{cons}}$$

such that, for every declared window W , threshold τ , and monitored degree i , the repaired evaluation

$$\text{Eval}_{i,W,\tau}(F_{\text{arith}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F_{\text{arith}}))$$

or the corresponding direct persistence expression

$$T_{\tau}^{\text{bar}}(W_W P_{\text{arith}})$$

is defined in the constructible persistence regime.

Definition 1.22 (Arithmetic certificate input). An *arithmetic certificate input* is a Core-readable arithmetic object together with:

- (i) declared windows;

- (ii) a threshold policy;
- (iii) a collapse policy;
- (iv) monitored degrees;
- (v) repaired evaluation records;
- (vi) any required admissible representative data;
- (vii) any required eligible realization data;
- (viii) any required tower comparison artifacts;
- (ix) ledger entries for arithmetic transfer, extraction, approximation, truncation, or comparison defects;
- (x) artifact references.

Definition 1.23 (Arithmetic Core-readability failure). An arithmetic source datum has *Core-readability failure* if the interface cannot produce constructible repaired evaluation records, admissible representatives when required, eligible realization data when required, or tower comparison artifacts when Type IV diagnostics are invoked.

Remark 1.24 (Constructibility requirement). The persistence object produced by an arithmetic interface must lie in the constructible one-parameter regime of Appendix A. If an arithmetic realization produces wild, nonconstructible, infinite-rank, or uncontrolled persistence data, it is not a Core-readable arithmetic object.

Remark 1.25 (No raw arithmetic-to-Eval shortcut). An arithmetic invariant may motivate the expected shape of

$$\text{Eval}_{i,W,\tau}(F_{\text{arith}}),$$

but it does not define that profile unless a declared realization route produces the corresponding constructible persistence data.

MS-A.4bis. One-parameter extraction discipline

Definition 1.26 (Arithmetic one-parameter extraction). An *arithmetic one-parameter extraction* is a declared rule that converts arithmetic data with possibly several natural parameters into a single ordered persistence parameter compatible with $\text{Pers}_k^{\text{cons}}$. It must declare:

- (i) the source parameters;
- (ii) the selected persistence parameter;
- (iii) the order convention;
- (iv) the normalization;
- (v) the treatment of ties and incomparable data;
- (vi) the information intentionally discarded;
- (vii) any defect introduced by the extraction.

Definition 1.27 (Extraction defect). An *extraction defect* is a declared discrepancy introduced by replacing multi-parameter, tower-indexed, valuation-indexed, level-indexed, or analytically parameterized arithmetic data by one-parameter persistence data. It is denoted

$$\delta^{\text{arith-extract}}.$$

Proposition 1.28 (No multi-parameter shortcut). *Multi-parameter arithmetic data are not Core-readable merely because each parameter is mathematically meaningful.*

Proof. The repaired Core persistence regime is constructible one-parameter persistence over a field. A multi-parameter arithmetic object supplies no object of $\text{Pers}_k^{\text{cons}}$ until a declared one-parameter extraction or reduction has been provided. Any loss or discrepancy introduced by that extraction must be recorded as a defect if it is consumed by the certificate. $\square$

Remark 1.29 (Common arithmetic extraction risks). Typical extraction risks include collapsing conductor and level into one scale, replacing a two-variable p -adic L -function by a branch, selecting one direction in a cyclotomic or anticyclotomic tower, or projecting congruence depth and valuation depth onto a single filtration. These choices are not neutral unless proved neutral for the declared purpose.

MS-A.5. Arithmetic pre-gates

Definition 1.30 (Arithmetic pre-gate). An *arithmetic pre-gate* is a domain-specific screening or preparation condition applied before Core certification. Examples include:

- (i) conductor bounds;
- (ii) good, ordinary, semistable, or supersingular case distinctions;
- (iii) local condition checks;
- (iv) rank, torsion, or reduction-type prechecks;
- (v) congruence compatibility checks;
- (vi) Selmer control checks;
- (vii) Iwasawa tower admissibility checks;
- (viii) modularity or automorphic availability assumptions;
- (ix) p -adic interpolation or branch-selection conditions;
- (x) truncation or finite-layer adequacy checks.

Definition 1.31 (Arithmetic pre-gate record). An *arithmetic pre-gate record* records:

- (i) the arithmetic condition being checked;
- (ii) the source datum to which it applies;
- (iii) pass/fail/not-applicable status;
- (iv) whether the pre-gate is required for the declared realization route;
- (v) artifact references supporting the status.

Proposition 1.32 (Arithmetic pre-gates do not replace Core gates). *An arithmetic pre-gate may determine which arithmetic objects are submitted to the Core, but it does not replace B-Gate⁺, monitored Type IV diagnostics, declared tower comparison artifacts, or the budget condition.*

Proof. By Chapter 8, pre-gates are external screening devices. Core verdicts are determined only by Core gate conditions, monitored diagnostics, declared artifacts, and the budget comparison. Thus arithmetic pre-gates cannot replace Core gates. $\square$

Remark 1.33 (Pre-gate failure vs. Core reject). Failure of an arithmetic pre-gate means that the object is not suitable for the declared interface route. It is not automatically a Core reject. Conversely, passing all arithmetic pre-gates does not imply Core acceptance.

MS-A.6. Eligible regime for arithmetic Ext¹ use

Definition 1.34 (Arithmetic eligible regime). An arithmetic interface is in the *arithmetic eligible regime* on $(F; W, \tau)$ if:

(i) the arithmetic realization produces an admissible representative

$$C_{\tau, W}F;$$

(ii) for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau, W}F) \cong \text{Eval}_{i, W, \tau}(F);$$

(iii) the realization functor

$$\mathcal{R}(C_{\tau, W}F)$$

is defined in $D^b(k\text{-mod})$;

(iv) one has

$$\mathcal{R}(C_{\tau, W}F) \in D^{[-1, 0]}(k\text{-mod});$$

(v) the edge-realization condition of Appendix C holds:

$$H^{-1}(\mathcal{R}(C_{\tau, W}F)) \cong H_1^{\tau, W}(F).$$

Proposition 1.35 (No arithmetic Ext¹-discharge without eligibility). *An arithmetic interface may invoke the one-way bridge*

$$\text{Eval}_{1, W, \tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0$$

only in the arithmetic eligible regime.

Proof. The one-way bridge is proved in Chapter 4 and Appendix C under the eligible regime. Arithmetic origin does not weaken those hypotheses. Therefore the arithmetic interface must satisfy eligibility before invoking the Ext¹-clause. $\square$

Remark 1.36 (Arithmetic Ext is not automatically Core Ext). Arithmetic Ext-groups, Selmer Ext-groups, deformation Ext-groups, Galois-cohomological Ext-groups, or motivic Ext-groups are not automatically the Core group

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k).$$

Any identification must be declared as an interface hypothesis or proved as an interface theorem.

MS-A.7. Arithmetic transport statements

Definition 1.37 (Arithmetic transport statement). An *arithmetic transport statement* is a statement of either form:

arithmetic property $\implies$ Core-visible persistence, representative, diagnostic, or certificate behavior,

or

Core certificate behavior $\implies$ arithmetic conclusion.

Definition 1.38 (Arithmetic transport theorem). An *arithmetic transport theorem* is an arithmetic transport statement supplied with:

- (i) a precise arithmetic source category;
- (ii) a purpose-faithful arithmetic realization hypothesis;
- (iii) a Core-readable realization target;
- (iv) repaired evaluation records;
- (v) admissible representative data when invoked;
- (vi) all eligibility conditions, if Ext^1 is used;
- (vii) all tower comparison artifacts, if Type IV diagnostics are used;
- (viii) all ledgered transfer, truncation, or comparison defects;
- (ix) a proof that the stated arithmetic and Core-side conditions correspond.

Definition 1.39 (Default status of arithmetic transport). Every arithmetic transport statement is classified as [Spec] by default unless it is separately promoted and proved under the interface protocol.

Proposition 1.40 (Arithmetic transport does not enlarge the Core). *An arithmetic transport theorem, even when proved, does not enlarge the Core mathematical package unless explicitly imported and reproved under the Core guard-rails.*

Proof. Arithmetic transport uses external arithmetic semantics. By Chapter 7 and Chapter 8, problem-specific interface material remains outside the Core unless explicitly imported and reproved. Therefore arithmetic transport does not enlarge the Core by default. $\square$

Remark 1.41 (Two directions must be separated). The directions

arithmetic property $\implies$ Core behavior

and

Core behavior $\implies$ arithmetic conclusion

are logically distinct. The first may make an arithmetic object Core-readable. The second may be needed to derive an arithmetic theorem from a Core certificate. Neither direction is automatic.

MS-A.7bis. Arithmetic comparison modes

Definition 1.42 (Arithmetic comparison). An *arithmetic comparison* is a comparison between arithmetic source data, arithmetic realizations, finite layers, tower limits, congruent objects, Selmer-type objects, p -adic profiles, or their Core-readable images.

Definition 1.43 (Arithmetic comparison mode). Every arithmetic comparison consumed by an interface record must declare exactly one local mode:

arithmetic_exact, realization_exact, metric_controlled, advisory_only, unavailable, failed.

Its projection to the Core comparison modes is governed as follows:

- (i) realization_exact projects to Core mode exact only when an actual equality or persistence-profile isomorphism of the declared Core-readable profiles is supplied;
- (ii) metric_controlled projects to Core mode metric_gap_certified only after a certified bottleneck bound, scalarization route, and two-sided threshold-gap record are supplied;
- (iii) arithmetic_exact remains external unless it is transported to a Core-readable exact comparison;
- (iv) advisory_only and unavailable project to not_compared;
- (v) failed blocks the invoked comparison route.

Proposition 1.44 (Arithmetic equality is not Core exactness). *An arithmetic equality, isomorphism, congruence, control statement, or equality of invariants is not a Core exact comparison unless it produces an actual equality or persistence-profile isomorphism between the declared Core-readable profiles.*

Proof. Core exact comparison is a statement about the declared Core-readable objects and profiles. Arithmetic equality is a statement in the source domain. A transport or realization theorem is required to identify the two comparison types. $\square$

Proposition 1.45 (Arithmetic closeness is not metric-gap certification). *An arithmetic bound, congruence depth, valuation estimate, finite-layer approximation, or numerical closeness statement is not a metric-gap-certified comparison unless it supplies a certified bottleneck upper bound on the actual windowed pre-threshold profiles and satisfies the two-sided threshold-gap condition.*

Proof. Metric-gap certification consumes a bottleneck discrepancy bound for the profiles to which hard threshold deletion is applied, together with a threshold-clearance certificate. Arithmetic closeness does not have this type by default. It must be converted into the required metric record. $\square$

MS-A.8. Arithmetic defects and ledger entries

Definition 1.46 (Arithmetic transfer defect). An *arithmetic transfer defect* is a declared discrepancy arising when arithmetic data are transferred into a Core-readable persistence or filtered object. It is denoted

$$\delta^{\text{arith}}.$$

Definition 1.47 (Arithmetic comparison defect). An *arithmetic comparison defect* is a declared discrepancy arising when comparing two arithmetic realizations, two arithmetic towers, or two arithmetic-to-Core interface routes. It is denoted

$$\delta^{\text{arith-comp}}.$$

Definition 1.48 (Arithmetic truncation defect). An *arithmetic truncation defect* is a declared discrepancy arising from replacing an infinite arithmetic object, tower, series, module, or p -adic analytic object by finite-layer, finite-precision, finite-rank, or finite-window data. It is denoted

$$\delta^{\text{arith-trunc}}.$$

Remark 1.49 (Arithmetic defect namespace and ledger-catalog registration). The symbols

$$\delta^{\text{arith}}, \quad \delta^{\text{arith-comp}}, \quad \delta^{\text{arith-trunc}}, \quad \delta^{\text{arith-extract}}$$

are interface-layer defect namespaces. They enter Core audit only through a declared metric-ledger slot and scalarization route compatible with the global ledger catalog. If the global ledger catalog has not yet assigned a dedicated arithmetic-interface slot, the interface record must declare the slot locally and mark global-catalog harmonization as a governance dependency rather than silently aggregating the defect.

Definition 1.50 (Arithmetic ledger entry). An *arithmetic ledger entry* records:

- (i) the arithmetic source datum;
- (ii) the realization route;
- (iii) the defect type;
- (iv) the defect value or bound;
- (v) the aggregation rule;
- (vi) how the defect enters $\Sigma\delta$;
- (vii) artifact references.

Proposition 1.51 (Arithmetic defects affect Core only through ledger). *Arithmetic transfer, extraction, truncation, or comparison defects affect Core audit only if they are included in the declared δ -ledger.*

Proof. Core audit depends on gate verdicts, monitored diagnostics, declared artifacts, and the budget comparison. Arithmetic defects are not gate verdicts or monitored diagnostics. They affect Core audit only by entering the total budget $\Sigma\delta$. $\square$

Remark 1.52 (Arithmetic elegance is not zero defect). A natural arithmetic construction may appear canonical or elegant. That does not mean its transfer defect is zero. Zero defect, controlled defect, or unavailable comparison must be explicitly declared.

MS-A.8bis. Finite approximation, truncation, and numerical evidence

Definition 1.53 (Arithmetic finite approximation). An *arithmetic finite approximation* is a finite-layer, finite-precision, finite-rank, finite-window, sampled, truncated, or computational replacement for an arithmetic object, tower, series, p -adic analytic object, Selmer complex, or L -profile.

Definition 1.54 (Approximation status). An arithmetic finite approximation has exactly one of the following statuses:

sample, calibration, bounded_approximation, certified_approximation, failed, undefined.

Only a certified_approximation with a declared defect bound may enter the metric ledger.

Proposition 1.55 (Finite evidence is not infinite arithmetic theorem). *Finite-layer, finite-precision, sampled, or numerical arithmetic evidence does not prove an infinite tower statement, p -adic analytic identity, Selmer finiteness theorem, or arithmetic transport theorem unless a theorem reduces the claimed infinite statement to the certified finite data.*

Proof. Finite evidence has finite scope. Infinite arithmetic conclusions quantify beyond that scope. A reduction theorem or certified approximation theorem is required to bridge the finite and infinite statements. Without such a theorem, the evidence remains calibration, advisory, or bounded approximation data. $\square$

Proposition 1.56 (Unledged approximation is non-verdict). *If an arithmetic approximation is used to support a Core-readable comparison or evaluation and its defect is nonzero, unknown, or uncontrolled, then the approximation must be ledgered or the affected verifier-side obligation remains undefined.*

Proof. Approximation may change the Core-readable profile, comparison, or diagnostic. Since the Core budget requires declared scalarized discrepancy bounds, an uncontrolled or unledged approximation supplies no admissible verifier-side evidence. $\square$

Remark 1.57 (Numerical arithmetic artifacts). Numerical computations of ranks, regulators, local factors, p -adic valuations, finite-layer Selmer groups, or L -values may support calibration or conjecture generation. They do not become theorem evidence unless their verification environment, exactness, error bounds, and transport role are declared.

MS-A.9. Arithmetic towers and monitored Type IV

Definition 1.58 (Arithmetic tower). An *arithmetic tower* is a directed system

$$A_0 \longrightarrow A_1 \longrightarrow A_2 \longrightarrow \cdots$$

of arithmetic source data, such as cyclotomic layers, anticyclotomic layers, Iwasawa towers, congruence towers, level towers, finite-layer Selmer towers, or local-condition towers.

Definition 1.59 (Core-readable arithmetic tower). An arithmetic tower is *Core-readable* if its realization yields a directed tower

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

in the filtered or persistence regime, together with a declared apex object F_∞ , so that a tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

can be declared in the sense of Appendix D.

Definition 1.60 (Arithmetic tower comparison artifact). An *arithmetic tower comparison artifact* is a tower comparison artifact

$$\phi_{i,W,\tau}^{\text{arith}} : \text{ColimProfile}_{i,W,\tau}(F_\bullet^{\text{arith}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty^{\text{arith}})$$

whose tower-side and apex-side profiles are produced by a declared arithmetic realization route.

Definition 1.61 (Arithmetic Type IV proxy). An *arithmetic Type IV proxy* is an arithmetic signal, such as failure of control, anomalous growth, defect in congruence propagation, unexpected limit behavior, nontrivial Iwasawa growth, or unstable finite-layer-to-limit comparison, intended to suggest possible monitored Type IV obstruction.

Proposition 1.62 (Arithmetic Type IV proxy is not Type IV diagnosis). *An arithmetic Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \neq (0, 0).$$

Proof. Monitored Type IV diagnostics are defined by the window-relative terminal-generic dimensions tgdim_W of the kernel and cokernel defect profiles of the declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

An arithmetic proxy may suggest related behavior, but it is not the diagnostic unless it computes or is proved equivalent to those window-relative terminal-generic defect profiles under the declared realization. $\square$

Remark 1.63 (Arithmetic μ versus Collapse μ). Any arithmetic invariant named μ , including Iwasawa-type μ -invariants, is not automatically the same as

$$\mu_{\text{Collapse}}.$$

The latter is a monitored generic kernel-defect dimension defined in Appendix D. Identifying the two requires a separate arithmetic transport theorem.

MS-A.9bis. Arithmetic invariant namespace separation

Definition 1.64 (Arithmetic invariant namespace). An arithmetic invariant namespace records the semantic origin of an invariant before any attempted Core transport. Typical namespaces include:

$$\mu_{\text{Iw}}, \quad \lambda_{\text{Iw}}, \quad r_{\text{Sel}}, \quad \text{ord}_{s=s_0} L, \quad v_p, \quad \text{rank}, \quad \text{char}_\Lambda.$$

These are not Core diagnostic symbols unless a transport theorem identifies them with Core-readable records under declared hypotheses.

Proposition 1.65 (Namespace collision is invalid). *A certificate or interface record is invalid if it uses the same undecorated symbol for an arithmetic invariant and a Collapse diagnostic in a way that affects a verifier-side conclusion.*

Proof. Arithmetic invariants and Collapse diagnostics have different source types, definitions, and verifier-side roles. Using one undecorated symbol for both destroys type reconstruction and may falsely promote external arithmetic information into Core diagnostic evidence. $\square$

Proposition 1.66 (Selmer rank, analytic order, and Iwasawa growth are not residual PH or Type IV by default). *Selmer rank, analytic order of vanishing, Iwasawa λ -growth, Iwasawa μ -growth, valuation growth, or characteristic-ideal behavior is not automatically residual persistent homology, an Ext^1 -edge obstruction, or monitored Type IV defect.*

Proof. Residual persistent homology, the Core Ext^1 -edge, and monitored Type IV diagnostics are defined through repaired persistence profiles, eligible realized objects, and tower comparison artifacts. The listed arithmetic invariants live in external arithmetic namespaces. Any identification requires a separate arithmetic transport theorem. $\square$

Remark 1.67 (μ -collision warning). When μ_{Iw} and μ_{Collapse} both appear, the subscript is mandatory. A statement of the form $\mu = 0$ is inadmissible unless the namespace is uniquely declared.

MS-A.10. Selmer and Iwasawa interface placeholders

This section introduces only meta-theoretic placeholders. It does not prove any Selmer, Iwasawa, or p -adic L -function theorem.

Definition 1.68 (Selmer interface datum). A *Selmer interface datum* is an arithmetic source datum involving Selmer groups, Selmer complexes, local conditions, or Pontryagin dual Selmer modules, together with a proposed realization route into Core-readable filtered or persistence data.

Definition 1.69 (Iwasawa interface datum). An *Iwasawa interface datum* is an arithmetic source datum involving an infinite arithmetic tower, Iwasawa module, characteristic ideal, Iwasawa invariant, control theorem, or finite-layer-to-limit comparison, together with a proposed realization route into a Core-readable arithmetic tower.

Definition 1.70 (p -adic L -profile interface datum). A *p -adic L -profile interface datum* is an arithmetic source datum involving a p -adic L -function, p -adic valuation sequence, Newton polygon, Weierstrass preparation profile, zero structure, or special-value profile, together with a proposed realization route into constructible persistence or filtered data.

Remark 1.71 (Placeholders only). The definitions in this section are placeholders for later interface appendices. They do not assert that Selmer groups, Iwasawa modules, or p -adic L -functions have already been faithfully realized in the AK Core. They only specify the admissible types of data that later appendices may attempt to realize.

Proposition 1.72 (Selmer/Iwasawa/ p -adic L placeholders do not prove transport). *The existence of a Selmer, Iwasawa, or p -adic L -profile interface datum does not by itself prove an arithmetic transport theorem.*

Proof. A placeholder identifies source data and an intended realization direction. A transport theorem additionally requires a purpose-faithful realization, repaired evaluation records, eligibility data if invoked, tower artifacts if invoked, ledgered defects, and a proof of correspondence. Therefore the placeholder alone is not a transport theorem. $\square$

MS-A.11. Arithmetic certificate records

Definition 1.73 (Arithmetic interface manifest entry). An *arithmetic interface manifest entry* records:

- (i) the arithmetic source datum;
- (ii) the arithmetic source category or family;
- (iii) the purpose-faithful realization hypothesis;
- (iv) the purpose profile;
- (v) the realization target;
- (vi) declared windows and thresholds;
- (vii) the collapse policy;
- (viii) monitored degrees;
- (ix) repaired evaluation records;

- (x) admissible representative data, if invoked;
- (xi) eligibility data, if Ext^1 is invoked;
- (xii) tower comparison artifacts, if Type IV diagnostics are invoked;
- (xiii) arithmetic transfer, extraction, approximation, truncation, or comparison defects;
- (xiv) ledger entries;
- (xv) Core gate verdicts;
- (xvi) monitored Type IV diagnostics;
- (xvii) artifact references;
- (xviii) the status of any arithmetic transport statement.

Definition 1.74 (Arithmetic certificate record). An *arithmetic certificate record* is a certificate record whose source data are arithmetic and whose Core-readable data are obtained through a declared arithmetic interface manifest entry.

Proposition 1.75 (Arithmetic certificate records must satisfy Minimal Manifest). An *arithmetic certificate record* is admissible only if it satisfies the Minimal Manifest Axiom of Appendix G.

Proof. Arithmetic source data do not exempt a certificate from Core auditability. The Minimal Manifest Axiom requires declared objects, windows, threshold and collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledger policy, diagnostics, tower artifacts when invoked, gate verdicts, artifact references, and final status. Therefore arithmetic certificate records must satisfy the same manifest requirements. $\square$

Remark 1.76 (Arithmetic artifacts). Arithmetic artifact references may include source equations, Galois data, Selmer complexes, tower layers, congruence data, computational algebra outputs, realization scripts, formal proof traces, or literature references. They support reconstruction; they are not themselves Core verdicts.

MS-A.12. Arithmetic pass, Core pass, and arithmetic conclusion

Definition 1.77 (Arithmetic interface pass). An *arithmetic interface pass* means that the arithmetic data have been successfully realized into Core-readable form under the declared interface protocol.

Definition 1.78 (Core pass for arithmetic data). A *Core pass for arithmetic data* means that the corresponding Core certificate record passes B-Gate⁺, monitored Type IV diagnostics, required artifact checks, and the budget condition.

Definition 1.79 (Arithmetic conclusion). An *arithmetic conclusion* is a problem-specific statement about the original arithmetic source datum, such as a rank formula, vanishing statement, finiteness statement, control theorem, main conjecture implication, special-value identity, characteristic-ideal relation, or L -function statement.

Proposition 1.80 (Core pass is not arithmetic conclusion). A *Core pass for arithmetic data* does not imply an *arithmetic conclusion* unless a separate arithmetic transport theorem proves that implication.

Proof. A Core pass certifies Core-visible persistence, diagnostic, artifact, and budget conditions. An arithmetic conclusion concerns external arithmetic semantics. A theorem connecting the two is required. Without such a theorem, the implication is not valid. $\square$

Remark 1.81 (Three-layer distinction). The following must remain distinct:

arithmetic interface pass, Core pass, arithmetic theorem.

Confusing these layers would violate the non-claim discipline of Chapter 7.

MS-A.12bis. Core-to-arithmetic theorem promotion schema

Definition 1.82 (Core-to-arithmetic transport theorem). A *Core-to-arithmetic transport theorem* is a theorem of the form

Core certificate behavior $\implies$ arithmetic conclusion

together with:

- (i) the exact arithmetic source category;
- (ii) the realized Core-readable object or tower;
- (iii) the arithmetic conclusion;
- (iv) the Core certificate clauses consumed;
- (v) a proof that the consumed Core clauses imply the stated arithmetic conclusion;
- (vi) all non-scalar hypotheses;
- (vii) all ledgered defects;
- (viii) all artifact references needed for independent reconstruction.

Definition 1.83 (Arithmetic theorem promotion). *Arithmetic theorem promotion* is the operation of changing a problem-specific arithmetic implication from [Spec] or bridge-candidate status to theorem status. Promotion requires a proved Core-to-arithmetic transport theorem or arithmetic-to-Core-and-back bridge theorem, together with updated claim-register status.

Proposition 1.84 (No promotion by successful certification alone). *A successful Core certificate for arithmetic data does not promote any arithmetic conclusion to theorem status by itself.*

Proof. A Core certificate concerns Core-visible persistence, representative, diagnostic, artifact, and budget conditions. An arithmetic theorem concerns the original arithmetic semantics. The implication between them is an additional theorem obligation and is not supplied by certification alone. $\square$

Proposition 1.85 (Transport hypotheses are non-scalar). *Missing hypotheses of a Core-to-arithmetic transport theorem cannot be repaired by metric slack, favorable advisory data, or successful arithmetic pre-gates.*

Proof. Transport hypotheses determine whether the Core-side statement has the intended arithmetic meaning. They are logical and semantic obligations, not numerical perturbation components. Therefore they cannot be discharged by ledger slack or advisory indicators. $\square$

MS-A.13. Summary of arithmetic interface meta-theory

Theorem 1.86 (Appendix MS-A arithmetic interface package). *Within the Problem Interface layer, the following package is available.*

- (i) *Arithmetic source data may be connected to the AK Core only through a declared purpose-faithful arithmetic realization hypothesis.*
- (ii) *Arithmetic interface sovereignty forbids arithmetic meaning, arithmetic theorems, computations, or literature artifacts from overriding verifier-side Core requirements.*
- (iii) *The realization target and purpose profile must be declared.*
- (iv) *Purpose-faithfulness carries an explicit burden of proof and is feature-relative.*
- (v) *Arithmetic data become Core-readable only when they produce constructible persistence or filtered data compatible with the repaired evaluation*

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

- (vi) *Multi-parameter arithmetic data require a declared one-parameter extraction before entering the constructible persistence regime.*
- (vii) *Arithmetic pre-gates may screen objects, but they do not replace Core gates.*
- (viii) *The Ext^1 -clause may be invoked only in the arithmetic eligible regime.*
- (ix) *Arithmetic transport statements are [Spec] by default.*
- (x) *Arithmetic exactness, congruence, control, or closeness is not Core exactness or metric-gap certification without a declared Core-readable comparison record.*
- (xi) *Arithmetic transfer, extraction, approximation, truncation, and comparison defects affect Core audit only through the declared δ -ledger.*
- (xii) *Finite-layer, finite-precision, sampled, or numerical evidence does not prove an infinite arithmetic statement unless a reduction theorem is supplied.*
- (xiii) *Arithmetic towers become Type IV-readable only through declared tower comparison artifacts*

$$\phi_{i,W,\tau}.$$

- (xiv) *Arithmetic Type IV proxies do not replace monitored Type IV diagnostics.*
- (xv) *Arithmetic invariant namespaces must remain distinct from Collapse diagnostic namespaces.*
- (xvi) *Arithmetic invariants named μ are not automatically equal to μ_{Collapse} .*
- (xvii) *Selmer, Iwasawa, and p -adic L -profile interface data are placeholders until realized and transported by later interface theorems.*
- (xviii) *Arithmetic certificate records must satisfy the Minimal Manifest Axiom.*
- (xix) *A Core pass for arithmetic data does not imply an arithmetic theorem without a separate arithmetic transport theorem.*

(xx) *Arithmetic theorem promotion requires a separate Core-to-arithmetic transport theorem and cannot be achieved by successful Core certification alone.*

Proof. Items concerning arithmetic realization, realization targets, purpose profiles, Core-readable arithmetic objects, pre-gates, eligible Ext^1 -use, default [Spec] status, defects, towers, Type IV proxies, placeholders, manifest records, and the separation between Core pass and arithmetic conclusion follow from Definitions 1.11, 1.12, 1.13, 1.21, 1.30, 1.34, 1.39, 1.50, 1.59, 1.60, 1.61, 1.68, 1.69, 1.70, 1.74, and 1.79, together with Propositions 1.15, 1.32, 1.35, 1.40, 1.51, 1.62, 1.72, 1.75, and 1.80.

The additional sovereignty, purpose-faithfulness, one-parameter extraction, comparison-mode, finite-approximation, namespace, and promotion clauses follow respectively from Definitions 1.4, 1.16, 1.17, 1.26, 1.43, 1.53, 1.64, 1.82, and 1.83, together with their associated propositions. $\square$

MS-A.14. Explicit exclusions

The following are not asserted in Appendix MS-A.

- No arithmetic theorem is proved.
- No Birch–Swinnerton-Dyer statement is proved.
- No ABC statement is proved.
- No Riemann hypothesis statement is proved.
- No Iwasawa main conjecture statement is proved.
- No modularity theorem is proved.
- No Selmer finiteness, rank formula, or regulator formula is proved.
- No p -adic L -function identity is proved.
- No arithmetic realization is assumed to be faithful without declaration.
- No arithmetic pre-gate replaces B-Gate⁺.
- No arithmetic invariant replaces the repaired evaluation condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No arithmetic Ext-group is automatically identified with the Core Ext^1 -clause.
- No arithmetic μ -invariant is automatically identified with

$$\mu_{\text{Collapse}}.$$

- No arithmetic Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No arithmetic Type IV proxy replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No Iwasawa tower is automatically a Core-readable tower.
- No Selmer group or Selmer complex is automatically a Core-readable object.
- No p -adic L -function is automatically a Core-readable persistence profile.
- No Core pass is interpreted as an arithmetic theorem without a separate transport theorem.
- No arithmetic transport statement is Core theorem by default.
- No Search, advisory, computational, or literature arithmetic artifact is promoted to proof status by itself.
- No arithmetic equality, congruence, control theorem, or invariant equality is automatically a Core exact comparison.
- No arithmetic estimate, valuation bound, congruence depth, or finite-layer approximation is automatically a metric-gap-certified comparison.
- No multi-parameter arithmetic object is automatically a one-parameter constructible persistence object.
- No one-parameter extraction is assumed lossless without proof.
- No finite-layer computation proves an infinite tower theorem without a reduction theorem.
- No finite-precision p -adic computation proves a p -adic L -function identity.
- No Selmer rank, analytic order of vanishing, Iwasawa λ -invariant, valuation profile, or characteristic-ideal relation is automatically identified with residual PH_1 , the Core Ext^1 -edge, or monitored Type IV diagnostics.
- No successful Core certificate promotes an arithmetic implication from [Spec] to theorem status without a registered Core-to-arithmetic transport theorem.
- No metric ledger slack repairs a missing purpose-faithfulness theorem, missing transport theorem, missing one-parameter extraction, missing eligibility record, or missing tower comparison artifact.
- No advisory, calibration, numerical, AI-generated, or literature-derived arithmetic record changes the verifier-side status unless it re-enters through a declared Core-readable record.

MS-A.15. Provenance and status

The material in this appendix depends on:

- (i) the Core claim taxonomy and Minimal Manifest discipline of Chapter 1;
- (ii) the repaired constructible persistence framework of Chapter 2 and Appendix A;
- (iii) the admissible representative discipline of Appendix B;
- (iv) the eligible realization and one-way bridge of Chapter 4 and Appendix C;
- (v) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Chapter 5 and Appendix D;
- (vi) the audit and Core sovereignty discipline of Chapter 6;

- (vii) the non-claim discipline of Chapter 7;
- (viii) the interface protocol of Chapter 8;
- (ix) the manifest schema of Appendix G;
- (x) the ledger catalog of Appendix M.

Its role is to define how arithmetic data may enter the AK framework without allowing arithmetic meaning to silently enlarge the Core.

Remark 1.87 (Canonical Type IV notation in MS-A). Throughout Appendix MS-A the cokernel-side Collapse diagnostic is denoted by u_{Collapse} . No Latin-letter cokernel diagnostic is introduced or revived.

Status labels for Appendix MS-A. *Problem-interface definitions:* Definitions [1.7](#), [1.8](#), [1.9](#), [1.11](#), [1.12](#), [1.13](#), [1.21](#), [1.22](#), [1.23](#), [1.30](#), [1.31](#), [1.34](#), [1.37](#), [1.38](#), [1.39](#), [1.46](#), [1.47](#), [1.48](#), [1.50](#), [1.58](#), [1.59](#), [1.60](#), [1.61](#), [1.68](#), [1.69](#), [1.70](#), [1.73](#), [1.74](#), [1.77](#), [1.78](#), [1.79](#).

Problem-interface propositions/theorems: Propositions [1.15](#), [1.32](#), [1.35](#), [1.40](#), [1.51](#), [1.62](#), [1.72](#), [1.75](#), [1.80](#), Theorem [1.86](#).

Operational cautions: Remarks [1.1](#), [1.2](#), [1.3](#), [1.10](#), [1.14](#), [1.24](#), [1.25](#), [1.33](#), [1.36](#), [1.41](#), [1.52](#), [1.63](#), [1.71](#), [1.76](#), [1.81](#).

Additional v19.0 hardening remarks: Remarks [1.6](#), [1.20](#), [1.29](#), [1.57](#), [1.49](#), [1.67](#), [1.87](#).

Additional v19.0 hardening definitions: Definitions [1.4](#), [1.16](#), [1.17](#), [1.26](#), [1.27](#), [1.42](#), [1.43](#), [1.53](#), [1.54](#), [1.64](#), [1.82](#), [1.83](#).

Additional v19.0 hardening propositions: Propositions [1.5](#), [1.18](#), [1.19](#), [1.28](#), [1.44](#), [1.45](#), [1.55](#), [1.56](#), [1.65](#), [1.66](#), [1.84](#), [1.85](#).

Explicit exclusions: Section MS-A.14.

2 Appendix MS-B: Cyclotomic / Iwasawa / Congruence Interfaces

MS-B.1. Scope and interface status

This appendix specializes the arithmetic interface meta-theory of Appendix MS-A to cyclotomic towers, Iwasawa-type structures, and congruence interfaces. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Cyclotomic, Iwasawa, and congruence data are interface inputs, not Core theorems.

In particular, this appendix does not prove any Iwasawa main conjecture, control theorem, Selmer-rank formula, congruence theorem, modularity theorem, automorphy theorem, p -adic L -function identity, or arithmetic finiteness theorem.

All Core-relevant persistence data produced by the interfaces in this appendix must be evaluated through the repaired windowed profile:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Whenever tower diagnostics are invoked, the required comparison is not an automatic colimit map. It must be a declared tower comparison artifact:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Remark 2.1 (Interface status). All arithmetic implications in this appendix are [Spec] by default unless separately promoted and proved under the interface protocol of Chapter 8 and Appendix MS-A.

Remark 2.2 (Relation to Appendix MS-A). Appendix MS-A supplies the general arithmetic interface discipline. The present appendix provides structured interface templates for:

cyclotomic towers, Iwasawa modules, congruence defects, control statements, finite-layer-to-limit comparison

These templates do not alter B-Gate⁺, monitored Type IV diagnostics, tower comparison artifact requirements, or the budget condition.

Remark 2.3 (No arithmetic tower shortcut). An arithmetic tower is not automatically a Core tower. A cyclotomic or Iwasawa tower becomes Type IV-readable only after it is realized as a constructible filtered or persistence tower, equipped with repaired evaluation records and a declared tower comparison artifact.

MS-B.1bis. v19.0 hardening inheritance

Definition 2.4 (MS-B v19 dependency baseline). The statements in Appendix MS-B are interpreted relative to:

- (i) the v19.0.0 Core guard-rails of Chapters 1–8;
- (ii) the repaired window-first persistence semantics of Appendix A;
- (iii) the arithmetic interface sovereignty, one-parameter extraction discipline, comparison-mode discipline, finite-approximation discipline, and theorem-promotion discipline of Appendix MS-A;
- (iv) the monitored Type IV tower-artifact discipline of Appendix D;
- (v) the manifest requirements of Appendix G;
- (vi) the interface-layer defect namespace requirements of the ledger catalog, when such defects are consumed.

No cyclotomic, Iwasawa, congruence, control-theoretic, or p -adic analytic statement is imported at a stronger status than these dependencies permit.

Definition 2.5 (MS-B source-side status). Cyclotomic towers, Iwasawa modules, congruence relations, arithmetic control statements, finite-layer computations, and p -adic L -profiles are source-side data until a declared realization produces Core-readable verifier-side records. Their default role is interface input, calibration input, advisory input, or [Spec] bridge material.

Proposition 2.6 (Named arithmetic structure does not strengthen Core status). *Naming a source datum cyclotomic, Iwasawa-theoretic, congruential, controlled, ordinary, modular, or p -adic analytic does not strengthen any Core claim or verifier-side status.*

Proof. The Core verdict depends on repaired evaluations, representatives and eligibility records when invoked, tower artifacts, diagnostics, ledgers, gate verdicts, and manifest data. The listed names describe arithmetic provenance and structure. They do not by themselves create any of the verifier-side records consumed by the Core. $\square$

Remark 2.7 (MS-B hardening scope). The purpose of the present appendix is to prevent arithmetic tower language from bypassing the v19 comparison, tower-artifact, stable-band, and ledger requirements. It does not attempt to prove any arithmetic source theorem.

MS-B.2. Cyclotomic tower interface

Definition 2.8 (Cyclotomic tower). A *cyclotomic tower* is a directed arithmetic system

$$K_0 \subset K_1 \subset K_2 \subset \cdots \subset K_\infty$$

where the transition maps are induced by cyclotomic, p -power, or otherwise declared arithmetic extension data.

Definition 2.9 (Cyclotomic arithmetic datum). A *cyclotomic arithmetic datum* is a family

$$\mathcal{A}_{\text{cyc}} = \{A_n\}_{n \geq 0}$$

of arithmetic objects attached to the layers K_n , such as:

- (i) class groups;
- (ii) Selmer groups;
- (iii) Galois cohomology groups;
- (iv) local condition modules;
- (v) unit groups or cyclotomic units;
- (vi) congruence modules;
- (vii) Hecke or Galois cohomological complexes;
- (viii) arithmetic profiles derived from L -data;
- (ix) finite-layer truncations of infinite arithmetic objects.

Definition 2.10 (Cyclotomic realization hypothesis). A *cyclotomic realization hypothesis* is a purpose-faithful realization

$$\mathfrak{R}_{\text{cyc}} : \{A_n\}_{n \geq 0} \longrightarrow \{F_n\}_{n \geq 0},$$

where each

$$F_n \in \text{FiltCh}(\cdot)k \quad \text{or} \quad \mathbf{P}_i(F_n) \in \text{Pers}_k^{\text{cons}},$$

and the resulting tower is Core-readable in the sense of Appendix MS-A.

Definition 2.11 (Cyclotomic Core-readable tower). A cyclotomic datum $\{A_n\}$ is *Core-readable* if its realization produces a directed tower

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

with a declared apex object F_∞ , such that for each monitored degree i , window W , and threshold τ , the repaired profiles

$$\text{Eval}_{i,W,\tau}(F_n)$$

and

$$\text{Eval}_{i,W,\tau}(F_\infty)$$

are constructible and a tower comparison artifact

$$\phi_{i,W,\tau}^{\text{cyc}} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

is declared when Type IV diagnostics are invoked.

Definition 2.12 (Cyclotomic tower defect). A *cyclotomic tower defect* is a declared discrepancy

$$\delta^{\text{cyc}}$$

arising from realizing, truncating, approximating, or comparing the cyclotomic tower inside the Core-readable tower regime.

Remark 2.13 (Cyclotomic tower does not imply Type IV tower). A cyclotomic tower is an arithmetic tower. It becomes a Type IV-diagnostic tower only after a Core-readable realization and a declared tower comparison artifact have been supplied.

Proposition 2.14 (Cyclotomic data require realization before diagnostics). *No monitored Type IV diagnostic may be computed from a cyclotomic datum until a Core-readable cyclotomic realization and a declared tower comparison artifact have been supplied.*

Proof. Monitored Type IV diagnostics are defined from window-relative terminal-generic dimensions tgdim_W of kernel and cokernel defect profiles of a declared comparison artifact

$$\phi_{i,W,\tau}.$$

A raw cyclotomic tower does not provide such an artifact or such defect profiles. A Core-readable realization and artifact declaration are therefore required. $\square$

MS-B.2bis. Tower-axis selection and apex identity

Definition 2.15 (Tower-axis selection). A *tower-axis selection* is the declaration of the single ordered parameter by which a cyclotomic or Iwasawa-type tower is read as one-parameter persistence data. It records:

- (i) the arithmetic index set;
- (ii) the chosen persistence direction;
- (iii) the normalization of the layer parameter;
- (iv) the treatment of branch choices, incomparable refinements, and skipped layers;
- (v) the information discarded by the projection to one parameter;
- (vi) the extraction or truncation defect introduced by the choice.

Definition 2.16 (Realized tower morphism record). A *realized tower morphism record* records the actual morphisms

$$F_n \longrightarrow F_{n+1}$$

in $\text{FiltCh}(()k)$ or $\text{Pers}_k^{\text{cons}}$ induced by the arithmetic tower under the declared realization. Object-level layer data without such morphisms are not sufficient for invoked Type IV diagnostics.

Definition 2.17 (Apex identity record). An *apex identity record* specifies whether F_∞ is:

- (i) a genuine realized limiting object;
- (ii) a completed object;
- (iii) a formally declared apex;

- (iv) a finite-stage proxy;
- (v) an advisory or [Spec] placeholder.

Only the first three may support an invoked tower comparison artifact, and only after their construction and compatibility with the repaired profiles are artifact-backed.

Definition 2.18 (Apex vocabulary alignment). MS-B apex terminology is interpreted through the Appendix D tower-provenance and stable-band vocabulary as follows:

- (i) a genuine realized limiting object corresponds to a declared colimit, limit, or realized terminal object with artifact-backed transition data;
- (ii) a completed object corresponds to a completion-type apex provenance record and requires a completion artifact;
- (iii) a formally declared apex corresponds to an implementation or formal artifact and may be consumed only at the declared formal scope;
- (iv) a finite-stage proxy corresponds to stabilization or proved finite-reduction provenance and cannot be consumed as an apex without the reduction theorem;
- (v) an advisory or [Spec] placeholder is non-verdict and cannot support an invoked tower comparison artifact.

This alignment is a vocabulary map only; it does not create the required Appendix D artifact.

Proposition 2.19 (Tower index is not automatically a persistence parameter). *The arithmetic layer index of a cyclotomic or Iwasawa tower is not automatically an admissible persistence parameter.*

Proof. A persistence parameter must define a one-parameter constructible object with declared ordering and transition data. An arithmetic tower index may carry additional branches, completions, local conditions, congruence directions, or analytic parameters. A tower-axis selection and realized transition record are therefore required before the tower becomes Core-readable. $\square$

Proposition 2.20 (Apex identity is non-scalar). *A missing or ambiguous apex identity record cannot be repaired by ledger slack, finite-layer evidence, advisory stability, or a favorable arithmetic invariant.*

Proof. The apex identity determines the target of the tower comparison artifact. It is a non-scalar object and provenance obligation, not a numerical discrepancy. Without a declared apex object or admissible apex profile, the invoked tower comparison is undefined. $\square$

MS-B.3. Iwasawa-layer interface

Definition 2.21 (Iwasawa module datum). An *Iwasawa module datum* is an arithmetic module

$$M_{\text{Iw}}$$

over a declared Iwasawa algebra

$$\Lambda$$

or over a specified arithmetic tower, together with the transition, control, local condition, characteristic ideal, or cohomological data required by the chosen interface.

Definition 2.22 (Iwasawa interface). An *Iwasawa interface* is a purpose-faithful realization hypothesis

$$\mathfrak{R}_{\text{Iw}} : M_{\text{Iw}} \longrightarrow F_{\text{Iw}},$$

where F_{Iw} is a filtered or persistence object compatible with the AK Core protocol.

Definition 2.23 (Iwasawa-layer filtration). An *Iwasawa-layer filtration* is a filtration on F_{Iw} whose filtration parameter records a declared arithmetic layer variable, such as:

- (i) cyclotomic height;
- (ii) conductor depth;
- (iii) p -power level;
- (iv) local-condition depth;
- (v) congruence depth;
- (vi) valuation depth;
- (vii) finite-layer truncation depth;
- (viii) growth scale of a chosen arithmetic invariant.

Definition 2.24 (Iwasawa Core profile). An *Iwasawa Core profile* is the repaired windowed collapsed profile

$$\text{Eval}_{i,W,\tau}(F_{\text{Iw}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F_{\text{Iw}}))$$

associated to an Iwasawa interface, for declared W , τ , and i .

Definition 2.25 (Iwasawa tower realization). An *Iwasawa tower realization* is a realization of finite-layer or tower-level Iwasawa data as a Core-readable tower

$$F_0^{\text{Iw}} \longrightarrow F_1^{\text{Iw}} \longrightarrow F_2^{\text{Iw}} \longrightarrow \dots$$

with a declared apex object F_{∞}^{Iw} and, when Type IV diagnostics are invoked, a declared tower comparison artifact

$$\phi_{i,W,\tau}^{\text{Iw}} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}^{\text{Iw}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}^{\text{Iw}}).$$

Remark 2.26 (Iwasawa data are not automatically persistent). An Iwasawa module may have rich algebraic structure over Λ . That structure is not automatically a one-parameter constructible persistence module. The interface must declare the one-parameter filtration or persistence extraction explicitly.

Remark 2.27 (Finite-layer data are not automatically apex data). Finite-layer Iwasawa data may suggest a limiting pattern, but they do not automatically define the apex profile

$$\text{ApexProfile}_{i,W,\tau}(F_{\infty}^{\text{Iw}}).$$

The apex object or apex profile must be declared and justified by the interface.

MS-B.3bis. Iwasawa realization burden and invariant profiles

Definition 2.28 (Iwasawa realization burden). An Iwasawa interface must declare whether the realized object is built from the module itself, a finite-layer tower, a Selmer-type complex, a characteristic ideal profile, a valuation profile, a local-condition diagram, or a mixed construction. For each choice it must record the one-parameter extraction, constructibility evidence, transition morphisms, and any transfer defect.

Definition 2.29 (Iwasawa invariant profile declaration). An Iwasawa invariant profile declaration records which arithmetic invariants are used as source features, such as

$$\mu_{\text{Iw}}, \quad \lambda_{\text{Iw}}, \quad \text{char}_{\Lambda}, \quad \text{rank}, \quad v_p, \quad \text{or finite-layer length.}$$

The declaration must state whether the invariant is used as advisory data, a source feature for realization, a ledger component after certified conversion, or a candidate transport target.

Definition 2.30 (Iwasawa realization defect). An *Iwasawa realization defect* is a declared discrepancy

$$\delta^{\text{Iw}}$$

arising from converting Iwasawa module, tower, characteristic-ideal, local-condition, or finite-layer growth data into Core-readable filtered or persistence data.

Proposition 2.31 (Iwasawa module structure is not persistence structure). *The Λ -module structure of an Iwasawa module is not, by itself, a constructible one-parameter persistence module or a repaired evaluation profile.*

Proof. A Λ -module is an arithmetic-algebraic object. The Core persistence regime requires a declared one-parameter filtration or persistence object over the ground field, with repaired window-first evaluation. The module structure may guide realization, but it does not supply that realization automatically. $\square$

Proposition 2.32 (Characteristic ideal is not a defect profile). *An Iwasawa characteristic ideal, valuation profile, or elementary divisor decomposition is not automatically the kernel or cokernel defect profile of a Core tower comparison artifact.*

Proof. A defect profile in the Core Type IV sense is derived from the kernel or cokernel of the declared persistence morphism $\phi_{i,W,\tau}$. An Iwasawa characteristic ideal is defined in the arithmetic source category. A transport theorem is required to identify these data. $\square$

MS-B.4. Iwasawa μ and Collapse μ

Definition 2.33 (Arithmetic Iwasawa μ -invariant). An *arithmetic Iwasawa μ -invariant*, denoted

$$\mu_{\text{Iw}},$$

is an invariant attached to an Iwasawa module or tower according to the declared arithmetic convention of the interface.

Definition 2.34 (Collapse μ -diagnostic). The *Collapse μ -diagnostic* is the AK tower diagnostic

$$\mu_{\text{Collapse}}$$

defined in Appendix D as the sum of window-relative terminal-generic dimensions tgdim_W of the kernel defect profiles of declared repaired tower comparison artifacts over the declared finite monitored scope.

Theorem 2.35 (Iwasawa μ is not Collapse μ by default). *The arithmetic invariant*

$$\mu_{\text{Iw}}$$

is not identified with

$$\mu_{\text{Collapse}}$$

unless a separate arithmetic transport theorem proves that identification under a declared realization.

Proof. The invariant μ_{Iw} belongs to Iwasawa theory and is defined by arithmetic module structure. The invariant μ_{Collapse} is defined by window-relative terminal-generic dimensions of kernel defect profiles of declared repaired persistence tower comparison artifacts. These definitions live in different categories and measure different structures. An identification requires a theorem connecting the arithmetic module data to the persistence defect profiles. $\square$

Remark 2.36 (Notation collision warning). The shared symbol μ is not evidence of equality. All appearances of μ_{Iw} and μ_{Collapse} must be kept distinct unless a declared transport theorem identifies them.

Definition 2.37 (Iwasawa-to-Collapse μ transport statement). An *Iwasawa-to-Collapse μ transport statement* is a statement of either form:

$$\mu_{\text{Iw}} = 0 \implies \mu_{\text{Collapse}} = 0,$$

or

$$\mu_{\text{Collapse}} = 0 \implies \mu_{\text{Iw}} = 0,$$

under a declared Iwasawa interface.

Definition 2.38 (Iwasawa-to-Collapse diagnostic transport theorem). An *Iwasawa-to-Collapse diagnostic transport theorem* is an Iwasawa-to-Collapse transport statement supplied with:

- (i) the declared Iwasawa source category;
- (ii) the realization route;
- (iii) the repaired evaluation records;
- (iv) the declared tower comparison artifact;
- (v) the identification of arithmetic growth data with persistence defect profiles;
- (vi) all ledgered transfer and truncation defects;
- (vii) a proof of the stated implication.

Definition 2.39 (Default status of Iwasawa-to-Collapse transport). Every Iwasawa-to-Collapse transport statement is [Spec] by default unless separately proved as an interface theorem.

MS-B.5. Iwasawa λ , u , and finite-layer growth

Definition 2.40 (Arithmetic Iwasawa λ -invariant). An *arithmetic Iwasawa λ -invariant*, denoted

$$\lambda_{\text{Iw}},$$

is an invariant attached to an Iwasawa module or tower according to the declared arithmetic convention of the interface.

Definition 2.41 (Arithmetic finite-layer growth profile). A *finite-layer growth profile* is a sequence or function recording the growth of declared arithmetic quantities across finite layers:

$$g(n) = \text{declared size, rank, valuation, length, or defect at layer } n.$$

Definition 2.42 (Collapse u -diagnostic). The *Collapse u -diagnostic* is the AK tower diagnostic

$$u_{\text{Collapse}},$$

defined in Appendix D as the sum, over the declared finite monitored scope $\mathcal{M}$, of the window-relative terminal-generic dimensions tgdim_W of the cokernel defect profiles of declared repaired tower comparison artifacts.

Remark 2.43 (Arithmetic λ and Collapse u). The arithmetic invariant

$$\lambda_{\text{IW}}$$

is not automatically equal to, controlled by, or equivalent to

$$u_{\text{Collapse}}.$$

A separate diagnostic transport theorem is required to relate finite-layer arithmetic growth to window-relative terminal-generic cokernel defects of the Core tower comparison artifact.

Proposition 2.44 (Finite-layer growth is not Type IV diagnosis). A *finite-layer arithmetic growth profile does not determine*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}})$$

unless a declared transport theorem identifies the growth profile with the kernel and cokernel defect profiles of

$$\phi_{i,W,\tau}.$$

Proof. Finite-layer arithmetic growth is source-side arithmetic data. The Collapse diagnostics are defined from window-relative terminal-generic kernel and cokernel defect profiles of a declared persistence tower comparison artifact. A theorem connecting the two is required. $\square$

MS-B.6. Congruence interface

Definition 2.45 (Congruence datum). A *congruence datum* is a collection of arithmetic objects together with congruence relations among them. Examples include:

- (i) congruent modular forms;
- (ii) congruent Galois representations;
- (iii) congruence modules;
- (iv) Hecke algebra congruence ideals;
- (v) congruent Selmer data;
- (vi) congruent local conditions;
- (vii) congruence classes across tower layers;

(viii) congruence relations among p -adic L -profiles.

Definition 2.46 (Congruence realization hypothesis). A *congruence realization hypothesis* is a purpose-faithful realization

$$\mathfrak{R}_{\text{cong}} : C_{\text{cong}} \longrightarrow \text{FiltCh}(()k) \quad \text{or} \quad \mathfrak{R}_{\text{cong}} : C_{\text{cong}} \longrightarrow \text{Pers}_k^{\text{cons}}$$

that maps congruence data to Core-readable filtered or persistence data.

Definition 2.47 (Congruence Core profile). A *congruence Core profile* is a repaired evaluation profile

$$\text{Eval}_{i,W,\tau}(F_{\text{cong}})$$

produced from a congruence realization route.

Definition 2.48 (Congruence defect). A *congruence defect* is a declared measure

$$\delta^{\text{cong}}$$

of the failure of an arithmetic congruence relation to be preserved under the chosen realization or under a comparison of realized objects.

Definition 2.49 (Congruence ledger entry). A *congruence ledger entry* records:

- (i) the congruence datum;
- (ii) the realization route;
- (iii) the compared arithmetic objects or Core-readable objects;
- (iv) the repaired evaluation profiles compared, if applicable;
- (v) the congruence defect δ^{cong} ;
- (vi) how the defect enters the δ -ledger;
- (vii) artifact references.

Proposition 2.50 (Congruence defect affects Core only through ledger). A *congruence defect affects Core audit only if it is included in the declared δ -ledger*.

Proof. A congruence defect is not a Core gate verdict and not a monitored Type IV diagnostic. It affects Core audit only by contributing to the total budget $\Sigma\delta$. $\square$

Remark 2.51 (Congruence is not equality after realization). Arithmetic congruence does not automatically imply equality of realized persistence objects. Any persistence-level equality, interleaving bound, repaired evaluation comparison, or defect estimate must be declared and supported.

MS-B.6bis. Congruence comparison-mode projection

Definition 2.52 (Congruence comparison mode). Every congruence comparison consumed by an MS-B record declares exactly one local mode:

source_congruence, realized_exact, realized_metric, advisory_congruence, unavailable, failed.

Its projection to Core comparison modes is inherited from the Appendix MS-A arithmetic comparison-mode discipline:

- (i) source_congruence remains external unless transported to realized comparison data;
- (ii) realized_exact projects to Core mode exact only when an actual equality or persistence-profile isomorphism of the declared Core-readable profiles is supplied;
- (iii) realized_metric projects to Core mode metric_gap_certified only after a certified bottleneck bound, scalarization route, and two-sided threshold-gap record are supplied;
- (iv) advisory_congruence and unavailable project to not_compared;
- (v) failed blocks the invoked comparison route.

Definition 2.53 (Congruence-to-metric conversion). A congruence-to-metric conversion is a theorem or verified computation converting congruence depth, valuation distance, or congruence defect into a certified bottleneck bound on the actual windowed pre-threshold profiles consumed by a Core comparison. It must include all scalarization, crop, window-edge, and threshold-gap data required by the v19 metric protocol.

Proposition 2.54 (Source congruence is not persistence equality). *A source-side arithmetic congruence is not a persistence-profile equality, interleaving bound, or repaired evaluation comparison unless a declared realization theorem supplies that comparison.*

Proof. Congruence is a relation in the arithmetic source domain. Persistence equality, interleaving, and metric-gap certification are relations between Core-readable profiles. The latter do not follow from the former without a realization and comparison theorem. $\square$

Proposition 2.55 (Congruence defect is not Type IV defect). *A congruence defect δ^{cong} is not the kernel or cokernel defect profile of $\phi_{i,W,\tau}$ unless a transport theorem identifies them.*

Proof. The congruence defect records loss or discrepancy in arithmetic congruence transport. Type IV defects are computed from the kernel and cokernel of a declared persistence comparison artifact. They have different source types and cannot be identified by notation alone. $\square$

MS-B.7. Congruence failure and Type IV

Definition 2.56 (Congruence Type IV proxy). A *congruence Type IV proxy* is a congruence-level anomaly suggesting that a realized arithmetic tower may have monitored Type IV obstruction.

Proposition 2.57 (Congruence failure is not Type IV diagnosis). *Failure of a congruence relation is not equivalent to*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0)$$

unless a separate transport theorem proves that equivalence under the declared realization.

Proof. A congruence failure is an arithmetic relation failure. Monitored Type IV obstruction is defined by window-relative terminal-generic kernel and cokernel defects of a declared repaired tower comparison artifact. These are different notions. An equivalence requires a theorem identifying the congruence defect with the relevant persistence defect profiles. $\square$

Remark 2.58 (Correct use of congruence proxies). Congruence anomalies may be useful for deciding where to compute tower diagnostics. They may not replace the computation of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

MS-B.8. Control-theorem interfaces

Definition 2.59 (Arithmetic control statement). An *arithmetic control statement* is a statement asserting that arithmetic data at finite layers control, approximate, determine, or compare with arithmetic data at a limiting layer.

Definition 2.60 (Core-readable control statement). A control statement is *Core-readable* if, after realization, it induces a statement about a declared tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Definition 2.61 (Control defect). A *control defect* is a declared discrepancy

$$\delta^{\text{ctrl}}$$

measuring the failure of an arithmetic control statement or its realized persistence-level counterpart.

Definition 2.62 (Control artifact). A *control artifact* is a proof, computation, diagram, finite-layer comparison, or literature-supported reference used to support a control statement or a realized tower comparison artifact.

Proposition 2.63 (Arithmetic control does not imply Core tower cleanliness by default). An *arithmetic control statement does not imply*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

unless it is realized as a Core-readable control statement and proved to identify or kill the relevant kernel and cokernel defect profiles of the declared tower comparison artifact.

Proof. Arithmetic control concerns arithmetic layers and their transition behavior. Core tower cleanliness concerns repaired persistence defect profiles associated to

$$\phi_{i,W,\tau}.$$

The former implies the latter only when the realization preserves the relevant tower comparison data and the control statement is strong enough to eliminate window-relative terminal-generic kernel and cokernel defects. $\square$

Remark 2.64 (Control theorem status). Control-theorem interfaces are [Spec] by default. A classical arithmetic control theorem may be used as an external hypothesis, but its translation into Core diagnostics still requires an interface proof.

MS-B.9. Finite-layer truncation and apex policy

Definition 2.65 (Finite-layer truncation). A *finite-layer truncation* is the replacement of an infinite arithmetic tower

$$A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow \cdots$$

by a finite subsystem

$$A_0 \rightarrow A_1 \rightarrow \cdots \rightarrow A_N.$$

Definition 2.66 (Apex policy). An *apex policy* specifies how the limiting object

$$F_\infty$$

or the apex profile

$$\text{ApexProfile}_{i,W,\tau}(F_\infty)$$

is declared, approximated, completed, or reconstructed from arithmetic tower data.

Definition 2.67 (Finite-layer truncation defect). A *finite-layer truncation defect* is a declared discrepancy

$$\delta_N^{\text{trunc}}$$

arising from using only layers $0, \dots, N$ to represent, approximate, or compare with the declared apex behavior.

Proposition 2.68 (Finite-layer truncation does not define the apex by default). A *finite-layer truncation does not define*

$$F_\infty$$

or

$$\text{ApexProfile}_{i,W,\tau}(F_\infty)$$

unless an apex policy or convergence theorem is declared.

Proof. A finite subsystem contains only finite-layer data. The apex object represents limiting, completed, or externally supplied data. A declaration or theorem is required to pass from finite layers to the apex. $\square$

Remark 2.69 (Truncation must be ledgered). If finite-layer truncation is used in a certificate record, the truncation defect

$$\delta_N^{\text{trunc}}$$

must enter the declared ledger unless it is proved to vanish under the declared apex policy.

MS-B.9bis. Finite-to-infinite and apex-proxy discipline

Definition 2.70 (Finite-to-infinite reduction theorem). A *finite-to-infinite reduction theorem* is a theorem proving that a declared finite-layer computation or finite subsystem suffices to determine a stated infinite tower, completed apex, or terminal diagnostic conclusion. It must state its scope, hypotheses, finite bound, artifact references, and failure mode.

Definition 2.71 (Apex proxy status). An apex proxy has exactly one of the following statuses:

genuine_apex, proved_finite_reduction, bounded_approximation, advisory, undefined, failed.

Only genuine_apex or proved_finite_reduction may support an exact tower artifact. This status is interpreted through Definition 2.18 and the Appendix D apex provenance discipline.

Definition 2.72 (Aggregated truncation defect). When finite-layer defects δ_N^{trunc} are consumed through a common ledger, the aggregate notation

$$\delta^{\text{trunc}} := \bigoplus_{N \in \mathcal{N}_{\text{used}}} \delta_N^{\text{trunc}}$$

means the declared ledger aggregation of the actually used truncation defects. It is not a new primitive defect and must not double-count the same underlying discrepancy as $\delta^{\text{arith-trunc}}$ from Appendix MS-A.

Proposition 2.73 (Observed stabilization is not eventual stabilization). *Observed stabilization through finitely many tower layers does not prove eventual stabilization or apex equality unless a finite-to-infinite reduction theorem is supplied.*

Proof. Finite observation has bounded scope. Eventual stabilization and apex equality are statements over all sufficiently large layers or over a limiting object. The universal statement requires a theorem reducing it to the finite evidence. $\square$

Proposition 2.74 (Terminal diagnostics require terminal evidence). *A terminal-generic Type IV conclusion for an arithmetic tower requires terminal-tameness and terminal-scope evidence for the realized source, target, kernel, and cokernel profiles.*

Proof. The diagnostic uses the window-relative terminal-generic dimension of defect profiles. Without terminal-tameness and terminal-scope evidence, the diagnostic value is undefined rather than zero or obstructed. $\square$

MS-B.10. p -adic L -profile compatibility

Definition 2.75 (p -adic L -profile datum). A p -adic L -profile datum is a sequence, function, Newton polygon, valuation profile, zero profile, or finite-precision approximation arising from a declared p -adic L -function or related p -adic analytic object.

Definition 2.76 (p -adic L -profile realization). A p -adic L -profile realization is a purpose-faithful realization of a p -adic L -profile datum into a constructible persistence object, filtered object, repaired evaluation profile, or certificate-record component.

Definition 2.77 (p -adic L -profile defect). A p -adic L -profile defect is a declared discrepancy

$$\delta^{pL}$$

arising from finite precision, branch selection, interpolation, truncation, zero detection, Newton polygon approximation, or comparison with a realized Core profile.

Remark 2.78 (p -adic L -data are not Core profiles). A p -adic L -function or valuation sequence is not automatically a constructible persistence module. It becomes Core-readable only through a declared realization into the repaired evaluation regime.

Proposition 2.79 (p -adic L -profile compatibility is [Spec] by default). *Any statement identifying p -adic L -profile behavior with Core persistence, tower diagnostics, or Collapse indicators is [Spec] by default unless separately proved as an interface theorem.*

Proof. A p -adic L -profile belongs to arithmetic analytic data. Core persistence and tower diagnostics belong to repaired persistence and certificate records. Identifying them requires a transport theorem. $\square$

MS-B.10bis. p -adic branch, precision, and zero-detection discipline

Definition 2.80 (p -adic branch policy). A p -adic branch policy declares the chosen branch, variable, specialization, interpolation domain, normalization, and one-parameter extraction used to convert p -adic analytic data into a Core-readable profile candidate.

Definition 2.81 (p -adic precision record). A p -adic precision record states the finite precision, truncation order, valuation bound, zero-detection rule, Newton polygon approximation, and error behavior used to form a p -adic L -profile datum.

Proposition 2.82 (Finite-precision p -adic evidence is not analytic identity). *Finite-precision p -adic computations do not prove a p -adic L -function identity, zero identity, interpolation identity, or arithmetic transport theorem unless a separate exact or certified-approximation theorem supplies that conclusion.*

Proof. Finite-precision data have bounded precision and declared numerical scope. Analytic identities and transport theorems assert exact or universally quantified statements. A reduction, exactness, or certified-approximation theorem is therefore required. $\square$

Proposition 2.83 (Zero detection is not Type IV vanishing). *Detecting or approximating zeros of a p -adic L -profile does not prove*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

unless a transport theorem identifies the zero behavior with the relevant Core defect profiles.

Proof. Zero detection concerns arithmetic analytic data. Type IV vanishing concerns window-relative terminal-generic kernel and cokernel defect dimensions of a declared persistence artifact. The two predicates have different types and require a transport theorem for identification. $\square$

MS-B.11. Cyclotomic / Iwasawa / congruence certificate records

Definition 2.84 (Cyclotomic certificate record). A *cyclotomic certificate record* is an arithmetic certificate record whose source datum is a cyclotomic tower and whose Core-readable object is produced by a cyclotomic realization hypothesis.

Definition 2.85 (Iwasawa certificate record). An *Iwasawa certificate record* is an arithmetic certificate record whose source datum includes an Iwasawa module, Iwasawa tower, or Iwasawa-layer object and whose Core-readable object is produced by an Iwasawa interface.

Definition 2.86 (Congruence certificate record). A *congruence certificate record* is an arithmetic certificate record whose source datum includes congruence relations and whose realized data include congruence defect ledger entries.

Definition 2.87 (MS-B manifest entry). An *MS-B manifest entry* records:

- (i) whether the source interface is cyclotomic, Iwasawa, congruence, p -adic L -profile, control-theoretic, or mixed;
- (ii) the arithmetic source data;
- (iii) the realization hypothesis;
- (iv) the purpose profile;
- (v) the realization target;

- (vi) tower data, if any;
- (vii) apex policy, if any;
- (viii) Iwasawa-layer data, if any;
- (ix) congruence data, if any;
- (x) p -adic L -profile data, if any;
- (xi) declared windows and thresholds;
- (xii) collapse policy;
- (xiii) monitored degrees;
- (xiv) repaired evaluation records;
- (xv) tower comparison artifacts, if Type IV diagnostics are invoked;
- (xvi) Type IV diagnostics;
- (xvii) arithmetic defect ledger entries;
- (xviii) gate verdicts;
- (xix) artifact references;
- (xx) status of all arithmetic transport claims.

Proposition 2.88 (MS-B records must satisfy Appendix MS-A). *Every cyclotomic, Iwasawa, congruence, control-theoretic, or p -adic L -profile certificate record must satisfy the arithmetic interface requirements of Appendix MS-A.*

Proof. Cyclotomic, Iwasawa, congruence, control-theoretic, and p -adic L -profile interfaces are special cases of arithmetic interfaces. Therefore they inherit the realization, eligibility, ledger, diagnostic, manifest, and non-claim requirements of Appendix MS-A. $\square$

MS-B.12. Mixed interfaces

Definition 2.89 (Mixed cyclotomic–Iwasawa interface). *A mixed cyclotomic–Iwasawa interface combines a cyclotomic tower with Iwasawa module data and declares how both are realized into the same Core-readable tower or compatible family of Core-readable objects.*

Definition 2.90 (Mixed Iwasawa–congruence interface). *A mixed Iwasawa–congruence interface combines Iwasawa module data with congruence data and declares how congruence defects interact with the Iwasawa-layer realization.*

Definition 2.91 (Mixed Iwasawa– p -adic L -profile interface). *A mixed Iwasawa– p -adic L -profile interface combines Iwasawa module data with p -adic L -profile data and declares how both are realized into compatible Core-readable profiles, towers, or certificate-record components.*

Definition 2.92 (Mixed arithmetic ledger). *A mixed arithmetic ledger aggregates all declared arithmetic defects:*

$$\delta_{\text{mixed}}^{\text{arith}} := \delta^{\text{cyc}} \oplus \delta^{\text{Iw}} \oplus \delta^{\text{cong}} \oplus \delta^{\text{ctrl}} \oplus \delta^{\text{trunc}} \oplus \delta^{pL},$$

with aggregation interpreted in the declared ledger semantics.

Proposition 2.93 (Mixed interfaces require compatible realization targets). *A mixed arithmetic interface is Core-admissible only if all participating arithmetic structures are realized into compatible Core-readable targets.*

Proof. If the cyclotomic, Iwasawa, congruence, control, and p -adic L -profile components produce incompatible targets, then there is no single Core object or compatible family of Core objects on which gate and diagnostic conditions can be evaluated. Thus compatibility of realization targets is required. $\square$

Remark 2.94 (Mixed does not mean stronger). Combining several arithmetic structures does not automatically strengthen a Core certificate. It may add constraints, but it also adds transfer, truncation, comparison, and compatibility defects that must be ledgered.

MS-B.12bis. Mixed-interface non-compensation

Definition 2.95 (Mixed compatibility record). A mixed compatibility record declares the common Core-readable target type, shared windows, thresholds, monitored degrees, comparison modes, artifact identities, and ledger aggregation policy across the participating cyclotomic, Iwasawa, congruence, control, and p -adic L -profile components.

Proposition 2.96 (Mixed components do not compensate). *A pass, favorable advisory value, or strong source theorem in one mixed-interface component cannot compensate for a failed or undefined obligation in another invoked component.*

Proof. The v19 verifier-side obligation calculus is conjunctive on required obligations and does not permit pass by cancellation. Each invoked component must satisfy its own realization, comparison, artifact, diagnostic, and ledger requirements. $\square$

Proposition 2.97 (Cross-component agreement requires a comparison record). *Agreement between Iwasawa, congruence, cyclotomic, control, and p -adic L -profile components is not verifier-side evidence unless a declared exact or metric-gap-certified comparison record connects their Core-readable profiles.*

Proof. Cross-component agreement is a relation between realized outputs only after the outputs have compatible target types and a declared comparison mode. Without that record, the agreement remains advisory or source-side. $\square$

MS-B.13. Arithmetic stable bands

Definition 2.98 (Arithmetic advisory stable band). An *arithmetic advisory stable band* is a threshold interval on which arithmetic indicators, such as Iwasawa-layer growth, congruence defect size, p -adic valuation behavior, or cyclotomic transition behavior, appear stable under the declared advisory policy.

Definition 2.99 (Arithmetic Core stable band). An *arithmetic Core stable band* is a threshold interval on which the realized repaired tower diagnostics satisfy

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

for all declared monitored degrees and all thresholds in the band, relative to declared tower comparison artifacts.

Proposition 2.100 (Arithmetic advisory stability is not Core stability). *An arithmetic advisory stable band is not an arithmetic Core stable band unless the monitored Type IV diagnostics are explicitly verified throughout the band.*

Proof. Advisory stability concerns arithmetic indicators. Core stability concerns the vanishing of

$$\mu_{\text{Collapse}} \quad \text{and} \quad u_{\text{Collapse}}$$

computed from declared tower comparison artifacts. These are distinct unless connected by a proved transport theorem. $\square$

MS-B.13bis. Threshold-sweep and stable-band certification

Definition 2.101 (Arithmetic threshold sweep record). An arithmetic threshold sweep record declares the threshold interval, sampled thresholds, monitored degrees, window policy, tower artifacts, advisory indicators, diagnostic records, and any theorem reducing continuum behavior to finite samples.

Definition 2.102 (Certified Core stable band). A certified Core stable band is an arithmetic Core stable band that is artifact-coherent in the sense of the Appendix D stable-band discipline. It requires a fixed declared monitored scope, compatible tower comparison artifact family, stable provenance, terminal-tameness evidence, unchanged terminal mode, and the required exact or metric comparison records throughout the band. In addition, the realized monitored Type IV diagnostics must have status `certified_zero` throughout the band, either directly or by a theorem reducing the band statement to finitely many verified records.

Proposition 2.103 (Sampled stability is not certified stable band). *Stability observed at finitely or countably many sampled thresholds does not certify a Core stable band unless a theorem reduces the whole band to the sampled data under the artifact-coherent stable-band conditions.*

Proof. Threshold crossings, changes in hard-deletion status, artifact changes, provenance changes, or diagnostic changes may occur between samples. A universal band claim therefore requires either direct verification throughout the band or a reduction theorem compatible with the Appendix D stable-band discipline. $\square$

Proposition 2.104 (Stable band requires threshold-gap discipline when metric comparisons are used). *If a certified Core stable band uses metric comparisons through hard deletion, each consumed comparison must satisfy the v19 two-sided threshold-gap requirement on the corresponding threshold range.*

Proof. Hard threshold deletion is not globally non-expansive. Metric transport through deletion is valid only under the repaired threshold-gap-safe conditions for the profiles and thresholds consumed. $\square$

MS-B.14. Summary of cyclotomic / Iwasawa / congruence interfaces

Theorem 2.105 (Appendix MS-B interface package). *Within the Problem Interface layer, the following package is available.*

- (i) *Cyclotomic towers become Core-readable only after a declared cyclotomic realization hypothesis.*
- (ii) *Iwasawa modules become Core-readable only after a declared Iwasawa interface and one-parameter filtration or persistence extraction.*
- (iii) *Congruence data become Core-readable only after a declared congruence realization hypothesis.*
- (iv) *p -adic L -profiles become Core-readable only after a declared profile realization.*
- (v) *All Core-relevant profiles are evaluated through*

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

(vi) Tower diagnostics require declared tower comparison artifacts

$$\phi_{i,W,\tau}.$$

(vii) μ_{IW} is not identified with μ_{Collapse} by default.

(viii) λ_{IW} or finite-layer growth is not identified with u_{Collapse} by default.

(ix) Congruence failure is not Type IV diagnosis by default.

(x) Arithmetic control statements do not imply Core tower cleanliness unless realized and proved to control the relevant repaired defect profiles.

(xi) Finite-layer truncation does not define the apex without an apex policy or convergence theorem.

(xii) Arithmetic defects affect Core audit only through the declared δ -ledger.

(xiii) Mixed cyclotomic–Iwasawa–congruence– p -adic L interfaces require compatible realization targets.

(xiv) Arithmetic advisory stable bands are not Core stable bands without diagnostic verification.

(xv) All cyclotomic, Iwasawa, congruence, control-theoretic, and p -adic L -profile certificate records must satisfy Appendix MS-A and the Minimal Manifest Axiom.

(xvi) Tower-axis selection, realized tower morphism records, apex identity records, and apex vocabulary alignment are non-scalar obligations.

(xvii) Iwasawa module structure, characteristic ideals, and module-theoretic invariants are not persistence structures or Type IV defect profiles by default.

(xviii) Source-side congruence becomes Core comparison evidence only through realized exact comparison or metric-gap-certified comparison.

(xix) Arithmetic control theorems do not themselves supply Core tower artifacts.

(xx) Observed finite-layer stabilization is not eventual stabilization without a finite-to-infinite reduction theorem.

(xxi) Finite-precision p -adic evidence does not prove analytic identity or Type IV vanishing.

(xxii) Mixed-interface components do not compensate for one another; every invoked component must satisfy its own obligations.

(xxiii) Sampled threshold stability is not a certified Core stable band without an artifact-coherent band-reduction theorem.

(xxiv) Metric comparisons used across stable bands remain subject to the v19 two-sided threshold-gap discipline.

Proof. Items (i)–(iv) follow from the corresponding realization definitions. Items (v)–(vi) follow from the repaired Core evaluation and tower artifact requirements. Item (vii) is Theorem 2.35. Item (viii) follows from Remark 2.43 and Proposition 2.44. Item (ix) is Proposition 2.57. Item (x) is Proposition 2.63. Item (xi) is Proposition 2.68. Item (xii) follows from Propositions 2.50 and the arithmetic ledger discipline. Item (xiii) is Proposition 2.93. Item (xiv) is Proposition 2.100. Item (xv) is Proposition 2.88. $\square$

MS-B.15. Explicit exclusions

The following are not asserted in Appendix MS-B.

- No Iwasawa main conjecture is proved.
- No cyclotomic control theorem is proved.
- No Selmer control theorem is proved.
- No congruence theorem is proved.
- No modularity or automorphy theorem is proved.
- No arithmetic finiteness theorem is proved.
- No p -adic L -function identity is proved.
- No μ_{Iw} -vanishing theorem is proved.
- No equality

$$\mu_{\text{Iw}} = \mu_{\text{Collapse}}$$

is asserted by default.

- No implication

$$\mu_{\text{Iw}} = 0 \Rightarrow \mu_{\text{Collapse}} = 0$$

is asserted by default.

- No implication

$$\mu_{\text{Collapse}} = 0 \Rightarrow \mu_{\text{Iw}} = 0$$

is asserted by default.

- No equality or implication between

$$\lambda_{\text{Iw}}$$

and

$$u_{\text{Collapse}}$$

is asserted by default.

- No finite-layer growth profile is treated as a Type IV diagnostic.
- No finite-layer truncation is treated as an apex object without an apex policy.
- No congruence failure is identified with Type IV obstruction by default.
- No arithmetic advisory stable band is treated as a Core stable band.
- No arithmetic control statement replaces monitored Type IV diagnostics.
- No arithmetic tower replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No arithmetic pre-gate replaces B-Gate⁺.
- No p -adic L -profile is treated as a Core persistence profile without realization.
- No Core pass is interpreted as an arithmetic theorem without a separate arithmetic transport theorem.
- No source-side congruence is treated as Core exact comparison without realized profile equality or isomorphism.
- No congruence depth, valuation estimate, or arithmetic closeness statement is treated as metric-gap-certified comparison without the v19 two-sided threshold-gap record.
- No finite observation of layer stabilization is treated as eventual stabilization without a finite-to-infinite reduction theorem.
- No finite-stage proxy is treated as a genuine apex without an apex identity record and Appendix D-compatible apex provenance.
- No certified Core stable band is asserted without artifact-coherent tower artifacts, terminal-tameness evidence, provenance control, and required exact or metric comparison records throughout the band.
- No finite-precision p -adic computation proves analytic identity or Type IV vanishing without a separate theorem.
- No mixed-interface component compensates for a failed or undefined obligation in another component.
- No sampled threshold behavior proves a certified Core stable band without an artifact-coherent threshold-band reduction theorem.
- No metric comparison through hard deletion is valid without the v19 two-sided threshold-gap record when such comparison is consumed.

MS-B.16. Provenance and status

Remark 2.106 (Ledger namespace registration for MS-B). The symbols

$$\delta^{\text{cyc}}, \delta^{\text{lw}}, \delta^{\text{cong}}, \delta^{\text{ctrl}}, \delta_N^{\text{trunc}}, \delta^{\text{trunc}}, \delta^{\text{arith-extract}}, \delta^{pL}$$

are interface-layer defect names. They require registration in the ledger catalog or an equivalent interface-layer namespace table before they are consumed by an audit record. The symbols $\delta^{\text{cyc}}, \delta^{\text{lw}}, \delta^{\text{cong}}, \delta^{\text{ctrl}}, \delta^{\text{trunc}},$ and δ^{pL} are MS-B specializations or aggregates of Appendix MS-A arithmetic transfer, comparison, extraction, and truncation defects. A record must declare whether an MS-B symbol aliases, refines, or aggregates an Appendix MS-A defect; the same primitive discrepancy may not be double-counted.

The material in this appendix depends on:

- (i) the arithmetic interface meta-theory and v19.0 hardening discipline of Appendix MS-A;
- (ii) the repaired constructible persistence framework of Appendix A;
- (iii) the admissible representative discipline of Appendix B;
- (iv) the eligible realization regime of Appendix C;
- (v) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;

- (vi) the window and local-to-global discipline of Appendix E;
- (vii) the manifest requirements of Appendix G;
- (viii) the advisory discipline of Appendix H;
- (ix) the discretization and measurement discipline of Appendices I and J;
- (x) the ledger semantics of Appendix M, including interface-layer defect namespaces when registered;
- (xi) the interface protocol of Chapter 8;
- (xii) the non-claim discipline of Chapter 7.

Its role is to provide disciplined interface templates for cyclotomic, Iwasawa, congruence, control, and p -adic L -profile structures without converting them into arithmetic theorems.

Status labels for Appendix MS-B. *Problem-interface definitions:* Definitions [2.8](#), [2.9](#), [2.10](#), [2.11](#), [2.12](#), [2.21](#), [2.22](#), [2.23](#), [2.24](#), [2.25](#), [2.33](#), [2.34](#), [2.37](#), [2.38](#), [2.39](#), [2.40](#), [2.41](#), [2.42](#), [2.45](#), [2.46](#), [2.47](#), [2.48](#), [2.49](#), [2.56](#), [2.59](#), [2.60](#), [2.61](#), [2.62](#), [2.65](#), [2.66](#), [2.67](#), [2.75](#), [2.76](#), [2.77](#), [2.84](#), [2.85](#), [2.86](#), [2.87](#), [2.89](#), [2.90](#), [2.91](#), [2.92](#), [2.98](#), [2.99](#).

Problem-interface propositions/theorems: Propositions [2.14](#), [2.44](#), [2.50](#), [2.57](#), [2.63](#), [2.68](#), [2.79](#), [2.88](#), [2.93](#), [2.100](#), Theorems [2.35](#), [2.105](#).

Operational cautions: Remarks [2.1](#), [2.2](#), [2.3](#), [2.13](#), [2.26](#), [2.27](#), [2.36](#), [2.43](#), [2.51](#), [2.58](#), [2.64](#), [2.69](#), [2.78](#), [2.94](#).

Explicit exclusions: Section MS-B.15.

3 Appendix MS-C: Mirror Reflection and Categorical Comparison

MS-C.1. Scope and interface status

This appendix defines the interface discipline for Mirror Reflection and categorical comparison in the arithmetic, motivic, geometric, and categorical surroundings of the AK framework. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Mirror reflection and categorical comparison are interface structures, not Core theorems.

In particular, this appendix does not prove mirror symmetry, categorical equivalence, motivic equivalence, derived equivalence, Fukaya equivalence, Langlands correspondence, modularity, automorphy, or any arithmetic theorem. It only specifies how such comparison structures may be made Core-readable and audit-compatible.

All Core-relevant persistence data produced by a Mirror Reflection or categorical comparison interface must be evaluated through the repaired windowed profile:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Whenever tower diagnostics are invoked, the required comparison is a declared tower comparison artifact:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

Remark 3.1 (Interface status). All problem-specific implications involving Mirror Reflection or categorical comparison are [Spec] by default unless separately promoted and proved under the interface protocol of Chapter 8 and Appendix MS-A.

Remark 3.2 (Relation to Appendices L and N). Appendix L supplies the Mirror/Transfer 2-cell ledger discipline. Appendix N supplies the derived, sheaf, stack, and higher-categorical extension discipline. The present appendix applies those disciplines to arithmetic, motivic, geometric, and categorical comparison interfaces.

Remark 3.3 (No exact-collapse restoration). This appendix does not restore the old exact-collapse language. Any collapse-related Mirror or categorical comparison must use

$$T_{\tau}^{\text{bar}}$$

at the barcode/profile level, or the repaired evaluation

$$\text{Eval}_{i,w,\tau}.$$

No global exact collapse functor or positive-threshold Serre localization is assumed.

MS-C.1bis. Reflection-interface sovereignty and claim inflation control

Definition 3.4 (Reflection-interface sovereignty). Reflection-interface sovereignty is the rule that Mirror Reflection, categorical comparison, derived equivalence, motivic comparison, sheaf-theoretic transformation, Fukaya-side structure, or any higher-categorical coherence datum remains outside the AK Core unless it re-enters through declared verifier-side data. The admissible re-entry routes are:

- (i) Core-readable filtered or persistence objects;
- (ii) repaired evaluation records $\text{Eval}_{i,w,\tau}(F)$;
- (iii) admissible representatives $C_{\tau,w}F$;
- (iv) eligible realized objects for an invoked Ext^1 -clause;
- (v) declared tower comparison artifacts $\phi_{i,w,\tau}$;
- (vi) typed Type IV diagnostic records;
- (vii) certified metric-ledger components;
- (viii) audit-readable certificate records satisfying the Minimal Manifest Axiom.

Definition 3.5 (Reflection claim inflation). *Reflection claim inflation* occurs when a Mirror candidate, comparison diagram, natural transformation, derived equivalence candidate, weak equivalence, adjunction-like datum, or higher coherence cell is described or consumed as if it were a Core theorem, Core exact comparison, Type IV diagnostic, or problem-specific theorem without the required verifier-side record.

Proposition 3.6 (No reflection sovereignty override). *No Mirror, categorical, derived, sheaf, stack, Fukaya, motivic, or higher-categorical structure overrides the repaired Core verifier-side requirements.*

Proof. The AK Core consumes typed verifier-side records: repaired evaluations, representatives, eligibility records, tower artifacts, diagnostics, ledgers, gate verdicts, and manifest data. Reflection and categorical structures have external or extension-layer semantics. Unless they are realized as declared verifier-side records, they cannot affect a Core verdict. $\square$

Proposition 3.7 (Diagrammatic elegance is not proof). *Commutative-looking diagrams, naturality squares, mirror-symmetric pictures, adjunction-like patterns, or higher coherence sketches are not proof of a Core claim unless the claimed implication is verified through the declared interface protocol.*

Proof. Such diagrams may guide proof construction, but the Core verdict is determined only by typed, reconstructible verifier-side data. A diagrammatic pattern supplies no repaired evaluation, no eligible realization, no tower artifact, no budget comparison, and no manifest evidence by itself. $\square$

Remark 3.8 (Reflection meaning is proposer-side by default). Mirror or categorical meaning may guide the choice of realization, comparison cells, windows, thresholds, or candidate transport statements. By default it is proposer-side structure, not verifier-side theorem evidence.

MS-C.2. Mirror Reflection setting

Definition 3.9 (Mirror Reflection setting). A *Mirror Reflection setting* consists of:

- (i) a source domain $\mathcal{A}$, typically arithmetic, motivic, geometric, symplectic, sheaf-theoretic, or categorical;
- (ii) a reflected or mirror domain $\mathcal{A}^\vee$;
- (iii) a declared comparison operation

$$\text{Mir} : \mathcal{A} \dashrightarrow \mathcal{A}^\vee;$$

- (iv) a declared realization target for both sides;
- (v) a declared policy for comparing the two realized outputs;
- (vi) a declared ledger policy for all nonzero comparison defects.

Definition 3.10 (Mirror Reflection datum). A *Mirror Reflection datum* is a tuple

$$\mathfrak{M} = (A, A^\vee, \text{Mir}, \mathfrak{R}, \mathfrak{R}^\vee, \Theta),$$

where:

- (i) $A \in \mathcal{A}$ is a source object;
- (ii) $A^\vee \in \mathcal{A}^\vee$ is a reflected object or mirror candidate;
- (iii) Mir is the declared mirror comparison;
- (iv) $\mathfrak{R}$ and $\mathfrak{R}^\vee$ are realization routes into Core-readable or extension-readable objects;
- (v) Θ is the declared comparison data between the realized outputs.

Definition 3.11 (Mirror realization pair). A *Mirror realization pair* is a pair of realized objects

$$F_A := \mathfrak{R}(A), \quad F_{A^\vee} := \mathfrak{R}^\vee(A^\vee),$$

where each side is either Core-readable or extension-readable with a declared extraction route to Core-readable data.

Remark 3.12 (Mirror Reflection is not assumed symmetric). The word “mirror” does not imply an involution, equivalence, adjunction, duality, or categorical symmetry unless such structure is explicitly declared and supported. A Mirror Reflection datum may be one-directional, partial, approximate, or controlled by a ledgered defect.

Remark 3.13 (Mirror candidate is not mirror theorem). Declaring $A^\vee$ as a mirror candidate does not prove that it is a true mirror of A . It only creates a comparison target whose status must be established by strict, zero-cost, or controlled comparison data, and by any separate problem-specific transport theorem if a domain conclusion is claimed.

MS-C.2bis. Mirror directionality, variance, and involution discipline

Definition 3.14 (Mirror directionality record). A *Mirror directionality record* specifies whether the declared Mirror operation is covariant, contravariant, partially defined, multi-valued, correspondence-valued, span-valued, adjoint-like, or only advisory. It also records the domain of definition, target class, inverse or pseudo-inverse status, and all known failure cases.

Definition 3.15 (Mirror involution claim). A *Mirror involution claim* is a statement that a Mirror operation satisfies a relation of the form

$$\text{Mir}^\vee \circ \text{Mir} \simeq \text{id}$$

at a declared equivalence strength and on a declared scope.

Proposition 3.16 (Mirror terminology does not imply involution). *The term “mirror” does not imply involution, bidirectional equivalence, duality, full faithfulness, essential surjectivity, or information preservation.*

Proof. A Mirror Reflection datum supplies a source, a candidate reflected object, comparison routes, and ledger policies. None of these data includes an inverse, a unit-counit equivalence, or an information-preservation theorem unless such structure is explicitly declared and proved. $\square$

Proposition 3.17 (Mirror-side identity is external until realized). *An identity, equivalence, or isomorphism asserted on the Mirror side is not a Core exact comparison unless it is transported to an actual equality or persistence-profile isomorphism of the declared Core-readable data.*

Proof. Mirror-side identity is a statement in the source or reflected domain. Core exact comparison is a statement about repaired Core-readable profiles or verifier-side records. A realization and comparison theorem is required to move from the former to the latter. $\square$

Remark 3.18 (Mirror defect may be directional). The defect from A to $A^\vee$ need not equal the defect from $A^\vee$ back to A . Any symmetric or reversible defect policy must be separately declared.

MS-C.3. Categorical comparison setting

Definition 3.19 (Categorical comparison setting). A *categorical comparison setting* consists of:

- (i) two declared categories, bicategories, dg-categories, derived categories, sheaf categories, stack-theoretic categories, or higher categories

$$C_1, \quad C_2;$$

- (ii) a comparison functor, correspondence, span, adjunction-like datum, profunctor, kernel, or higher relation

$$\text{Comp} : C_1 \dashrightarrow C_2;$$

- (iii) a declared realization target into Core-readable data or technical-extension data;

- (iv) declared comparison cells;

- (v) declared comparison defects or 2-cell costs;
- (vi) artifact references supporting the comparison.

Definition 3.20 (Categorical comparison datum). A *categorical comparison datum* is a tuple

$$\mathfrak{C} = (C_1, C_2, \text{Comp}, X_1, X_2, \Gamma),$$

where $X_1 \in C_1$, $X_2 \in C_2$, and Γ records the declared comparison cells, defects, transformations, or equivalence candidates between their realized outputs.

Definition 3.21 (Core-readable categorical comparison). A categorical comparison is *Core-readable* if its realized outputs can be evaluated by the AK Core through:

$$\text{Eval}_{i,W,\tau}(-) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(-)),$$

or through an admissible certificate-record component feeding into B-Gate⁺, monitored Type IV diagnostics, a declared tower comparison artifact, or the δ -ledger.

Remark 3.22 (Categorical comparison is not categorical equivalence). A comparison datum is not automatically an equivalence of categories. It may be a partial functor, correspondence, span, derived comparison, sheaf transform, or higher cell. Any equivalence claim must be stated and proved separately.

MS-C.3bis. Categorical equivalence and realization-faithfulness burden

Definition 3.23 (Categorical equivalence-strength record). A *categorical equivalence-strength record* declares the exact strength of a categorical comparison:

isomorphism, equivalence, derived equivalence, Morita equivalence, quasi-equivalence, adjunction, fully faithful

It must also declare whether monoidal, triangulated, t -exact, filtered, enriched, or higher-coherent structure is preserved.

Definition 3.24 (Realization-faithfulness burden). The *realization-faithfulness burden* of a categorical comparison is the obligation to prove that the features used by the comparison survive the realization and extraction route into Core-readable data. It records:

- (i) the categorical feature intended to be preserved;
- (ii) the Core-visible feature intended to represent it;
- (iii) the realization and extraction route;
- (iv) the hypotheses under which preservation is asserted;
- (v) all artifacts or proof references;
- (vi) the failure behavior when preservation is unavailable.

Proposition 3.25 (Categorical equivalence is not profile equality). A *categorical equivalence, derived equivalence, Morita equivalence, quasi-equivalence, or adjunction does not automatically imply equality, interleaving, or threshold-gap certification of the realized persistence profiles.*

Proof. Categorical equivalences preserve the structures specified by their equivalence-strength record. A repaired persistence profile depends on a realization route, filtration, window restriction, and hard barcode threshold deletion. Unless the equivalence is proved to preserve those data at the declared strength, no persistence-profile comparison follows. $\square$

Proposition 3.26 (Unproved realization-faithfulness blocks transport). *If realization-faithfulness is [Spec], unavailable, or failed for the feature used by a Mirror or categorical transport statement, then that transport statement cannot be invoked as theorem evidence.*

Proof. A transport theorem requires correspondence between external categorical structure and Core-visible behavior. Without proved or discharged conditional realization-faithfulness, the correspondence is missing. $\square$

Remark 3.27 (Equivalence strength is feature-relative). A comparison may preserve one feature while failing to preserve another. For example, it may preserve a triangulated structure while failing to preserve a filtration, a windowed barcode profile, an Ext^1 -eligible amplitude, or a tower comparison artifact.

MS-C.4. Realization targets

Definition 3.28 (Mirror/Categorical realization target). A Mirror Reflection or categorical comparison interface must declare its realization target as one of:

- (i) a filtered object in $\text{FiltCh}(()k)$;
- (ii) a constructible persistence object in $\text{Pers}_k^{\text{cons}}$;
- (iii) a repaired windowed collapsed profile
$$\text{Eval}_{i,w,\tau}(F);$$
- (iv) an admissible representative
$$C_{\tau,w}F;$$
- (v) a realized eligible object in $D^{[-1,0]}(k\text{-mod})$;
- (vi) a declared tower comparison artifact
$$\phi_{i,w,\tau};$$
- (vii) a derived, sheaf, stack, dg, or higher-categorical object in a Technical Extension setting;
- (viii) a certificate-record component;
- (ix) an advisory or Search-side artifact.

Definition 3.29 (Core-testable realization target). A realization target is *Core-testable* if it supports the Core repaired windowed evaluation

$$\text{Eval}_{i,w,\tau}(-),$$

or if it supplies verifier-side data directly to a certificate record.

Definition 3.30 (Extraction route). An *extraction route* is a declared map, functor, construction, or protocol taking an extension-readable object to Core-readable data, such as:

$$\text{FiltCh}(()k), \quad \text{Pers}_k^{\text{cons}}, \quad \text{Eval}_{i,w,\tau}(F), \quad C_{\tau,w}F, \quad \phi_{i,w,\tau},$$

or a ledger component.

Proposition 3.31 (No Core use without Core-testable target). *A Mirror Reflection or categorical comparison interface cannot be used in Core certification unless it has a Core-testable realization target or a declared extraction route to verifier-side data.*

Proof. Core certification requires typed inputs for repaired evaluation, gate verification, diagnostics, budget comparison, or manifest reconstruction. A Mirror or categorical comparison without a Core-testable target or extraction route supplies no such input. Therefore it cannot support Core certification. $\square$

Remark 3.32 (Technical-extension target is not automatically Core target). A derived, sheaf, stack, or higher-categorical target may be meaningful in Part III. It becomes Core-relevant only after a declared extraction to Core-readable persistence, filtered, eligible, diagnostic, ledger, or certificate data.

MS-C.4bis. Extraction, filtration, and one-parameter reduction

Definition 3.33 (Reflection extraction defect). *A reflection extraction defect is a declared discrepancy introduced when extension-readable Mirror or categorical data are extracted into Core-readable filtered or persistence data. It is denoted*

$$\delta^{\text{ref-extract}}.$$

Definition 3.34 (Categorical one-parameter reduction). *A categorical one-parameter reduction is a declared rule converting multi-graded, filtered, stratified, derived, sheaf-theoretic, Fukaya-side, stack-theoretic, or higher-categorical data into a single ordered persistence parameter compatible with $\text{Pers}_k^{\text{cons}}$. It must declare the selected parameter, order convention, normalization, discarded information, and all induced defects.*

Proposition 3.35 (Extension-readable does not imply Core-readable). *An extension-readable derived, sheaf, stack, dg, Fukaya, or higher-categorical object is not Core-readable until a declared extraction route produces Core-readable filtered or persistence data, or a certificate-record component with verifier-side semantics.*

Proof. Extension-readable objects may organize external or higher-categorical structure. The Core verifier reads repaired persistence profiles, admissible representatives, eligible realized objects, tower artifacts, ledgers, and manifest components. A typed extraction is therefore required. $\square$

Proposition 3.36 (One-parameter reduction is not lossless by default). *A one-parameter reduction of categorical or Mirror data is not lossless by default. Any information loss relevant to the consumed Core claim must be proved irrelevant or recorded as a defect.*

Proof. Multi-graded, higher-categorical, or sheaf-theoretic data may contain relations not visible after projection to one parameter. If the certificate consumes a comparison depending on such data, the loss must either be shown irrelevant or ledgered. $\square$

Remark 3.37 (Common reflection extraction risks). Typical risks include forgetting A_∞ -operations, changing filtration conventions, projecting a multi-parameter family to one scale, replacing a stacky object by a coarse object, or turning a Fukaya-side energy filtration into a barcode filtration without a comparison theorem.

MS-C.5. Mirror comparison cells

Definition 3.38 (Mirror comparison cell). *A Mirror comparison cell is a declared 2-cell, higher cell, homotopy, natural transformation, weak equivalence, span comparison, kernel comparison, or controlled correspondence comparing two Mirror-related composites. It is denoted*

$$\alpha_{\text{Mir}}.$$

Definition 3.39 (Mirror comparison cost). A *Mirror comparison cost* is a declared ledger-valued quantity

$$\delta(\alpha_{\text{Mir}})$$

attached to a Mirror comparison cell.

Definition 3.40 (Mirror comparison status). A Mirror comparison has one of the following statuses:

- (i) *strict*: the compared composites agree strictly;
- (ii) *zero-cost*: the comparison is weak but has zero declared cost;
- (iii) *controlled*: the comparison has nonzero but ledgered cost;
- (iv) *advisory*: the comparison is suggestive but not ledger-certified;
- (v) *unavailable*: no admissible comparison is declared;
- (vi) *failed*: the comparison cost exceeds the available budget margin.

Definition 3.41 (Mirror comparison ledger). The *Mirror comparison ledger* is the aggregate

$$\delta^{\text{Mir}} := \bigoplus_{\alpha \in A_{\text{Mir}}} \delta(\alpha),$$

where A_{Mir} is the declared finite or summable family of Mirror comparison cells used in the certificate record.

Proposition 3.42 (Only strict, zero-cost, or controlled Mirror comparisons may support certification). *A Mirror comparison may support Core certification only if its status is strict, zero-cost, or controlled with ledgered cost and its declared comparison mode has an admissible projection to the relevant Core-readable record.*

Proof. Strict and zero-cost comparisons introduce no positive defect only after they have been realized at the declared Core-readable target or otherwise admitted as verifier-side records. Controlled comparisons introduce a defect but record it in the ledger, and still require any needed scalarization or profile-level comparison evidence. Advisory, unavailable, and failed comparisons do not supply audit-readable support for certification. $\square$

Remark 3.43 (Advisory Mirror comparison is non-verdict). An advisory Mirror comparison may guide exploration or suggest a candidate reflection. It is not verifier-side data unless converted into a controlled ledger-certified comparison or a Core-readable realization.

MS-C.5bis. Comparison-mode projection to Core

Definition 3.44 (Reflection comparison mode). A Mirror or categorical comparison consumed by a certificate record must declare exactly one local mode:

source_strict, source_equivalent, realized_exact, realized_metric, ledger_controlled, advisory_only, unavailable

Its projection to the Core comparison modes is:

- (i) realized_exact projects to exact only when an actual equality or persistence-profile isomorphism of the declared Core-readable profiles is supplied;

- (ii) `realized_metric` projects to `metric_gap_certified` only when a certified bottleneck bound and two-sided threshold-gap record are supplied for the windowed pre-threshold profiles;
- (iii) `ledger_controlled` may contribute to the metric ledger only when scalarized to an actual bottleneck discrepancy bound on the consumed profiles;
- (iv) `source_strict` and `source_equivalent` remain external until transported to `realized_exact` or `realized_metric`;
- (v) `advisory_only` and `unavailable` project to `not_compared`;
- (vi) failed blocks the invoked comparison route.

Definition 3.45 (Status–mode compatibility for reflection comparisons). A Mirror comparison status from Definition 3.40 is admissible only together with a declared local comparison mode from Definition 3.44. The compatibility rule is:

- (i) status *strict* supports Core certification only when the local mode is `realized_exact`, or when a `source_strict` comparison is separately transported to `realized_exact`;
- (ii) status *zero-cost* supports Core certification only after its zero-cost claim is realized at the declared Core-readable target or certified as a zero ledger component;
- (iii) status *controlled* must be paired with `ledger_controlled` or `realized_metric`; it does not project to Core mode `metric_gap_certified` until the required bottleneck and threshold-gap records exist;
- (iv) status *advisory* must be paired with `advisory_only`;
- (v) status *unavailable* must be paired with `unavailable`;
- (vi) status *failed* must be paired with `failed` and blocks the invoked comparison route.

In particular, a source-level strict comparison is never a Core exact comparison merely by being strict in the source domain.

Proposition 3.46 (Strict source comparison is not Core exact mode). *A strict or zero-cost comparison in a source, Mirror, or categorical domain is not Core mode exact unless it is realized as an actual exact comparison of the declared Core-readable profiles.*

Proof. Core exact mode is a verifier-side relation between declared Core-readable profiles or records. A strict source-domain equality has a different type. It must be transported through the realization and extraction route before it can become Core exact evidence. $\square$

Proposition 3.47 (Controlled comparison is not metric certification by name). *A controlled Mirror or categorical comparison is not metric-gap-certified merely because a cost is declared. It becomes metric-gap-certified only after the cost is converted into a certified bottleneck bound and protected by the two-sided threshold-gap record.*

Proof. The v19 metric theorem consumes a bottleneck bound for the actual windowed pre-threshold profiles and a two-sided threshold-gap certificate. A categorical or Mirror cost is not such a bound unless a conversion theorem or verified computation supplies it. $\square$

MS-C.6. Categorical comparison cells

Definition 3.48 (Categorical comparison cell). A *categorical comparison cell* is a declared cell

$$\alpha_{\text{cat}}$$

comparing two categorical pipelines, such as:

$$\text{Comp} \circ Q \quad \text{and} \quad Q' \circ \text{Comp},$$

where Q and Q' are declared operations in the respective categories.

Definition 3.49 (Categorical comparison cost). A *categorical comparison cost* is a declared ledger-valued quantity

$$\delta(\alpha_{\text{cat}})$$

attached to a categorical comparison cell.

Definition 3.50 (Categorical comparison ledger). The *categorical comparison ledger* is the aggregate

$$\delta^{\text{cat}} := \bigoplus_{\alpha \in A_{\text{cat}}} \delta(\alpha),$$

where A_{cat} is the declared finite or summable family of categorical comparison cells used in the certificate record.

Proposition 3.51 (Categorical comparison costs affect Core only through ledger). *Nonzero categorical comparison costs affect Core audit only through the declared δ -ledger.*

Proof. Categorical comparison costs are not Core gate verdicts and not monitored Type IV diagnostics. They affect Core audit only by contributing to the total budget $\Sigma\delta$. $\square$

Remark 3.52 (Categorical naturality is not free). A diagram that appears natural in a categorical model may still have a nonzero comparison defect relative to the Core-readable realization. If it is used in certification, that defect must be declared and ledgered.

MS-C.7. Mirror/categorical comparison with repaired evaluation

Definition 3.53 (Mirror-evaluation reflection). A *Mirror-evaluation reflection* compares the two paths:

$$A \mapsto \text{Mir}(A) \mapsto \text{Eval}_{i,w,\tau}(\mathfrak{R}^\vee(\text{Mir}(A)))$$

and

$$A \mapsto \text{Eval}_{i,w,\tau}(\mathfrak{R}(A)) \mapsto \text{Mir}_{i,w,\tau}(\text{Eval}_{i,w,\tau}(\mathfrak{R}(A))),$$

whenever these paths are defined in the declared realization target.

Definition 3.54 (Categorical evaluation comparison). A *categorical evaluation comparison* compares applying repaired windowed evaluation before or after a categorical comparison operation:

$$\text{Eval}_{i,w,\tau}(\text{Comp}(X)) \quad \text{versus} \quad \text{Comp}_{i,w,\tau}(\text{Eval}_{i,w,\tau}(X)).$$

Definition 3.55 (Evaluation reflection defect). The *evaluation reflection defect* is the declared discrepancy

$$\Delta_{\text{Eval,ref}}$$

associated to moving repaired windowed evaluation across a Mirror or categorical comparison operation.

Proposition 3.56 (Evaluation reflection is not automatic). *Mirror or categorical comparison does not automatically commute with repaired windowed evaluation.*

Proof. The repaired evaluation

$$\text{Eval}_{i,W,\tau} = T_{\tau}^{\text{bar}} W_W \mathbf{P}_i$$

depends on the persistence extraction, window restriction, and barcode-level threshold deletion. A Mirror or categorical comparison may change filtrations, bar lengths, realization targets, comparison domains, or extracted persistence profiles. Therefore the two operation orders need not agree. $\square$

Proposition 3.57 (Evaluation reflection defect must be budgeted). *If a certificate record uses Mirror-evaluation or categorical-evaluation replacement under nonzero defect, the defect must be included in the δ -ledger.*

Proof. The replacement changes the route by which Core-relevant repaired evaluation data are obtained. A nonzero discrepancy is admissible only if declared and budgeted. Therefore it must enter $\Sigma\delta$. $\square$

Remark 3.58 (Preferred Core route). The preferred verifier-side route is direct computation or reconstruction of

$$\text{Eval}_{i,W,\tau}(-)$$

on a declared Core-readable object. Mirror/categorical shortcuts require strict, zero-cost, or controlled status.

MS-C.7bis. Window, crop, and threshold-gap discipline for reflection

Definition 3.59 (Reflection metric record). A *reflection metric record* is a metric comparison record for Mirror or categorical data after realization. It must identify the actual windowed pre-threshold profiles being compared, the window W , threshold τ , bottleneck bound, crop and window-edge contributions, scalarization route, and two-sided threshold-gap certificate.

Proposition 3.60 (Source-level closeness is not profile-level closeness). *A source-domain distance, categorical cost, homotopy size, energy estimate, or mirror-side discrepancy does not imply a bottleneck bound on the realized windowed pre-threshold profiles unless a comparison theorem or verified computation proves that implication.*

Proof. The bottleneck bound needed by the repaired Core is a metric statement about the profiles to which T_{τ}^{bar} is applied after windowing. Source-level quantities live in different spaces and may not control filtration shifts, window cropping, or barcode lengths. $\square$

Proposition 3.61 (Exact reflection comparison is not robustness). *An exact Mirror or categorical comparison of specific repaired profiles does not by itself certify positive-radius robustness under perturbation.*

Proof. Exact comparison proves equality or isomorphism at the declared profiles. Robustness through hard deletion requires a positive metric bound and two-sided threshold-gap evidence. These are separate verifier-side records. $\square$

Remark 3.62 (Reflection gap protection is measured after windowing). Any threshold-gap protection used by a reflection comparison is measured on the realized windowed pre-threshold profiles, not on source objects, unwinded diagrams, or after-deletion profiles.

MS-C.8. Mirror/categorical comparison with barcode-level collapse

Definition 3.63 (Mirror-collapse reflection). A *Mirror-collapse reflection* compares the two paths:

$$A \mapsto \text{Mir}(A) \mapsto T_\tau^{\text{bar}} \mathfrak{R}^\vee(\text{Mir}(A))$$

and

$$A \mapsto T_\tau^{\text{bar}} \mathfrak{R}(A) \mapsto \text{Mir}_\tau(T_\tau^{\text{bar}} \mathfrak{R}(A)),$$

whenever these paths are defined in the declared barcode/profile realization target.

Definition 3.64 (Categorical collapse comparison). A *categorical collapse comparison* compares applying barcode-level threshold deletion before or after a categorical comparison operation:

$$T_\tau^{\text{bar}} \text{Comp}(X) \quad \text{versus} \quad \text{Comp}_\tau(T_\tau^{\text{bar}} X).$$

Definition 3.65 (Collapse reflection defect). The *collapse reflection defect* is the declared discrepancy

$$\Delta_{T,\text{ref}}$$

associated to moving barcode-level threshold deletion across a Mirror or categorical comparison operation.

Proposition 3.66 (Collapse reflection is not automatic). *Mirror or categorical comparison does not automatically commute with barcode-level threshold deletion.*

Proof. Barcode-level threshold deletion removes bars according to the declared threshold τ . A Mirror or categorical comparison may change filtrations, bar lengths, realizations, or comparison targets. Therefore the two operation orders need not agree. Commutation requires a strict, zero-cost, or controlled comparison. $\square$

Proposition 3.67 (Collapse reflection defect must be budgeted). *If a certificate record uses Mirror-collapse or categorical-collapse replacement under nonzero defect, the defect must be included in the δ -ledger.*

Proof. The replacement changes the route by which after-collapse data are obtained. A nonzero discrepancy is admissible only if declared and budgeted. Therefore it must enter $\Sigma\delta$. $\square$

Remark 3.68 (Collapse comparison is weaker than evaluation comparison). A collapse comparison alone does not control the full repaired evaluation order unless the interaction with window restriction and persistence extraction is also declared. The full Core-relevant comparison is the comparison of

$$\text{Eval}_{i,W,\tau}.$$

MS-C.9. Mirror/categorical comparison with eligibility

Definition 3.69 (Eligible categorical realization). A categorical realization is *eligible* for Core Ext^1 -use if the realized admissible representative satisfies:

$$\mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod})$$

together with the edge-realization condition of Appendix C.

Proposition 3.70 (Mirror/categorical comparison does not replace eligibility). *A Mirror or categorical comparison does not replace the eligible realization requirements needed to invoke the Core Ext^1 -clause.*

Proof. Eligibility is a condition on the realized admissible representative and its edge relation to repaired residual degree-1 persistence. A Mirror or categorical comparison may relate objects or pipelines, but it does not by itself prove amplitude $[-1, 0]$ or edge realization. Therefore eligibility must still be verified. $\square$

Remark 3.71 (Comparison may support eligibility, not replace it). A comparison theorem may help prove eligibility for a realized object. However, it must do so explicitly. The existence of a comparison diagram alone does not license the Ext^1 -edge.

MS-C.10. Reflection and monitored Type IV

Definition 3.72 (Mirror Type IV proxy). A *Mirror Type IV proxy* is a Mirror-derived signal suggesting possible monitored Type IV obstruction.

Definition 3.73 (Categorical Type IV proxy). A *Categorical Type IV proxy* is a categorical comparison signal suggesting possible monitored Type IV obstruction.

Definition 3.74 (Actual Type IV diagnostic for reflection data). For a declared finite monitored scope

$$\mathcal{M} \subset \{(i, W, \tau)\},$$

the actual monitored Type IV diagnostic for reflection data is computed only from declared tower comparison artifacts

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

for $(i, W, \tau) \in \mathcal{M}$. When the source, target, kernel, and cokernel of each invoked artifact are terminal-tame on the declared window-relative terminal scope, set

$$\mu_{i,W,\tau} := \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad u_{i,W,\tau} := \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}).$$

The aggregate diagnostics are

$$\mu_{\text{Collapse}} := \sum_{(i,W,\tau) \in \mathcal{M}} \mu_{i,W,\tau}, \quad u_{\text{Collapse}} := \sum_{(i,W,\tau) \in \mathcal{M}} u_{i,W,\tau}.$$

No numerical pair is imputed when Type IV is not invoked or when the artifact or terminal-tameness evidence is undefined.

Proposition 3.75 (Mirror/categorical Type IV proxy is not Type IV diagnosis). *Neither a Mirror Type IV proxy nor a categorical Type IV proxy determines whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0).$$

Proof. Monitored Type IV diagnostics are computed from the window-relative terminal-generic dimensions tgdim_W of the kernel and cokernel of declared tower comparison artifacts. Mirror or categorical signals may suggest where such defects occur, but they are not the diagnostics unless they compute or are proved equivalent to those defect profiles under the declared realization. $\square$

Remark 3.76 (Correct use of reflection proxies). Reflection proxies may be used for attention, search, or candidate prioritization. They may not replace the computation of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

MS-C.10bis. Type IV artifact coherence for reflection interfaces

Definition 3.77 (Reflection Type IV artifact-coherence record). A *Reflection Type IV artifact-coherence record* declares that all Type IV diagnostics consumed by a Mirror or categorical interface use the same monitored scope, compatible tower comparison artifacts, compatible terminal-tameness evidence, and stable provenance across the claimed comparison range.

Proposition 3.78 (Changing reflection artifacts changes the diagnostic claim). *If a reflection interface changes the tower comparison artifact, apex object, terminal mode, realization route, or monitored scope, then the Type IV diagnostic claim is a new claim and cannot be silently identified with the previous one.*

Proof. Type IV diagnostics are defined from the declared artifact and terminal-generic kernel and cokernel data. Changing the artifact, apex, terminal mode, realization, or monitored scope changes the verifier-side object being diagnosed. A new record is therefore required. $\square$

Remark 3.79 (Reflection proxy escalation). A Mirror or categorical proxy may justify escalation to an actual Type IV computation. It does not determine the diagnostic status until the artifact-coherent Type IV record is supplied.

MS-C.11. Reflection transport statements

Definition 3.80 (Mirror transport statement). A *Mirror transport statement* is a statement of the form

$$\text{Mirror-side structure} \implies \text{Core-visible behavior},$$

or

$$\text{Core-visible behavior} \implies \text{Mirror-side conclusion}.$$

Definition 3.81 (Categorical transport statement). A *Categorical transport statement* is a statement of the form

$$\text{categorical comparison structure} \implies \text{Core-visible behavior},$$

or

$$\text{Core-visible behavior} \implies \text{categorical conclusion}.$$

Definition 3.82 (Reflection transport theorem). A *Reflection transport theorem* is a Mirror or categorical transport statement supplied with:

- (i) declared source and target domains;
- (ii) declared realization targets;
- (iii) repaired evaluation records;
- (iv) comparison cells and their status;
- (v) all required ledger entries;
- (vi) eligibility data if the Ext^1 -clause is invoked;
- (vii) tower comparison artifacts if Type IV diagnostics are invoked;
- (viii) a proof that the Mirror or categorical structure corresponds to the claimed Core-visible behavior or conclusion.

Definition 3.83 (Default status of Mirror/categorical transport). Every Mirror or categorical transport statement is classified as [Spec] by default unless separately proved under the interface protocol.

Proposition 3.84 (Mirror/categorical transport does not enlarge the Core). *A Mirror or categorical transport theorem does not enlarge the Core mathematical package unless explicitly imported and reproved under the Core guard-rails.*

Proof. Mirror and categorical transport statements use external semantics. By Chapter 7 and Chapter 8, such problem-specific or extension-specific material remains outside the Core unless explicitly imported and reproved. $\square$

Remark 3.85 (Transport directions must be separated). The directions

$$\text{Mirror/categorical structure} \Rightarrow \text{Core behavior}$$

and

$$\text{Core behavior} \Rightarrow \text{Mirror/categorical conclusion}$$

are distinct. Neither follows from the other without additional hypotheses.

MS-C.11bis. Transport theorem promotion and external theorem boundary

Definition 3.86 (Core-to-reflection transport theorem). A *Core-to-reflection transport theorem* is a theorem of the form

$$\text{Core certificate behavior} \implies \text{Mirror or categorical conclusion},$$

together with:

- (i) the exact source and target domains;
- (ii) the realized Core-readable data consumed;
- (iii) the external conclusion claimed;
- (iv) all Core certificate clauses consumed;
- (v) a proof that those Core clauses imply the external conclusion;
- (vi) all non-scalar hypotheses;
- (vii) all ledgered defects;
- (viii) artifact references sufficient for independent reconstruction.

Definition 3.87 (Reflection theorem promotion). *Reflection theorem promotion* is the operation of moving a Mirror or categorical implication from [Spec] or bridge-candidate status to theorem status. It requires a proved reflection transport theorem or a proved Core-to-reflection transport theorem, together with updated claim-register status.

Proposition 3.88 (No external theorem by successful reflection certification). *A successful Core certificate for reflection data does not prove mirror symmetry, categorical equivalence, derived equivalence, motivic equivalence, Langlands correspondence, modularity, automorphy, or any Fukaya-categorical theorem.*

Proof. A Core certificate establishes Core-visible persistence, diagnostic, artifact, and budget conditions. The listed conclusions concern external mathematical semantics. A separate transport theorem is required. $\square$

Proposition 3.89 (Transport hypotheses are non-scalar). *Missing hypotheses of a Mirror or categorical transport theorem cannot be repaired by metric slack, diagrammatic elegance, advisory coherence, or successful Core certification.*

Proof. Transport hypotheses state the semantic correspondence between Core-visible data and external Mirror or categorical conclusions. They are logical and structural obligations, not numerical defects. $\square$

MS-C.12. Reflection certificate records

Definition 3.90 (Mirror Reflection certificate record). A *Mirror Reflection certificate record* is a certificate record whose source data include a Mirror Reflection datum and whose Core-readable data are obtained through declared Mirror-compatible realization.

Definition 3.91 (Categorical comparison certificate record). A *categorical comparison certificate record* is a certificate record whose source data include a categorical comparison datum and whose Core-readable data are obtained through declared categorical realization or extraction.

Definition 3.92 (MS-C manifest entry). An *MS-C manifest entry* records:

- (i) the Mirror or categorical source data;
- (ii) the source and target categories or domains;
- (iii) the Mirror or comparison operation;
- (iv) the realization targets;
- (v) extraction routes, if technical-extension targets are used;
- (vi) the compared composites;
- (vii) the comparison cells;
- (viii) the comparison costs;
- (ix) the ledger aggregation rule;
- (x) declared windows and thresholds;
- (xi) collapse policy;
- (xii) monitored degrees;
- (xiii) repaired evaluation records;
- (xiv) admissible representative data, if invoked;
- (xv) eligibility data, if Ext¹ is invoked;
- (xvi) tower comparison artifacts, if Type IV diagnostics are invoked;
- (xvii) monitored Type IV diagnostics;
- (xviii) Core gate verdicts;
- (xix) artifact references;
- (xx) status of all transport statements.

Proposition 3.93 (MS-C records must satisfy Minimal Manifest). *Every Mirror Reflection or categorical comparison certificate record must satisfy the Minimal Manifest Axiom of Appendix G.*

Proof. Mirror and categorical data do not exempt a certificate from Core auditability. The Minimal Manifest requires declared objects, windows, thresholds, collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledger policy, diagnostics, tower artifacts when invoked, gate verdicts, artifact references, and final status. Therefore MS-C records must satisfy those requirements. $\square$

Remark 3.94 (Artifact examples). Artifacts for MS-C may include category definitions, functor definitions, correspondence data, comparison diagrams, 2-cell data, derived or dg models, persistence extraction data, repaired evaluation records, tower comparison artifacts, ledger records, and formal proof traces. They support reconstruction but do not themselves determine Core pass.

MS-C.13. Reflection pass, Core pass, and categorical conclusion

Definition 3.95 (Reflection interface pass). A *Reflection interface pass* means that Mirror or categorical comparison data have been successfully realized into Core-readable or ledger-compatible form under the declared interface protocol.

Definition 3.96 (Core pass for reflection data). A *Core pass for reflection data* means that the corresponding certificate record passes B-Gate⁺, monitored Type IV diagnostics, required artifact checks, and the budget condition.

Definition 3.97 (Mirror/categorical conclusion). A *Mirror/categorical conclusion* is a problem-specific or extension-specific statement about mirror correspondence, categorical equivalence, derived equivalence, functorial comparison, structural reflection, motivic comparison, or geometric duality.

Proposition 3.98 (Core pass is not Mirror/categorical conclusion). A *Core pass for reflection data does not imply a Mirror or categorical conclusion unless a separate transport theorem proves that implication.*

Proof. A Core pass certifies Core-visible persistence, diagnostic, artifact, and budget conditions. A Mirror or categorical conclusion concerns external categorical semantics. A transport theorem is required to connect them. $\square$

Remark 3.99 (Three-layer distinction). The following must remain distinct:

Reflection interface pass, Core pass, Mirror/categorical theorem.

Confusing these layers would violate the non-claim discipline of Chapter 7.

MS-C.14. Relation to Fukaya-categorical extension

Remark 3.100 (Fukaya extension deferred). Fukaya-categorical structures are deferred to a successor interface appendix, planned as Appendix MS-F when that appendix is included in the release scope. The present appendix only sets up the general Mirror/categorical comparison interface. No existence, release inclusion, or theorem status of Appendix MS-F is asserted here, and no Fukaya-category theorem, Floer-theoretic theorem, or homological mirror symmetry theorem is asserted here.

Definition 3.101 (Fukaya-ready comparison). A Mirror/categorical comparison is *Fukaya-ready* if it declares:

- (i) a symplectic or Fukaya-side source category;
- (ii) an algebraic, arithmetic, geometric, or mirror-side target;
- (iii) comparison cells;

- (iv) realization targets;
- (v) persistence extraction routes;
- (vi) repaired evaluation records, if already available;
- (vii) ledger treatment of all comparison defects.

Proposition 3.102 (Fukaya-ready is not Fukaya-proved). *A Fukaya-ready comparison does not prove any Fukaya-categorical or homological mirror symmetry statement.*

Proof. Fukaya-ready means that the data are prepared for a successor Fukaya-categorical interface discipline, planned as Appendix MS-F when that appendix is included in the release scope. It is a readiness condition, not a theorem about Fukaya categories or mirror symmetry, and it does not assume that Appendix MS-F is present in the current release. $\square$

MS-C.15. Reflection-adjusted budget

Definition 3.103 (Reflection tower-artifact transport defect). *A reflection tower-artifact transport defect is a declared discrepancy*

$$\Delta_{\phi, \text{ref}}$$

arising when a Mirror or categorical comparison transports, substitutes, compares, or reconstructs a tower comparison artifact

$$\phi_{i, W, \tau} : \text{ColimProfile}_{i, W, \tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i, W, \tau}(F_{\infty}).$$

It is a nonzero ledger component only when the certificate consumes such artifact transport. When Appendix L Mirror/Transfer notation is also used, the relation between $\Delta_{\phi, \text{ref}}$ and the corresponding Mirror/Transfer artifact-transport component must be declared as alias, refinement, aggregate, or disjoint contribution.

Definition 3.104 (Reflection-adjusted budget). If a certificate uses Mirror or categorical comparison data, the reflection-adjusted budget is

$$\Sigma \delta_{\text{ref}} := \Sigma \delta_{\text{base}} \oplus \delta^{\text{Mir}} \oplus \delta^{\text{cat}} \oplus \Delta_{\text{Eval}, \text{ref}} \oplus \Delta_{T, \text{ref}} \oplus \Delta_{\phi, \text{ref}},$$

with omitted components set to the ledger identity and aggregation interpreted in the declared ledger semantics.

Proposition 3.105 (Reflection evidence must pass the adjusted scalarized budget). *A certificate record relying on nonzero Mirror or categorical comparison defects may pass Audit Mode only if a declared metric-ledger quantale and bottleneck scalarization supply a protected profile family $\mathfrak{B}_{\text{ref}}$ such that*

$$\text{val}_B(\Sigma \delta_{\text{ref}}) < \text{gap}_{\tau}(\mathfrak{B}_{\text{ref}}),$$

together with evidence that the scalarized value upper-bounds the actual bottleneck discrepancies consumed by the reflection comparison.

Proof. All nonzero defects used by a certificate must enter the typed δ -ledger. If Mirror or categorical comparison contributes nonzero defect, it is part of the total budget. The v19 budget comparison is made only after scalarization to a real bottleneck-discrepancy bound and only relative to the protected windowed pre-threshold profile family. Therefore Audit Mode requires the displayed strict scalarized comparison. $\square$

Remark 3.106 (Reflection is not budget-free). Mirror or categorical comparison may be conceptually elegant. However, elegance is not zero cost. If the comparison is not strict or zero-cost, its defect must be ledgered.

MS-C.15bis. Ledger namespace, non-compensation, and double-count prevention

Definition 3.107 (Reflection ledger namespace). The reflection ledger namespace contains, at minimum, the declared components

$$\delta^{\text{Mir}}, \quad \delta^{\text{cat}}, \quad \Delta_{\text{Eval,ref}}, \quad \Delta_{T,\text{ref}}, \quad \Delta_{\phi,\text{ref}}, \quad \delta^{\text{ref-extract}}.$$

Each component must be registered as a metric, extraction, comparison, or advisory-to-ledger component before it is consumed by a certificate.

Definition 3.108 (Reflection primitive discrepancy). A *reflection primitive discrepancy* is the underlying mismatch from which one or more displayed reflection ledger components are derived. If the same primitive discrepancy contributes to multiple displayed components, the ledger must declare whether those components are aliases, refinements, disjoint contributions, or aggregates.

Proposition 3.109 (No double counting of reflection defects). *The same primitive reflection discrepancy may not be counted twice in the metric ledger under different names unless the ledger explicitly proves that the two entries represent disjoint contributions.*

Proof. The metric budget is intended to upper-bound actual discrepancies consumed by the proof. Double-counting a single primitive discrepancy under multiple names obscures the relationship between the ledger and the actual bound. The alias/refinement/disjointness relation must therefore be declared. $\square$

Proposition 3.110 (Reflection defects do not compensate for missing artifacts). *Favorable or zero reflection-ledger values do not repair a missing realization target, missing extraction route, missing eligibility record, missing tower comparison artifact, missing terminal-tameness evidence, or missing transport theorem.*

Proof. Those missing items are non-scalar verifier-side or transport obligations. Numerical ledger values can bound metric discrepancies, but they cannot manufacture missing typed objects or missing proofs. $\square$

Remark 3.111 (Ledger-catalog registration). The reflection-specific components δ^{Mir} , δ^{cat} , $\Delta_{\text{Eval,ref}}$, $\Delta_{T,\text{ref}}$, $\Delta_{\phi,\text{ref}}$, and $\delta^{\text{ref-extract}}$ should be registered in the ledger catalog before theorem-evidence consumption. When Appendix L Mirror/Transfer ledger components such as $\mathfrak{d}_{MT}$, $\Delta_{\text{Eval},MT}$, $\Delta_{T,MT}$, or $\Delta_{\phi,MT}$ are also present, the certificate must declare whether each MS-C component is an alias, refinement, aggregate, or disjoint contribution relative to the Appendix L component. Until registered, they remain interface-local notation.

MS-C.16. Summary of Mirror Reflection and categorical comparison

Theorem 3.112 (Appendix MS-C interface package). *Within the Problem Interface layer, the following package is available.*

- (i) *Mirror Reflection settings require declared source domains, reflected domains, comparison operations, realization targets, comparison policies, and ledger policies.*
- (ii) *Categorical comparison settings require declared categories, comparison structures, realization targets, comparison cells, and comparison defects.*
- (iii) *Mirror or categorical comparison data become Core-relevant only through Core-testable realization targets, extraction routes, or certificate-record components.*
- (iv) *All Core-relevant persistence data are evaluated through*

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

- (v) *Mirror and categorical comparison cells must be strict, zero-cost, or controlled with ledgered cost and an admissible status–mode projection to support certification.*
- (vi) *Mirror-evaluation and categorical-evaluation commutation are not automatic.*
- (vii) *Mirror-collapse and categorical-collapse commutation are not automatic.*
- (viii) *Nonzero comparison defects must be included in the δ -ledger.*
- (ix) *Mirror/categorical comparison does not replace eligibility.*
- (x) *Mirror or categorical Type IV proxies do not replace monitored Type IV diagnostics.*
- (xi) *Type IV diagnostics require declared tower comparison artifacts*

$$\phi_{i,W,\tau}.$$

- (xii) *Mirror and categorical transport statements are [Spec] by default.*
- (xiii) *Mirror/categorical transport does not enlarge the Core unless explicitly imported and reproved.*
- (xiv) *MS-C certificate records must satisfy the Minimal Manifest Axiom.*
- (xv) *A Core pass for reflection data does not imply a Mirror or categorical theorem without a separate transport theorem.*
- (xvi) *Fukaya-ready comparison does not prove a Fukaya-categorical theorem.*
- (xvii) *Reflection evidence using nonzero comparison defects must pass the scalarized budget comparison*

$$\text{val}_B(\Sigma\delta_{\text{ref}}) < \text{gap}_\tau(\mathfrak{B}_{\text{ref}}).$$

- (xviii) *Reflection-interface sovereignty forbids Mirror, categorical, derived, Fukaya, motivic, or higher-categorical meaning from overriding verifier-side Core requirements.*
- (xix) *Mirror directionality, variance, and involution status must be declared; the word “mirror” does not imply bidirectional equivalence.*
- (xx) *Categorical equivalence strength and realization-faithfulness are feature-relative and must be proved for the Core-visible feature consumed.*
- (xxi) *Extension-readable objects and one-parameter reductions require declared extraction routes and defect treatment before Core use.*
- (xxii) *Source-strict or source-equivalent comparison is not Core exact mode without realized exact comparison.*
- (xxiii) *Controlled reflection comparison is not metric-gap certification without a certified bottleneck bound and two-sided threshold-gap record.*
- (xxiv) *Reflection metric protection is measured on realized windowed pre-threshold profiles.*
- (xxv) *Type IV claims for reflection interfaces require artifact-coherent records; changing artifacts changes the diagnostic claim.*
- (xxvi) *External theorem promotion requires a separate reflection or Core-to-reflection transport theorem.*

(xxvii) *Reflection ledger components require namespace registration, alias/refinement control, and double-count prevention.*

(xxviii) *Reflection ledger slack cannot compensate for missing non-scalar obligations.*

Proof. Items (i)–(ii) follow from the definitions of Mirror Reflection and categorical comparison settings. Item (iii) is Proposition 3.31. Item (iv) follows from the repaired Core evaluation discipline. Item (v) is Proposition 3.42 together with Definition 3.45 and the categorical ledger rule. Item (vi) is Proposition 3.56. Item (vii) is Proposition 3.66. Item (viii) follows from Propositions 3.57, 3.67, and 3.51. Item (ix) is Proposition 3.70. Item (x) is Proposition 3.75. Item (xi) follows from Definition 3.74. Items (xii)–(xiii) follow from Definition 3.83 and Proposition 3.84. Item (xiv) is Proposition 3.93. Item (xv) is Proposition 3.98. Item (xvi) is Proposition 3.102. Item (xvii) is Proposition 3.105. The additional sovereignty, directionality, equivalence-strength, extraction, comparison-mode, metric-record, artifact-coherence, theorem-promotion, and ledger-namespace clauses follow respectively from Definitions 3.4, 3.14, 3.23, 3.34, 3.44, 3.59, 3.77, 3.86, and Definition 3.107, together with their associated propositions. $\square$

MS-C.17. Explicit exclusions

The following are not asserted in Appendix MS-C.

- No mirror symmetry theorem is proved.
- No categorical equivalence theorem is proved.
- No derived equivalence theorem is proved.
- No motivic equivalence theorem is proved.
- No Langlands correspondence is proved.
- No modularity theorem is proved.
- No automorphy theorem is proved.
- No homological mirror symmetry theorem is proved.
- No Fukaya-categorical theorem is proved.
- No Mirror Reflection operation is assumed to be an involution or equivalence by default.
- No categorical comparison is assumed to be fully faithful, essentially surjective, conservative, or monoidal by default.
- No Mirror/categorical comparison replaces B-Gate⁺.
- No Mirror/categorical comparison replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No Mirror/categorical comparison replaces eligibility.
- No Mirror/categorical comparison replaces monitored Type IV diagnostics.

- No Mirror/categorical comparison replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No Mirror/categorical comparison replaces the scalarized budget condition

$$\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

- No source-level strict or zero-cost comparison is automatically Core mode exact.
- No reflection transport of a tower artifact is consumed without a declared $\Delta_{\phi,\text{ref}}$ or a declared alias/refinement relation to the Appendix L artifact-transport ledger.

- No Mirror/categorical Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No Core pass is interpreted as a Mirror or categorical theorem without a separate transport theorem.
- No advisory Mirror or categorical diagram supports Core acceptance.
- No global exactness of T_τ^{bar} is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift C_τ is assumed.
- No source-side strict equality, equivalence, naturality, adjunction, or coherence cell is automatically a Core exact comparison.
- No categorical cost, Mirror discrepancy, homotopy size, or energy estimate is automatically a metric-gap-certified comparison.
- No extension-readable derived, sheaf, stack, Fukaya, or higher-categorical object is automatically Core-readable.
- No one-parameter reduction of Mirror or categorical data is assumed lossless by default.
- No categorical equivalence is assumed to preserve the filtration, windowed barcode profile, threshold-gap certificate, eligible amplitude, or tower artifact unless proved.
- No Mirror or categorical proxy determines Type IV status without the declared tower comparison artifact and terminal-tameness evidence.
- No artifact switch inside a claimed reflection Type IV comparison or stable range is silent; changing the artifact creates a new diagnostic claim.
- No successful Core certificate promotes a Mirror or categorical implication from [Spec] to theorem status without a registered transport theorem.
- No reflection ledger slack repairs a missing realization, extraction route, eligibility record, tower artifact, terminal-tameness record, or transport theorem.
- No reflection defect may be double-counted under multiple names without an alias, refinement, or disjointness declaration.

MS-C.18. Provenance and status

The material in this appendix depends on:

- (i) the arithmetic interface meta-theory of Appendix MS-A;
- (ii) the Mirror/Transfer 2-cell ledger of Appendix L;
- (iii) the derived, sheaf, stack, and higher-categorical extension discipline of Appendix N;
- (iv) the repaired barcode-level collapse and constructible persistence framework of Appendix A;
- (v) the admissible representative discipline of Appendix B;
- (vi) the eligible realization regime of Appendix C;
- (vii) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;
- (viii) the manifest schema of Appendix G;
- (ix) the ledger catalog of Appendix M;
- (x) the interface protocol of Chapter 8;
- (xi) the non-claim discipline of Chapter 7.

Its role is to provide a disciplined interface for Mirror Reflection and categorical comparison without converting such structures into Core or problem-specific theorems.

Status labels for Appendix MS-C. *Problem-interface definitions:* Definitions [3.9](#), [3.10](#), [3.11](#), [3.19](#), [3.20](#), [3.21](#), [3.28](#), [3.29](#), [3.30](#), [3.38](#), [3.39](#), [3.40](#), [3.41](#), [3.48](#), [3.49](#), [3.50](#), [3.53](#), [3.54](#), [3.55](#), [3.63](#), [3.64](#), [3.65](#), [3.69](#), [3.72](#), [3.73](#), [3.74](#), [3.80](#), [3.81](#), [3.82](#), [3.83](#), [3.90](#), [3.91](#), [3.92](#), [3.95](#), [3.96](#), [3.97](#), [3.101](#), [3.103](#), [3.104](#).

Additional v19.0 hardening definitions: Definitions [3.4](#), [3.5](#), [3.14](#), [3.15](#), [3.23](#), [3.24](#), [3.33](#), [3.34](#), [3.44](#), [3.45](#), [3.59](#), [3.77](#), [3.86](#), [3.87](#), [3.107](#), [3.108](#).

Problem-interface propositions/theorems: Propositions [3.31](#), [3.42](#), [3.51](#), [3.56](#), [3.57](#), [3.66](#), [3.67](#), [3.70](#), [3.75](#), [3.84](#), [3.93](#), [3.98](#), [3.102](#), [3.105](#), Theorem [3.112](#).

Additional v19.0 hardening propositions: Propositions [3.6](#), [3.7](#), [3.16](#), [3.17](#), [3.25](#), [3.26](#), [3.35](#), [3.36](#), [3.46](#), [3.47](#), [3.60](#), [3.61](#), [3.78](#), [3.88](#), [3.89](#), [3.109](#), [3.110](#).

Operational cautions: Remarks [3.1](#), [3.2](#), [3.3](#), [3.12](#), [3.13](#), [3.22](#), [3.32](#), [3.43](#), [3.52](#), [3.58](#), [3.68](#), [3.71](#), [3.76](#), [3.85](#), [3.94](#), [3.99](#), [3.100](#), [3.106](#).

Additional v19.0 hardening remarks: Remarks [3.8](#), [3.18](#), [3.27](#), [3.37](#), [3.62](#), [3.79](#), [3.111](#).

Explicit exclusions: Section MS-C.17.

4 Appendix MS-F: Fukaya-Categorical Extension

MS-F.1. Scope and interface status

This appendix defines the interface discipline for Fukaya-categorical and Floer-theoretic extensions of the AK framework. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Fukaya-categorical structure is external interface data, not a Core theorem.

In particular, this appendix does not prove:

homological mirror symmetry, Fukaya-category equivalence,
 Floer-theoretic unobstructedness, symplectic classification,
 Fukaya–Seidel equivalence, microlocal sheaf equivalence,

or any arithmetic, geometric, symplectic, or categorical theorem.

All Core-relevant persistence data produced by a Fukaya-categorical interface must be evaluated through the repaired windowed profile:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Whenever tower diagnostics are invoked, the required comparison is a declared tower comparison artifact:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

Remark 4.1 (Interface status). All Fukaya-categorical implications are [Spec] by default unless separately promoted and proved under the interface protocol of Chapter 8, the Mirror/categorical discipline of Appendix MS-C, and the higher-categorical discipline of Appendix N.

Remark 4.2 (Relation to Appendix MS-C). Appendix MS-C defined the general Mirror Reflection and categorical comparison interface. The present appendix specializes that interface to symplectic, Floer-theoretic, Fukaya-categorical, and microlocal-sheaf-adjacent structures. It is a controlled problem-interface extension, not a Fukaya-theoretic theorem package.

Remark 4.3 (No exact-collapse restoration). This appendix does not restore the old exact-collapse language. Fukaya-side persistence data must be evaluated through

$$\text{Eval}_{i,W,\tau}$$

and barcode-level threshold deletion

$$T_{\tau}^{\text{bar}}.$$

No global exact collapse functor, positive-threshold Serre localization, or global strict filtered lift C_{τ} is assumed.

MS-F.1bis. Fukaya interface sovereignty

Definition 4.4 (Fukaya interface sovereignty). *Fukaya interface sovereignty* is the rule that Fukaya categories, Floer complexes, symplectic meanings, microlocal models, homological mirror symmetry heuristics, continuation data, and mirror-side categorical artifacts remain outside the AK Core unless they re-enter through declared verifier-side records. The admissible re-entry routes are:

- (i) constructible one-parameter persistence objects in $\text{Pers}_k^{\text{cons}}$;
- (ii) filtered objects in $\text{FiltCh}(()k)$;
- (iii) repaired evaluation records $\text{Eval}_{i,W,\tau}(F)$;
- (iv) admissible representatives $C_{\tau,W}F$;
- (v) eligible realized objects for the invoked Ext^1 -clause;
- (vi) declared tower comparison artifacts $\phi_{i,W,\tau}^{\text{Fuk}}$;

- (vii) monitored Type IV diagnostic records;
- (viii) typed ledger entries, including extraction, perturbation, continuation, model-replacement, mirror-comparison, and tower-artifact transport defects;
- (ix) certificate records satisfying the Minimal Manifest Axiom.

Proposition 4.5 (No Fukaya sovereignty override). *No Fukaya-theoretic theorem, mirror-symmetry assertion, Floer-theoretic construction, microlocal-sheaf model, or categorical equivalence overrides the verifier-side Core requirements.*

Proof. The AK Core consumes typed verifier-side records. Fukaya-side and mirror-side structures have external symplectic and categorical semantics. Unless those structures are realized as Core-readable records, ledger entries, tower artifacts, or manifest data, they are not inputs to the Core verdict. $\square$

Remark 4.6 (Fukaya meaning is proposer-side by default). Fukaya-side meaning may guide the choice of objects, filtrations, perturbations, windows, thresholds, mirror candidates, or tower tests. By default it is proposer-side motivation, not verifier-side evidence.

MS-F.2. Fukaya-categorical source data

Definition 4.7 (Fukaya-categorical source datum). *A Fukaya-categorical source datum is a declared symplectic, Floer-theoretic, or categorical object intended to support a Fukaya-side interface. Typical examples include:*

- (i) a symplectic manifold (X, ω) ;
- (ii) an exact, monotone, or Calabi–Yau symplectic manifold;
- (iii) a Liouville sector or Weinstein manifold;
- (iv) a collection of Lagrangian branes;
- (v) a filtered Floer complex;
- (vi) an A_∞ -category associated to Floer theory;
- (vii) a wrapped or partially wrapped Fukaya category;
- (viii) a Fukaya–Seidel category;
- (ix) an action-filtered or energy-filtered Floer-theoretic object;
- (x) a symplectic or microlocal sheaf model proposed as a Fukaya-side replacement;
- (xi) a mirror-side algebraic, categorical, or sheaf-theoretic candidate paired with Fukaya data.

Definition 4.8 (Fukaya-side category). *A Fukaya-side category is a declared category, dg-category, A_∞ -category, stable category, or higher category*

$$\mathcal{F}(X)$$

associated to a symplectic source datum X , together with all auxiliary choices needed for its use in the interface.

Definition 4.9 (Auxiliary Fukaya data). *Auxiliary Fukaya data include choices such as:*

- (i) coefficient field or ring;
- (ii) grading structure;
- (iii) spin or relative spin structure;
- (iv) brane structure;
- (v) bounding cochains;
- (vi) perturbation data;
- (vii) almost complex structures;
- (viii) Hamiltonian wrapping data;
- (ix) stop or sector data;
- (x) Novikov parameters;
- (xi) compactness assumptions;
- (xii) transversality assumptions;
- (xiii) unobstructedness assumptions;
- (xiv) orientation conventions;
- (xv) filtration conventions.

Definition 4.10 (Fukaya source status). A Fukaya-categorical source datum has one of the following interface statuses:

- (i) *declared*: the datum is specified but not yet realized;
- (ii) *realized*: a Fukaya-side category or complex has been produced;
- (iii) *Core-readable*: a constructible persistence or filtered extraction is declared;
- (iv) *eligible*: the Core Ext¹-eligible regime is verified;
- (v) *tower-readable*: a Core-readable tower and tower comparison artifact are declared;
- (vi) *advisory*: the datum is used only for guidance or search;
- (vii) *failed*: required auxiliary, realization, eligibility, artifact, or ledger data fail.

Remark 4.11 (Auxiliary data are not optional). Fukaya-categorical objects are not determined by a symplectic manifold alone. If such objects are used in a certificate interface, the auxiliary data and their status must be declared. Undeclared auxiliary choices are not audit-readable.

Remark 4.12 (Source status is not Core status). A Fukaya source may be declared, realized, or even Fukaya-theoretically meaningful without being Core-readable. Core status begins only after repaired persistence data, admissible representatives, eligibility records, tower artifacts, diagnostics, ledgers, and gate verdicts enter the certificate record.

MS-F.2bis. Auxiliary-data audit discipline

Definition 4.13 (Auxiliary data package). An *auxiliary data package* for a Fukaya interface is the complete declared collection of choices and hypotheses needed to make the source datum audit-readable. It records at least:

- (i) coefficient and Novikov conventions;
- (ii) grading, spin, brane, and orientation data;
- (iii) perturbation and transversality data;
- (iv) compactness and bubbling-control hypotheses;
- (v) bounding-cochain or unobstructedness choices;
- (vi) Hamiltonian, stop, wrapping, or sector data;
- (vii) filtration and normalization conventions;
- (viii) artifact references and failure modes.

Definition 4.14 (Source-status transition record). A *source-status transition record* records the transition from one Fukaya source status to another, for example from *declared* to *realized*, from *realized* to *Core-readable*, or from *Core-readable* to *eligible*. Each transition must declare the data, theorem, computation, or artifact that justifies the transition.

Proposition 4.15 (Source-status transitions are not automatic). *No transition among the statuses declared, realized, Core-readable, eligible, tower-readable, advisory, and failed is automatic.*

Proof. Each status has a different verifier-side meaning. A realized Fukaya category need not provide a constructible persistence extraction, a Core-readable extraction need not satisfy the eligible regime, and a tower of Fukaya-side data need not provide a declared tower comparison artifact. Therefore status changes require explicit transition records. $\square$

Remark 4.16 (Undeclared auxiliary choices are interface failures). If a certificate depends on a Fukaya-side construction but omits auxiliary choices needed to reconstruct that construction, the affected interface obligation is undefined rather than silently accepted.

MS-F.3. Fukaya realization targets

Definition 4.17 (Fukaya realization target). A Fukaya-categorical interface must declare its realization target as one of:

- (i) a filtered Floer complex;
- (ii) a filtered object in $\text{FiltCh}(\cdot)k$;
- (iii) a constructible persistence object in $\text{Pers}_k^{\text{cons}}$;
- (iv) a repaired windowed collapsed profile

$$\text{Eval}_{i,w,\tau}(F);$$

- (v) an admissible representative

$$C_{\tau,w}F;$$

(vi) a realized eligible object in $D^{[-1,0]}(k\text{-mod})$;

(vii) a declared tower comparison artifact

$$\phi_{i,W,\tau};$$

(viii) a dg-category or A_∞ -category in a Technical Extension setting;

(ix) a sheaf-theoretic or microlocal model;

(x) a certificate-record component;

(xi) an advisory artifact.

Definition 4.18 (Core-readable Fukaya realization). A Fukaya-categorical realization is *Core-readable* if it supplies a filtered or persistence object on which the AK Core can evaluate:

$$\text{Eval}_{i,W,\tau}(-) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(-)),$$

or if it supplies verifier-side data directly to a certificate record.

Definition 4.19 (Fukaya extraction route). A *Fukaya extraction route* is a declared construction taking Fukaya-side data to one of:

$$\text{FiltCh}(()k), \quad \text{Pers}_k^{\text{cons}}, \quad \text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$

or a ledger component.

Proposition 4.20 (No Core use without Core-readable Fukaya realization). A *Fukaya-categorical source datum cannot support Core certification unless it has a declared Core-readable realization or extraction route to verifier-side data.*

Proof. The Core verifier evaluates repaired windows, thresholds, persistence objects, admissible representatives, diagnostics, ledgers, gate verdicts, and artifact references. A raw Fukaya category or Floer-theoretic object is not automatically one of these. Therefore a Core-readable realization or extraction route is required. $\square$

Remark 4.21 (Fukaya category is not automatically persistence). A Fukaya category may contain filtered Floer complexes, action filtrations, grading data, and energy estimates, but it is not automatically a one-parameter constructible persistence module. The persistence extraction must be declared explicitly.

MS-F.3bis. One-parameter filtration and normalization discipline

Definition 4.22 (Fukaya one-parameter filtration policy). A *Fukaya one-parameter filtration policy* is a declared rule converting Fukaya-side or Floer-side data into a single ordered persistence parameter compatible with $\text{Pers}_k^{\text{cons}}$. It must specify:

- (i) the source quantity, such as action, energy, area, Maslov-index-filtered level, wrapping depth, stop depth, or sheaf-theoretic scale;
- (ii) the order convention and normalization;
- (iii) the treatment of ties, non-comparable data, and reparametrizations;
- (iv) the declared windows and monitored degrees;

- (v) the information discarded by the one-parameter extraction;
- (vi) the defect introduced by the extraction.

Definition 4.23 (Filtration normalization defect). A *filtration normalization defect* is a declared discrepancy

$$\delta^{\text{Fuk-norm}}$$

introduced by choosing an action, energy, wrapping, sector, or microlocal scale normalization in order to obtain a one-parameter persistence object.

Proposition 4.24 (Action or energy is not a Core parameter by name). *An action filtration, energy filtration, or wrapping-depth filtration is not a Core-admissible persistence parameter merely by name.*

Proof. Core admissibility requires a constructible one-parameter persistence object on the declared windows. An action or energy quantity may be geometrically meaningful, but it must still be normalized, ordered, shown constructible on the relevant windows, and connected to repaired evaluation records. Otherwise it has no verifier-side persistence type. $\square$

Remark 4.25 (Filtration reparametrization risk). Changing an action scale, energy normalization, Novikov valuation convention, or wrapping parameter may change bar lengths and threshold comparisons. Such changes are exact only when proved exact for the realized profiles; otherwise the discrepancy must be ledgered.

MS-F.4. Persistence extraction from Fukaya data

Definition 4.26 (Fukaya persistence extraction). A *Fukaya persistence extraction* is a declared procedure

$$\mathfrak{E}_{\text{Fuk}} : \mathcal{F}(X) \longrightarrow \text{Pers}_k^{\text{cons}}$$

or

$$\mathfrak{E}_{\text{Fuk}} : \mathcal{F}(X) \longrightarrow \text{FiltCh}(()k),$$

which turns Fukaya-side data into Core-readable persistence or filtered objects.

Definition 4.27 (Action-filtration extraction). An *action-filtration extraction* is a Fukaya persistence extraction in which the persistence parameter is derived from the Floer action filtration or a declared action-like functional.

Definition 4.28 (Energy-filtration extraction). An *energy-filtration extraction* is a Fukaya persistence extraction in which the persistence parameter is derived from energy, area, or a declared symplectic energy functional.

Definition 4.29 (Microlocal-sheaf extraction). A *microlocal-sheaf extraction* is a Fukaya persistence extraction mediated by a sheaf-theoretic or microlocal sheaf model, followed by extraction into constructible persistence or filtered data.

Definition 4.30 (Fukaya extraction defect). A *Fukaya extraction defect* is a declared discrepancy

$$\delta^{\text{Fuk-ext}}$$

arising from action filtration, energy filtration, microlocal extraction, discretization, truncation, continuation, perturbation, or model replacement in the extraction route.

Proposition 4.31 (Extraction must preserve constructibility). *A Fukaya persistence extraction is Core-admissible only if the resulting persistence objects lie in $\text{Pers}_k^{\text{cons}}$ on the declared windows.*

Proof. The repaired Core operates in the constructible one-parameter persistence regime. If the extracted object is not constructible on the declared windows, then the repaired evaluation

$$\text{Eval}_{i,W,\tau}$$

is not Core-readable in the required sense. $\square$

Proposition 4.32 (Fukaya extraction defects must be ledgered). *If a certificate relies on a Fukaya extraction route with nonzero extraction defect, then*

$$\delta^{\text{Fuk-ext}}$$

must be included in the declared δ -ledger.

Proof. A nonzero extraction defect changes the data entering the repaired Core evaluation or certificate record. By audit discipline, such discrepancies affect acceptance only through the ledger. Therefore the defect must enter $\Sigma\delta$. $\square$

Remark 4.33 (Extraction is an interface hypothesis). A Fukaya persistence extraction is not automatic. It is an interface hypothesis or construction. Its correctness and purpose-faithfulness must be declared.

MS-F.4bis. Continuation, perturbation, and model-replacement defects

Definition 4.34 (Continuation defect). A *continuation defect* is a declared discrepancy

$$\delta^{\text{Fuk-cont}}$$

arising from replacing one Fukaya-side or Floer-side datum by another through continuation maps, Hamiltonian deformation, wrapping changes, stop removal, or continuation-type comparison.

Definition 4.35 (Perturbation defect). A *perturbation defect* is a declared discrepancy

$$\delta^{\text{Fuk-pert}}$$

arising from perturbation choices, transversality perturbations, almost-complex-structure choices, Hamiltonian perturbations, or auxiliary analytic choices used before extracting Core-readable data.

Definition 4.36 (Model-replacement defect). A *model-replacement defect* is a declared discrepancy

$$\delta^{\text{Fuk-model}}$$

arising when a Fukaya-side object is replaced by a dg model, sheaf model, microlocal model, mirror-side model, finite-dimensional model, or computational model before Core-readable extraction.

Proposition 4.37 (Continuation map is not tower comparison artifact). *A Fukaya-side continuation map is not a Core tower comparison artifact $\phi_{i,W,\tau}^{\text{Fuk}}$ unless it is realized as a comparison between the declared tower-side and apex-side repaired profiles.*

Proof. Continuation maps live in the Fukaya-side or Floer-side source structure. Type IV diagnostics consume the kernel and cokernel defect profiles of a declared Core-readable tower comparison artifact. A continuation map becomes such an artifact only after the required persistence extraction and profile-level comparison are supplied. $\square$

Proposition 4.38 (Model equivalence is not persistence-profile equality). *A quasi-equivalence, Morita equivalence, sheaf equivalence, mirror-model equivalence, or dg-model replacement is not automatically equality or isomorphism of repaired persistence profiles.*

Proof. Categorical equivalence and persistence-profile comparison have different types. A categorical replacement may preserve categorical structure while changing filtrations, bar lengths, windows, or threshold behavior. Equality or metric control of repaired profiles requires a separate realized comparison record. $\square$

Remark 4.39 (Continuation and perturbation must be ledgered when consumed). Continuation, perturbation, and model-replacement defects may be harmless, zero, controlled, or unavailable. If a certificate consumes them and they are nonzero or only bounded, they must enter the declared ledger before the budget comparison is evaluated.

MS-F.5. Fukaya-side obstructions and Core obstructions

Definition 4.40 (Fukaya-side obstruction). *A Fukaya-side obstruction is an obstruction arising in the construction, definition, or comparison of Fukaya-categorical objects. Examples include:*

- (i) obstruction to defining the Floer differential;
- (ii) obstruction to unobstructedness of a Lagrangian brane;
- (iii) obstruction from disk bubbling;
- (iv) transversality failure;
- (v) compactness failure;
- (vi) failure of A_∞ -relations;
- (vii) failure of functorial comparison;
- (viii) failure of mirror-side matching;
- (ix) uncontrolled continuation defect;
- (x) failure of action-filtration compatibility.

Definition 4.41 (Core obstruction). *A Core obstruction is one of the obstruction types defined by the AK Core, including:*

$$\begin{aligned} \text{Eval}_{1,W,\tau}(F) \text{ eq } 0, \quad \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) \text{ eq } 0 \quad \text{in the eligible regime,} \\ (\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0), \quad \text{val}_B(\Sigma\delta) \geq \text{gap}_\tau(\mathfrak{B}). \end{aligned}$$

Proposition 4.42 (Fukaya-side obstruction is not Core obstruction by default). *A Fukaya-side obstruction is not automatically a Core obstruction. Conversely, a Core obstruction is not automatically a Fukaya-side obstruction.*

Proof. Fukaya-side obstructions arise in symplectic and A_∞ -categorical constructions. Core obstructions are defined through repaired persistence, eligible realization, tower diagnostics, declared artifacts, and budget comparison. They belong to different structures unless an interface theorem identifies them. $\square$

Remark 4.43 (Obstruction transport is [Spec]). Any statement transporting Fukaya-side obstructions to Core obstructions, or Core obstructions to Fukaya-side obstructions, is [Spec] by default.

MS-F.5bis. Obstruction namespace separation

Definition 4.44 (Fukaya obstruction namespace). A *Fukaya obstruction namespace* records whether an obstruction belongs to Fukaya construction, Floer theory, analytic compactness, transversality, A_∞ -relations, mirror comparison, Core persistent homology, Core Ext^1 , Type IV diagnostics, or metric budget. Namespace declaration is mandatory whenever an obstruction symbol is consumed by a certificate.

Proposition 4.45 (Unobstructedness is not Core vanishing). *Fukaya-side unobstructedness does not imply*

$$\text{Eval}_{1,W,\tau}(F) = 0, \quad \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0),$$

by default.

Proof. Fukaya-side unobstructedness concerns the construction or deformation theory of Fukaya-side objects. The displayed conditions are verifier-side Core statements about repaired persistence, eligible realized objects, and tower comparison artifacts. A transport theorem is required to connect the namespaces. $\square$

Proposition 4.46 (Core obstruction is not Fukaya obstruction). *A Core obstruction does not imply disk bubbling, transversality failure, compactness failure, A_∞ -failure, or Fukaya-side non-unobstructedness unless a separate transport theorem proves that implication.*

Proof. Core obstructions and Fukaya obstructions live in different semantic namespaces. The Core obstruction may reflect a failure of a chosen extraction, threshold, tower comparison, or budget, rather than a Fukaya-theoretic construction failure. Therefore the implication is not automatic. $\square$

Remark 4.47 (Obstruction names are not evidence). Words such as “obstruction”, “unobstructed”, “vanishing”, and “defect” are overloaded across Fukaya theory and the AK Core. A certificate must use decorated names or explicit namespace declarations.

MS-F.6. Fukaya eligibility for Ext^1 -use

Definition 4.48 (Fukaya eligible regime). A Fukaya-categorical interface is in the *Fukaya eligible regime* on $(F; W, \tau)$ if:

(i) the Fukaya-side realization produces an admissible representative

$$C_{\tau,W}F;$$

(ii) for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \text{Eval}_{i,W,\tau}(F);$$

(iii) $\mathcal{R}(C_{\tau,W}F)$ is defined in $D^b(k\text{-mod})$;

(iv) one has

$$\mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod});$$

(v) the edge-realization condition holds:

$$H^{-1}(\mathcal{R}(C_{\tau,W}F)) \cong H_1^{\tau,W}(F).$$

Proposition 4.49 (Fukaya category does not replace eligibility). *The presence of a Fukaya-categorical structure does not replace the eligible realization requirements needed to invoke the Core Ext^1 -clause.*

Proof. The Core Ext^1 -edge theorem is proved in the eligible regime of Appendix C. Fukaya-categorical structure does not automatically imply amplitude $[-1, 0]$, the admissible representative condition, or the required edge-realization condition. Therefore eligibility must be separately verified. $\square$

Remark 4.50 (Floer grading is not Core amplitude). Floer grading data are not automatically the same as the two-term amplitude condition

$$D^{[-1,0]}(k\text{-mod}).$$

Any identification must be declared and proved as part of the interface.

MS-F.6bis. Floer grading and Core Ext^1 hardening

Definition 4.51 (Floer-to-Core amplitude claim). A *Floer-to-Core amplitude claim* is a claim that a graded Floer-theoretic or Fukaya-side object, a truncation of it, or a realized model of it determines an object in

$$D^{[-1,0]}(k\text{-mod})$$

compatible with the Core eligible regime.

Proposition 4.52 (Floer degree is not the Core Ext^1 -edge). A *Floer grading, Maslov grading, cohomological degree, or morphism-space degree is not automatically the Core residual degree-one edge used in the Ext^1 -clause.*

Proof. The Core Ext^1 -edge uses an eligible realized object in amplitude $[-1, 0]$ together with the edge-realization condition relating H^{-1} to $H_1^{\tau, W}(F)$. Floer-side gradings may motivate a candidate construction, but they do not by themselves establish this Core edge condition. $\square$

Proposition 4.53 (Unproved amplitude blocks Fukaya Ext^1 -use). *If the Floer-to-Core amplitude claim is unproved, unavailable, advisory, or merely conjectural, then the Fukaya interface may not invoke the Core Ext^1 -clause.*

Proof. The Ext^1 -clause is available only in the eligible regime. Missing amplitude and edge-realization hypotheses are logical obligations, not metric defects, and cannot be repaired by Fukaya-side intuition or by ledger slack. $\square$

MS-F.7. Fukaya/Mirror comparison

Definition 4.54 (Fukaya/Mirror comparison datum). A *Fukaya/Mirror comparison datum* is a tuple

$$(\mathcal{F}(X), C^\vee, \text{Mir}_F, \Theta_F),$$

where:

- (i) $\mathcal{F}(X)$ is a Fukaya-side category;
- (ii) $C^\vee$ is a mirror-side category, algebraic model, sheaf model, or arithmetic/geometric model;
- (iii) Mir_F is a declared Mirror comparison;
- (iv) Θ_F records comparison cells, costs, controlled equivalence data, or advisory comparison data.

Definition 4.55 (Fukaya/Mirror comparison cost). A *Fukaya/Mirror comparison cost* is a ledger-valued quantity

$$\delta^{\text{Fuk-Mir}}$$

recording the defect of using a Fukaya/Mirror comparison in a certificate record.

Definition 4.56 (Fukaya/Mirror adjusted budget). If a certificate uses Fukaya extraction, continuation, perturbation, model-replacement, tower-artifact, or Fukaya/Mirror comparison data, the adjusted budget is the declared ledger aggregate

$$\Sigma\delta_{\text{Fuk}} := \Sigma\delta_{\text{base}} \oplus \delta^{\text{Fuk-ext}} \oplus \delta^{\text{Fuk-norm}} \oplus \delta^{\text{Fuk-cont}} \oplus \delta^{\text{Fuk-pert}} \oplus \delta^{\text{Fuk-model}} \oplus \delta^{\text{Fuk-Mir}} \oplus \Delta_{\phi, \text{Fuk}},$$

with omitted components set to the ledger identity and aggregation interpreted in the declared ledger semantics. The scalar budget comparison uses the declared scalarization

$$\text{val}_B(\Sigma\delta_{\text{Fuk}}) < \text{gap}_\tau(\mathfrak{B}_{\text{Fuk}}).$$

Proposition 4.57 (Fukaya/Mirror comparison costs must be ledgered). *If a certificate record relies on a nonzero Fukaya/Mirror comparison cost, then that cost must be included in the δ -ledger.*

Proof. A nonzero comparison cost is a discrepancy introduced by the interface. By the audit discipline of Chapter 6, such discrepancies affect certification only through the total budget. Therefore the cost must be ledgered. $\square$

Proposition 4.58 (Fukaya/Mirror evidence must pass adjusted budget). *A certificate record relying on nonzero Fukaya extraction, continuation, perturbation, model-replacement, tower-artifact, or Fukaya/Mirror comparison defects may pass Audit Mode only if*

$$\text{val}_B(\Sigma\delta_{\text{Fuk}}) < \text{gap}_\tau(\mathfrak{B}_{\text{Fuk}}).$$

Proof. All nonzero extraction, continuation, perturbation, model-replacement, tower-artifact, and comparison defects used by a certificate must enter the declared total budget. Audit Mode requires the strict scalarized pass condition

$$\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

Thus the Fukaya-adjusted budget must pass. $\square$

Remark 4.59 (No free homological mirror symmetry). A Fukaya/Mirror comparison does not become homological mirror symmetry by being useful in the AK interface. Any HMS claim requires a separate theorem.

MS-F.7bis. Fukaya/Mirror comparison modes

Definition 4.60 (Fukaya/Mirror comparison mode). Every Fukaya/Mirror comparison consumed by an interface record must declare exactly one local comparison mode:

source_equivalence, source_quasi_equivalence, realized_exact, realized_metric, controlled_ledger, advisory_on

Its projection to Core comparison modes follows Appendix MS-C: only realized_exact may project to Core exact, and only realized_metric equipped with a certified bottleneck bound, scalarization route, and two-sided threshold-gap record may project to metric_gap_certified.

Definition 4.61 (Fukaya comparison status–mode compatibility). The status terms of Definition 4.10 and the comparison modes above are compatible only as follows:

- (i) a source-side Fukaya or mirror equivalence remains source_equivalence or source_quasi_equivalence until transported to realized profiles;
- (ii) a strict equality of realized repaired profiles may be recorded as realized_exact;

- (iii) a bounded realized profile discrepancy may be recorded as realized_metric only with the required metric-gap certification data;
- (iv) advisory comparisons remain advisory_only;
- (v) unavailable or failed comparisons cannot support the invoked verifier-side comparison.

Proposition 4.62 (Source equivalence is not Core exactness). *A Fukaya-side equivalence, quasi-equivalence, HMS candidate, or microlocal-sheaf equivalence is not a Core exact comparison unless it induces an actual equality or isomorphism of the declared repaired Core-readable profiles.*

Proof. Source-side equivalence is a categorical statement. Core exactness is a statement about the declared Core-readable objects or repaired profiles. A realization theorem connecting the two comparison types is therefore required. $\square$

Proposition 4.63 (Controlled comparison is not metric certification by name). *Calling a Fukaya/Mirror comparison controlled does not by itself provide metric-gap certification.*

Proof. Metric-gap certification requires a certified profile-level bottleneck bound, a scalarization route, and a two-sided threshold-gap record for the relevant windowed pre-threshold profiles. A controlled Fukaya-side comparison may provide a ledgered defect, but it does not automatically have this metric type. $\square$

Remark 4.64 (Comparison modes delegate to MS-C). Fukaya/Mirror comparison modes are special cases of the Mirror/categorical comparison modes of Appendix MS-C. When both MS-C and MS-F records are present, the stricter verifier-side mode governs.

MS-F.8. Fukaya-side towers and monitored Type IV

Definition 4.65 (Fukaya-side tower). *A Fukaya-side tower is a directed system*

$$\mathcal{F}_0 \longrightarrow \mathcal{F}_1 \longrightarrow \mathcal{F}_2 \longrightarrow \cdots$$

of Fukaya-side categories, filtered Floer complexes, Lagrangian collections, symplectic input data, action-filtered data, or mirror comparison data.

Definition 4.66 (Core-readable Fukaya tower). *A Fukaya-side tower is Core-readable if, after persistence extraction, it yields a directed tower*

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

with a declared apex object F_∞ , such that for each monitored degree i , window W , and threshold τ , the repaired profiles

$$\text{Eval}_{i,W,\tau}(F_n)$$

and

$$\text{Eval}_{i,W,\tau}(F_\infty)$$

are constructible and a tower comparison artifact

$$\phi_{i,W,\tau}^{\text{Fuk}} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

is declared when Type IV diagnostics are invoked.

Definition 4.67 (Fukaya tower comparison artifact). A *Fukaya tower comparison artifact* is a declared tower comparison artifact

$$\phi_{i,W,\tau}^{\text{Fuk}}$$

whose tower-side and apex-side repaired profiles are produced by a Fukaya persistence extraction route.

Definition 4.68 (Fukaya Type IV proxy). A *Fukaya Type IV proxy* is a Fukaya-side signal suggesting possible monitored generic Type IV obstruction. Examples include:

- (i) anomalous persistence of Floer generators;
- (ii) failure of tower stabilization;
- (iii) unexpected growth of action-filtered data;
- (iv) mismatch between Fukaya-side and mirror-side limits;
- (v) uncontrolled bubbling or compactness behavior interpreted as a tower anomaly;
- (vi) non-stabilization of continuation maps;
- (vii) uncontrolled change in extracted barcode profiles.

Definition 4.69 (Actual Fukaya-side Type IV diagnostic). The actual monitored Type IV diagnostic for a Fukaya-side tower is computed only for a declared finite monitoring scope

$$\mathcal{M} \subseteq \{(i, W, \tau)\}$$

and a declared artifact-coherent family of tower comparison artifacts

$$\phi_{i,W,\tau}^{\text{Fuk}} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

For each $(i, W, \tau) \in \mathcal{M}$, terminal-tameness is required and the diagnostic contributions are the window-relative terminal-generic dimensions

$$\text{tgdim}_W \ker(\phi_{i,W,\tau}^{\text{Fuk}}), \quad \text{tgdim}_W \text{coker}(\phi_{i,W,\tau}^{\text{Fuk}}).$$

The reported $(\mu_{\text{Collapse}}, u_{\text{Collapse}})$ is the declared finite-scope sum of these kernel and cokernel contributions. If the artifact, terminal mode, or terminal-tameness record is not available, the diagnostic is undefined rather than assigned $(0, 0)$.

Proposition 4.70 (Fukaya Type IV proxy is not Type IV diagnosis). A *Fukaya Type IV proxy* does not determine whether

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \neq (0, 0).$$

Proof. Monitored Type IV diagnostics are defined from window-relative terminal-generic dimensions of kernel and cokernel defect profiles of declared tower comparison artifacts. A Fukaya-side proxy may suggest possible tower pathology, but it does not compute those defect profiles unless a Core-readable tower and the corresponding artifact

$$\phi_{i,W,\tau}^{\text{Fuk}}$$

are explicitly constructed. □

Remark 4.71 (Correct use of Fukaya proxies). Fukaya-side proxies may guide inspection or candidate selection. They may not replace the computation of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

MS-F.8bis. Artifact-coherent Fukaya Type IV records

Definition 4.72 (Artifact-coherent Fukaya Type IV record). An *artifact-coherent Fukaya Type IV record* is a Type IV record in which the realized tower, apex object, terminal mode, extraction route, monitored finite scope

$$\mathcal{M} \subseteq \{(i, W, \tau)\}$$

and tower comparison artifacts $\phi_{i,W,\tau}^{\text{Fuk}}$ are fixed for the entire diagnostic claim. For each $(i, W, \tau) \in \mathcal{M}$, terminal-tameness and the window-relative terminal-generic dimensions

$$\text{tgdim}_W \ker(\phi_{i,W,\tau}^{\text{Fuk}}), \quad \text{tgdim}_W \text{coker}(\phi_{i,W,\tau}^{\text{Fuk}})$$

must be declared whenever Type IV diagnostics are invoked.

Definition 4.73 (Fukaya tower-artifact transport defect). A *Fukaya tower-artifact transport defect* is a declared discrepancy

$$\Delta_{\phi, \text{Fuk}}$$

arising from transporting, replacing, approximating, or identifying a Fukaya-side tower artifact with the Core-readable tower comparison artifact $\phi_{i,W,\tau}^{\text{Fuk}}$.

Proposition 4.74 (Changing Fukaya Type IV artifacts creates a new claim). *Changing the realized tower, apex object, terminal mode, extraction route, monitored scope, or tower comparison artifact changes the Fukaya Type IV claim unless an explicit artifact-coherence theorem identifies the two records.*

Proof. Type IV diagnostics are computed from the declared artifacts and their window-relative terminal-generic kernel and cokernel defects. Changing those artifacts changes the object being diagnosed. Hence the resulting statement is a new claim unless a theorem identifies the records. $\square$

Remark 4.75 (Terminal-tameness is not optional). If terminal-tameness or the relevant tgdim_W values are unavailable, the monitored Type IV diagnostic is not numerically assigned. In particular, non-invoked or undefined diagnostics are not to be recorded as $(0, 0)$.

MS-F.8ter. Fukaya stable-band discipline

Definition 4.76 (Fukaya advisory stable band). A *Fukaya advisory stable band* is a threshold interval on which Fukaya-side indicators, such as action-filtered generators, continuation behavior, mirror-side matching, or sampled barcode profiles, appear stable under an advisory policy.

Definition 4.77 (Fukaya certified Core stable band). A *Fukaya certified Core stable band* is an artifact-coherent stable band, in the sense of the Appendix D stable-band discipline, on which the realized Fukaya tower diagnostics are certified zero for the fixed tower artifacts, terminal mode, extraction route, monitored scope, and ledger records.

Proposition 4.78 (Sampled Fukaya stability is not a certified Core band). *Observed or sampled stability of Fukaya-side indicators does not define a Fukaya certified Core stable band unless the corresponding artifact-coherent Type IV diagnostics and threshold-gap records are verified throughout the declared band.*

Proof. Advisory stability concerns source-side or sampled behavior. A certified Core stable band concerns verifier-side diagnostics for fixed artifacts and a protected threshold-gap regime. The former does not imply the latter without the required diagnostic and gap records. $\square$

MS-F.9. Fukaya certificate records

Definition 4.79 (Fukaya-categorical certificate record). A *Fukaya-categorical certificate record* is a certificate record whose source data include Fukaya-side structures and whose Core-readable data are obtained through declared Fukaya persistence extraction or related realization.

Definition 4.80 (MS-F manifest entry). An *MS-F manifest entry* records:

- (i) the Fukaya-side source datum;
- (ii) the Fukaya source status;
- (iii) the auxiliary Fukaya data;
- (iv) the realization target;
- (v) the persistence extraction route;
- (vi) declared windows and thresholds;
- (vii) collapse policy;
- (viii) monitored degrees;
- (ix) repaired evaluation records
 $\text{Eval}_{i,w,\tau}(F)$;
- (x) admissible representative data, if invoked;
- (xi) eligibility data, if Ext^1 is invoked;
- (xii) Fukaya-side tower data, if used;
- (xiii) tower comparison artifacts, if Type IV diagnostics are invoked;
- (xiv) Fukaya/Mirror comparison data, if used;
- (xv) extraction, normalization, continuation, perturbation, model-replacement, tower-artifact, and comparison defects;
- (xvi) ledger entries and alias/refinement/disjointness declarations for primitive discrepancies;
- (xvii) monitored Type IV diagnostics;
- (xviii) Core gate verdicts;
- (xix) artifact references;
- (xx) status of all Fukaya-side transport statements.

Proposition 4.81 (Fukaya certificate records must satisfy Minimal Manifest). *Every Fukaya-categorical certificate record must satisfy the Minimal Manifest Axiom of Appendix G.*

Proof. Fukaya-categorical source data do not exempt a certificate from Core auditability. The Minimal Manifest requires declared objects, windows, thresholds, collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledger policy, diagnostics, tower artifacts when invoked, gate verdicts, artifact references, and final status. Therefore MS-F records must satisfy those requirements. $\square$

Remark 4.82 (Fukaya artifacts). Fukaya artifacts may include Floer complexes, action filtrations, A_∞ -structure maps, perturbation data, brane data, continuation maps, compactness assumptions, comparison functors, mirror-side models, extraction scripts, repaired evaluation records, tower comparison artifacts, ledger records, and formal proof traces. They support reconstruction but do not themselves determine Core pass.

MS-F.10. Fukaya transport statements

Definition 4.83 (Fukaya-to-Core transport statement). A *Fukaya-to-Core transport statement* is a statement of the form:

$$\text{Fukaya-side structure} \implies \text{Core-visible behavior}.$$

Definition 4.84 (Core-to-Fukaya transport statement). A *Core-to-Fukaya transport statement* is a statement of the form:

$$\text{Core certificate behavior} \implies \text{Fukaya-side conclusion}.$$

Definition 4.85 (Fukaya transport theorem). A *Fukaya transport theorem* is a Fukaya-to-Core or Core-to-Fukaya transport statement supplied with:

- (i) declared Fukaya-side source data;
- (ii) auxiliary Fukaya data;
- (iii) declared realization target;
- (iv) persistence extraction route;
- (v) repaired evaluation records;
- (vi) admissible representative data when invoked;
- (vii) eligibility data when Ext^1 is invoked;
- (viii) tower comparison artifacts when Type IV diagnostics are invoked;
- (ix) all extraction, comparison, continuation, and mirror defects;
- (x) ledger entries and artifact references;
- (xi) a proof of the stated transport direction.

Definition 4.86 (Default status of Fukaya transport). Every Fukaya transport statement is classified as [Spec] by default unless separately proved under the declared interface protocol.

Proposition 4.87 (Fukaya transport does not enlarge the Core). A *Fukaya transport theorem does not enlarge the Core mathematical package unless explicitly imported and reproved under the Core guard-rails*.

Proof. Fukaya transport statements use external symplectic and categorical semantics. By Chapter 7 and Chapter 8, such problem-specific or extension-specific material remains outside the Core unless explicitly imported and reproved. $\square$

Remark 4.88 (Transport directions are distinct). The directions

$$\text{Fukaya-side structure} \implies \text{Core behavior}$$

and

$$\text{Core behavior} \implies \text{Fukaya-side conclusion}$$

are logically distinct. Neither direction is automatic.

MS-F.10bis. Fukaya theorem promotion schema

Definition 4.89 (Fukaya theorem promotion). *Fukaya theorem promotion* is the operation of changing a Fukaya-side, Floer-theoretic, symplectic, microlocal-sheaf, or homological-mirror-symmetry implication from [Spec] or bridge-candidate status to theorem status. Promotion requires a proved Fukaya transport theorem and updated claim-register status.

Proposition 4.90 (No HMS promotion by Core pass). *A successful Core certificate for Fukaya data does not prove homological mirror symmetry, a Fukaya-category equivalence, a microlocal-sheaf equivalence, or a Floer-theoretic theorem by itself.*

Proof. The Core certificate verifies Core-visible persistence, diagnostic, artifact, and budget conditions. The listed conclusions concern external symplectic or categorical semantics. A separate transport theorem is required to connect the verifier-side certificate to those conclusions. $\square$

Proposition 4.91 (Fukaya transport hypotheses are non-scalar). *Missing Fukaya transport hypotheses cannot be repaired by metric slack, advisory mirror evidence, sampled stable bands, or successful source-side construction.*

Proof. Transport hypotheses determine whether the Core-visible statement has the intended Fukaya-side meaning. They are logical and semantic obligations, not numerical perturbation components. Therefore they cannot be discharged by ledger slack or advisory indicators. $\square$

MS-F.11. Fukaya pass, Core pass, and Fukaya theorem

Definition 4.92 (Fukaya interface pass). *A Fukaya interface pass* means that Fukaya-side data have been successfully realized into Core-readable or ledger-compatible form under the declared interface protocol.

Definition 4.93 (Core pass for Fukaya data). *A Core pass for Fukaya data* means that the corresponding certificate record passes B-Gate⁺, monitored Type IV diagnostics, required artifact checks, and the budget condition.

Definition 4.94 (Fukaya-side theorem). *A Fukaya-side theorem* is a problem-specific statement about Fukaya categories, Floer theory, symplectic geometry, mirror symmetry, microlocal sheaves, or A_∞ -categorical structure.

Proposition 4.95 (Core pass is not Fukaya theorem). *A Core pass for Fukaya data does not imply a Fukaya-side theorem unless a separate transport theorem proves that implication.*

Proof. A Core pass certifies Core-visible persistence, diagnostic, artifact, and budget conditions. A Fukaya-side theorem concerns external symplectic or categorical semantics. A transport theorem is required to connect them. $\square$

Remark 4.96 (Three-layer distinction). The following must remain distinct:

Fukaya interface pass, Core pass, Fukaya-side theorem.

Confusing these layers would violate the non-claim discipline of Chapter 7.

MS-F.12. Relation to the MS appendices

Definition 4.97 (MS interface closure). The *MS interface closure* is the requirement that every arithmetic, Iwasawa, congruence, Mirror/categorical, or Fukaya-side interface remain subordinate to:

$$\text{Eval}_{i,W,\tau}, \quad C_{\tau,W,F}, \quad \phi_{i,W,\tau}, \quad \text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}), \quad [\text{Spec}] \text{ by default.}$$

Proposition 4.98 (MS-F closes but does not upgrade the MS layer). *Appendix MS-F serves as the terminal closure of the released MS problem-interface stack in this revision unless a further MS appendix is explicitly declared; in either case, it does not upgrade any MS interface statement into a Core theorem or a problem-specific theorem.*

Proof. Appendix MS-F is itself a Problem Interface appendix. It introduces Fukaya-side interface discipline and may serve as the terminal closure of the released MS stack, but it does not reprove any Core theorem or any external arithmetic, mirror, symplectic, or categorical theorem. Therefore it can close the layer structurally without upgrading its mathematical status. $\square$

Remark 4.99 (End of MS layer). For the released MS stack ending at Appendix MS-F, the MS appendices provide a disciplined interface stack:

arithmetic meta-theory $\longrightarrow$ Iwasawa/congruence interfaces $\longrightarrow$ Mirror/categorical comparison $\longrightarrow$ Fukaya-categorical extensions

Each level is useful only insofar as it can return to the repaired Core verifier through manifest-readable data.

MS-F.12bis. Fukaya ledger namespace and closure discipline

Definition 4.100 (Fukaya primitive discrepancy). A *Fukaya primitive discrepancy* is a discrepancy source before aggregation. Primitive discrepancies include, when invoked:

$$\delta^{\text{Fuk-ext}}, \quad \delta^{\text{Fuk-norm}}, \quad \delta^{\text{Fuk-cont}}, \quad \delta^{\text{Fuk-pert}}, \quad \delta^{\text{Fuk-model}}, \quad \delta^{\text{Fuk-Mir}}, \quad \Delta_{\phi,\text{Fuk}}.$$

Each primitive discrepancy must be declared as an alias, refinement, aggregate, or disjoint component relative to the Appendix MS-C reflection-comparison namespace, the Appendix L Mirror/Transfer namespace, and the Appendix M ledger namespace whenever any of those namespaces are also present.

Proposition 4.101 (No double counting of Fukaya defects). *The same primitive discrepancy may not be counted twice under different Fukaya, Mirror/Transfer, or general ledger names in a single certificate record.*

Proof. The budget comparison is meaningful only when ledger components have declared aggregation semantics. If the same primitive discrepancy is counted twice without an alias or refinement declaration, the ledger no longer represents the intended total defect. Therefore alias, refinement, aggregate, or disjointness status is required. $\square$

Remark 4.102 (Ledger-catalog registration for MS-F). The symbols $\delta^{\text{Fuk-ext}}$, $\delta^{\text{Fuk-norm}}$, $\delta^{\text{Fuk-cont}}$, $\delta^{\text{Fuk-pert}}$, $\delta^{\text{Fuk-model}}$, $\delta^{\text{Fuk-Mir}}$, and $\Delta_{\phi,\text{Fuk}}$ are interface-layer ledger names. Before they are consumed by a final certificate, the ledger catalog must specify whether each is a new component, an alias, a refinement, an aggregate, or a disjoint component relative to the Appendix MS-C reflection namespace δ^{Mir} , δ^{cat} , $\Delta_{\text{Eval,ref}}$, $\Delta_{T,\text{ref}}$, $\Delta_{\phi,\text{ref}}$, $\delta^{\text{ref-extract}}$, the Appendix L Mirror/Transfer namespace, and the Appendix M ledger namespace. In particular, $\delta^{\text{Fuk-Mir}}$ should normally be declared as a Fukaya-side refinement of δ^{Mir} when both are used, and $\Delta_{\phi,\text{Fuk}}$ should be related either to Appendix MS-C's $\Delta_{\phi,\text{ref}}$ or to the Appendix L tower-artifact transport component. Undeclared overlap is not admissible, since it creates double counting or silent omission.

MS-F.13. Summary of Fukaya-categorical extension

Theorem 4.103 (Appendix MS-F interface package). *Within the Problem Interface layer, the following package is available.*

- (i) *Fukaya-categorical source data are external interface inputs, not Core objects by default.*
- (ii) *Auxiliary Fukaya data must be declared for audit readability.*
- (iii) *A Fukaya realization target must be declared before Core use.*
- (iv) *Fukaya-side data become Core-readable only through declared persistence extraction or verifier-side certificate data.*
- (v) *All Core-relevant Fukaya-side persistence data are evaluated through*

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

- (vi) *Fukaya extraction defects must be ledgered if used in certification.*
- (vii) *Fukaya-side obstructions are not Core obstructions by default.*
- (viii) *Fukaya eligibility for Ext¹-use must satisfy the same eligible regime as Appendix C.*
- (ix) *Fukaya/Mirror comparison costs must be ledgered if used in certification.*
- (x) *Fukaya-side towers require declared tower comparison artifacts*

$$\phi_{i,W,\tau}^{\text{Fuk}}$$

before Type IV diagnostics may be invoked.

- (xi) *Fukaya Type IV proxies do not replace monitored Type IV diagnostics.*
- (xii) *Fukaya certificate records must satisfy the Minimal Manifest Axiom.*
- (xiii) *Fukaya transport statements are [Spec] by default.*
- (xiv) *Core pass for Fukaya data does not imply a Fukaya-side theorem without a separate transport theorem.*
- (xv) *Appendix MS-F may serve as the terminal closure of the released MS problem-interface stack without upgrading it into Core theorem status.*
- (xvi) *Fukaya interface sovereignty prevents Fukaya-side meanings, HMS candidates, and microlocal models from overriding verifier-side Core requirements.*
- (xvii) *Auxiliary-data packages and source-status transition records are required whenever a Fukaya-side construction is consumed by the interface.*
- (xviii) *Action, energy, wrapping, and microlocal scales require a declared one-parameter filtration policy before entering the constructible persistence regime.*
- (xix) *Continuation, perturbation, model-replacement, and tower-artifact transport defects must be typed and ledgered when consumed.*

- (xx) *Fukaya-side unobstructedness, Floer grading, and source equivalence do not imply Core vanishing, Core amplitude, Core exactness, metric certification, or Type IV cleanliness by default.*
- (xxi) *Sampled Fukaya stability is not a certified Core stable band without artifact-coherent diagnostics and threshold-gap records.*
- (xxii) *Fukaya theorem promotion requires a separate transport theorem and cannot be obtained by Core pass alone.*

Proof. Items (i)–(iv) follow from the definitions of Fukaya source data, realization targets, and persistence extraction. Item (v) follows from the repaired Core evaluation discipline. Item (vi) is Proposition 4.32. Item (vii) is Proposition 4.42. Item (viii) is Proposition 4.49. Item (ix) is Proposition 4.57. Item (x) follows from the definition of Core-readable Fukaya tower. Item (xi) is Proposition 4.70. Item (xii) is Proposition 4.81. Item (xiii) follows from Definition 4.86. Item (xiv) is Proposition 4.95. Item (xv) is Proposition 4.98. Item (xvi) follows from Definition 4.4 and Proposition 4.5. Item (xvii) follows from Definitions 4.13 and 4.14, together with Proposition 4.15. Item (xviii) follows from Definition 4.22 and Proposition 4.24. Item (xix) follows from Definitions 4.34, 4.35, 4.36, and 4.73, together with Remark 4.39. Item (xx) follows from Propositions 4.45, 4.52, 4.62, and 4.63. Item (xxi) is Proposition 4.78. Item (xxii) follows from Definition 4.89 and Propositions 4.90 and 4.91. $\square$

MS-F.14. Explicit exclusions

The following are not asserted in Appendix MS-F.

- No Fukaya category is constructed.
- No Floer-theoretic transversality theorem is proved.
- No compactness theorem is proved.
- No unobstructedness theorem is proved.
- No A_∞ -category theorem is proved.
- No Fukaya-category equivalence is proved.
- No homological mirror symmetry theorem is proved.
- No microlocal sheaf equivalence theorem is proved.
- No Fukaya/Mirror comparison is assumed to be an equivalence by default.
- No Fukaya-side obstruction is identified with a Core obstruction by default.
- No Floer grading is identified with Core amplitude $[-1, 0]$ by default.
- No Fukaya persistence extraction is assumed without declaration.
- No Fukaya Type IV proxy replaces $(\mu_{\text{Collapse}}, u_{\text{Collapse}})$.
- No Fukaya Type IV proxy replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No Fukaya/Mirror comparison replaces B-Gate⁺.
- No Fukaya/Mirror comparison replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No Fukaya/Mirror comparison replaces eligibility.
- No Fukaya/Mirror comparison replaces the budget condition

$$\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

- No Core pass is interpreted as a Fukaya-side theorem without a separate transport theorem.
- No advisory Fukaya-side artifact supports Core acceptance.
- No MS appendix proves an arithmetic, Iwasawa, Mirror, categorical, Fukaya, or symplectic theorem merely by interface formulation.
- No global exactness of T_τ^{bar} is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift C_τ is assumed.
- No Fukaya-side equivalence, quasi-equivalence, HMS candidate, or microlocal-sheaf equivalence is automatically a Core exact comparison.
- No Fukaya-side controlled comparison is automatically a metric-gap-certified comparison.
- No action, energy, wrapping, or microlocal scale is automatically a constructible one-parameter persistence parameter.
- No continuation map is automatically a Core tower comparison artifact.
- No perturbation, continuation, model-replacement, or tower-artifact transport defect is ignored when consumed by a certificate.
- No sampled or advisory Fukaya stable band is a certified Core stable band.
- No Fukaya-side unobstructedness proves $\text{Eval}_{1,W,\tau}(F) = 0$, Core Ext^1 -vanishing, or Type IV cleanliness by default.
- No homological mirror symmetry, Fukaya equivalence, or microlocal equivalence is promoted from [Spec] by a successful Core pass alone.
- No missing Fukaya transport hypothesis is repaired by metric slack or favorable advisory evidence.

MS-F.15. Provenance and status

The material in this appendix depends on:

- (i) the arithmetic interface meta-theory of Appendix MS-A;
- (ii) the cyclotomic, Iwasawa, congruence, and p -adic profile interface discipline of Appendix MS-B;
- (iii) the Mirror Reflection and categorical comparison interface of Appendix MS-C;
- (iv) the Mirror/Transfer 2-cell ledger of Appendix L;
- (v) the higher-categorical extension discipline of Appendix N;
- (vi) the repaired barcode-level collapse and constructible persistence framework of Appendix A;
- (vii) the admissible representative discipline of Appendix B;
- (viii) the eligible realization regime of Appendix C;
- (ix) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;
- (x) the manifest schema of Appendix G;
- (xi) the ledger catalog of Appendix M;
- (xii) the interface protocol of Chapter 8;
- (xiii) the non-claim discipline of Chapter 7.

Its role is to provide a disciplined interface for Fukaya-categorical and Floer-theoretic structures and to close the MS problem-interface layer without converting it into Core or symplectic theorem content.

Status labels for Appendix MS-F. *Problem-interface definitions:* Definitions [4.7](#), [4.8](#), [4.9](#), [4.10](#), [4.17](#), [4.18](#), [4.19](#), [4.26](#), [4.27](#), [4.28](#), [4.29](#), [4.30](#), [4.40](#), [4.41](#), [4.48](#), [4.54](#), [4.55](#), [4.56](#), [4.65](#), [4.66](#), [4.67](#), [4.68](#), [4.69](#), [4.79](#), [4.80](#), [4.83](#), [4.84](#), [4.85](#), [4.86](#), [4.92](#), [4.93](#), [4.94](#), [4.97](#).

Additional v19.0 hardening definitions: Definitions [4.4](#), [4.13](#), [4.14](#), [4.22](#), [4.23](#), [4.34](#), [4.35](#), [4.36](#), [4.44](#), [4.51](#), [4.60](#), [4.61](#), [4.72](#), [4.73](#), [4.76](#), [4.77](#), [4.89](#), [4.100](#).

Problem-interface propositions/theorems: Propositions [4.20](#), [4.31](#), [4.32](#), [4.42](#), [4.49](#), [4.57](#), [4.58](#), [4.70](#), [4.81](#), [4.87](#), [4.95](#), [4.98](#), Theorem [4.103](#).

Additional v19.0 hardening propositions: Propositions [4.5](#), [4.15](#), [4.24](#), [4.37](#), [4.38](#), [4.45](#), [4.46](#), [4.52](#), [4.53](#), [4.62](#), [4.63](#), [4.74](#), [4.78](#), [4.90](#), [4.91](#), [4.101](#).

Operational cautions: Remarks [4.1](#), [4.2](#), [4.3](#), [4.11](#), [4.12](#), [4.21](#), [4.33](#), [4.43](#), [4.50](#), [4.59](#), [4.71](#), [4.82](#), [4.88](#), [4.96](#), [4.99](#).

Additional v19.0 hardening remarks: Remarks [4.6](#), [4.16](#), [4.25](#), [4.39](#), [4.47](#), [4.64](#), [4.75](#), [4.102](#).

Explicit exclusions: Section MS-F.14.

MS closure: Definition [4.97](#), Proposition [4.98](#), Theorem [4.103](#).

5 Appendix AGI-N: Resolution-Assisted Normalization Interface

AGI-N.1. Scope and interface status

This appendix defines the interface discipline for resolution-assisted normalization. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Resolution-assisted normalization is a pre-gate interface, not a Core proof.

Resolution, normalization, desingularization, smoothing, regularization, scale alignment, chart refinement, model replacement, or defect isolation may prepare an object for Core-readable evaluation. However, none of these operations replaces:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \phi_{i,W,\tau}, \quad \text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

All Core-relevant normalized persistence data are evaluated through the repaired windowed profile:

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Whenever tower diagnostics are invoked, the required comparison is a declared tower comparison artifact:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Remark 5.1 (Interface status). The material in this appendix is interface-level material. Any problem-specific implication from normalization to regularity, smoothness, arithmetic validity, PDE well-posedness, geometric resolution, categorical equivalence, or singularity removal is [Spec] by default unless separately proved under the relevant interface protocol.

Remark 5.2 (Why this appendix precedes Navier–Stokes appendices). Resolution-assisted normalization may be used by PDE, geometric, arithmetic, or categorical interfaces. It is placed before the Navier–Stokes appendices because normalization, scale selection, singularity preprocessing, localized chart refinement, and resolution of candidate defect regions are likely to appear as pre-gate operations in PDE exploration.

Remark 5.3 (No exact-collapse restoration). This appendix does not restore the old exact-collapse framework. In particular, it does not assume a global exact collapse functor, a positive-threshold Serre localization, or a global strict filtered lift. All Core-relevant normalized evaluations must be expressed through

$$\text{Eval}_{i,W,\tau}$$

and barcode-level threshold deletion

$$T_\tau^{\text{bar}}.$$

AGI-N.1bis. Normalization interface sovereignty

Definition 5.4 (Normalization interface sovereignty). *Normalization interface sovereignty* is the rule that resolution, normalization, smoothing, regularization, scale alignment, chart refinement, discretization, model replacement, and defect isolation remain proposer-side or pre-gate operations unless they re-enter the AK Core through declared verifier-side records. The admissible verifier-side re-entry routes are:

- (i) constructible persistence or filtered objects;
- (ii) repaired evaluation records $\text{Eval}_{i,W,\tau}(F)$;

- (iii) admissible representatives $C_{\tau, W}F$;
- (iv) eligibility records for any invoked Ext^1 -clause;
- (v) declared tower comparison artifacts $\phi_{i, W, \tau}$;
- (vi) monitored Type IV diagnostic records;
- (vii) scalarized metric-ledger entries;
- (viii) manifest-readable certificate records.

Definition 5.5 (Normalization verifier-side record). A *normalization verifier-side record* is a record produced after normalization that has one of the Core-readable types listed in Definition 5.4 and is sufficiently documented for independent manifest reconstruction.

Proposition 5.6 (No normalization sovereignty override). *No normalization, resolution, smoothing, regularization, numerical preprocessing, or model replacement overrides the verifier-side Core requirements.*

Proof. The Core verdict is determined by typed verifier-side data: repaired evaluations, admissible representatives, eligibility records, tower artifacts, diagnostics, ledgers, gates, and manifest entries. Normalization operations may prepare or transform data, but they do not themselves have those Core types unless the required records are supplied. $\square$

Remark 5.7 (Normalization evidence is proposer-side by default). Resolution charts, smoothing scripts, scale choices, numerical regularization, and local model replacements may guide construction. They are not verifier-side evidence until converted into declared Core-readable records or ledgered defects.

AGI-N.2. Normalization source data

Definition 5.8 (Normalization source datum). A *normalization source datum* is an object or dataset requiring preprocessing before Core-readable evaluation. Examples include:

- (i) a singular geometric object;
- (ii) a PDE field or weak solution candidate;
- (iii) a discretized numerical field;
- (iv) a multi-scale filtered object;
- (v) a noisy persistence dataset;
- (vi) an arithmetic family with incompatible local scales;
- (vii) a categorical comparison object with mismatched grading or filtration;
- (viii) a sheaf, derived, or higher-categorical object requiring extraction to persistence data;
- (ix) a tower whose finite-stage and apex behavior are not yet aligned;
- (x) a candidate singular region requiring local chart replacement or blow-up analysis.

Definition 5.9 (Resolution-assisted normalization). A *resolution-assisted normalization* is a declared operation

$$\mathfrak{N} : X \dashrightarrow X^{\text{nor}}$$

that transforms a source datum X into a normalized datum X^{nor} through one or more of:

- (i) resolution of singular features;
- (ii) scale normalization;
- (iii) coordinate normalization;
- (iv) blow-up or blow-down analysis;
- (v) regularization;
- (vi) smoothing;
- (vii) renormalization;
- (viii) local chart replacement;
- (ix) filtration realignment;
- (x) defect isolation;
- (xi) model refinement;
- (xii) finite-layer truncation;
- (xiii) apex approximation;
- (xiv) persistence extraction preparation.

Definition 5.10 (Normalization auxiliary data). *Normalization auxiliary data* are the choices required to make a normalization procedure audit-readable, including:

- (i) local charts or coordinate systems;
- (ii) scale parameters;
- (iii) smoothing or regularization parameters;
- (iv) truncation levels;
- (v) resolution centers;
- (vi) blow-up or blow-down maps;
- (vii) discretization scales;
- (viii) model-replacement policies;
- (ix) persistence extraction conventions;
- (x) artifact references supporting all choices.

Definition 5.11 (Normalization status). A normalization operation has one of the following statuses:

- (i) *declared*: the operation is specified but not yet verified;
- (ii) *defined*: the operation is defined on the source datum;
- (iii) *controlled*: the operation has declared defects and ledger treatment;
- (iv) *Core-readable*: the normalized output has a Core-readable realization;
- (v) *advisory*: the operation is used only for exploration;
- (vi) *failed*: the operation is undefined, uncontrolled, nonconstructible, or over budget.

Remark 5.12 (Dashed arrow). The notation

-->

indicates that normalization may be partial, conditional, approximate, algorithmic, noninvertible, or dependent on auxiliary choices. It is not assumed to be a strict globally defined functor.

Remark 5.13 (Auxiliary choices are part of the interface). Normalization choices are not informal implementation details. If normalization affects Core-readable data, the auxiliary choices must be declared in the manifest and their defects must be ledged when nonzero.

AGI-N.2bis. Normalization audit package and status transitions

Definition 5.14 (Normalization audit package). A *normalization audit package* consists of:

- (i) the source datum;
- (ii) the normalization operation;
- (iii) all auxiliary choices;
- (iv) the intended Core-readable target;
- (v) the status of the operation;
- (vi) the declared defect namespace;
- (vii) artifact references sufficient for reconstruction.

Definition 5.15 (Normalization status-transition certificate). A *normalization status-transition certificate* is a record justifying a transition between normalization statuses, for example from *declared* to *defined*, from *defined* to *controlled*, or from *controlled* to *Core-readable*.

Proposition 5.16 (Normalization status transitions are not automatic). *No normalization status transition is automatic.*

Proof. Each status asserts a different typed obligation. A declared operation need not be defined; a defined operation need not have controlled defect; a controlled operation need not produce constructible persistence; and a Core-readable output need not satisfy gates, diagnostics, or budget. Therefore each transition requires its own certificate. □

Proposition 5.17 (Defined normalization is not controlled normalization). *A normalization operation being defined on a source datum does not imply that its Core-relevant defect is zero, bounded, scalarized, or ledger-admissible.*

Proof. Definedness is a domain-of-operation condition. Controlledness is a quantitative or typed audit condition about how the operation changes Core-relevant data. The former does not imply the latter. $\square$

Remark 5.18 (Auxiliary-choice drift). Changing charts, scales, smoothing parameters, truncation levels, or extraction conventions changes the audit package. If the change affects consumed verifier-side data, it must be recorded as a new or updated normalization route rather than silently inherited from the previous route.

AGI-N.3. Normalization as pre-gate

Definition 5.19 (Normalization pre-gate). A *normalization pre-gate* is a declared condition determining whether a source datum X is suitable for conversion into a Core-readable normalized object. It may check:

- (i) whether the normalization procedure is defined;
- (ii) whether auxiliary choices are declared;
- (iii) whether normalization defects are bounded;
- (iv) whether the normalized object is constructible;
- (v) whether the normalized object admits a filtered or persistence realization;
- (vi) whether an admissible representative can be supplied when required;
- (vii) whether an eligible realized object can be supplied when Ext^1 is invoked;
- (viii) whether a tower comparison artifact can be supplied when Type IV diagnostics are invoked;
- (ix) whether the total normalization error can be ledgered below the declared gap.

Definition 5.20 (Normalization pre-gate record). A *normalization pre-gate record* records:

- (i) the source datum X ;
- (ii) the proposed normalization operation $\mathfrak{N}$;
- (iii) the pre-gate conditions checked;
- (iv) pass/fail/not-applicable status;
- (v) the reason for failure, if any;
- (vi) artifact references supporting the status.

Proposition 5.21 (Normalization pre-gate does not replace Core gate). A *normalization pre-gate* may determine whether a normalized object is submitted to Core evaluation, but it does not replace $B\text{-Gate}^+$, monitored Type IV diagnostics, required tower comparison artifacts, or the budget condition.

Proof. A pre-gate is an interface-level preparation condition. Core verdicts are determined by repaired evaluation records, Core gate clauses, monitored diagnostics, required artifacts, and budget comparison. Therefore normalization pre-gates cannot replace Core gates. $\square$

Remark 5.22 (Pre-gate pass is not Core pass). Passing a normalization pre-gate means only that the normalized datum is eligible for the next interface step. It does not imply that the normalized object passes the AK Core.

AGI-N.4. Normalization realization targets

Definition 5.23 (Normalized realization target). A normalization interface must declare the realization target of X^{nor} as one of:

- (i) a filtered object in $\text{FiltCh}((k))$;
- (ii) a constructible persistence object in $\text{Pers}_k^{\text{cons}}$;
- (iii) a repaired windowed collapsed profile

$$\text{Eval}_{i,W,\tau}(F^{\text{nor}});$$
- (iv) an admissible representative

$$C_{\tau,W}F^{\text{nor}};$$
- (v) a realized eligible object in $D^{[-1,0]}(k\text{-mod})$;
- (vi) a declared tower comparison artifact

$$\phi_{i,W,\tau}^{\text{nor}};$$
- (vii) a discretized or measurement-level object with declared error;
- (viii) a certificate-record component;
- (ix) an advisory artifact.

Definition 5.24 (Core-readable normalized object). A normalized object X^{nor} is *Core-readable* if it yields a filtered or persistence object

$$F^{\text{nor}} \in \text{FiltCh}((k)) \quad \text{or} \quad P^{\text{nor}} \in \text{Pers}_k^{\text{cons}}$$

such that the repaired windowed evaluation

$$\text{Eval}_{i,W,\tau}(F^{\text{nor}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F^{\text{nor}}))$$

or the direct persistence expression

$$T_{\tau}^{\text{bar}}(W_W P^{\text{nor}})$$

is defined in the constructible monitored regime.

Definition 5.25 (Normalized extraction route). A *normalized extraction route* is a declared process taking

$$X^{\text{nor}}$$

to one of:

$$\text{FiltCh}((k)), \quad \text{Pers}_k^{\text{cons}}, \quad \text{Eval}_{i,W,\tau}(F^{\text{nor}}), \quad C_{\tau,W}F^{\text{nor}}, \quad \phi_{i,W,\tau}^{\text{nor}},$$

or a ledger component.

Proposition 5.26 (No Core use without Core-readable normalized object). *A normalized object cannot support Core certification unless it is Core-readable or supplies verifier-side certificate data.*

Proof. The Core verifier operates on repaired evaluation records, admissible representatives, eligible realized objects, diagnostics, ledgers, tower artifacts, and certificate records. A normalized object without Core-readable realization provides none of these. Therefore it cannot support Core certification. $\square$

Remark 5.27 (Normalization target is not automatic). Normalization may produce a smoother or better-behaved object, but this does not automatically make it constructible persistence data. The extraction route must be declared.

AGI-N.4bis. One-parameter extraction and constructibility discipline

Definition 5.28 (Normalization one-parameter extraction). A *normalization one-parameter extraction* is a declared rule that selects a single ordered persistence parameter from normalized data that may naturally involve several scales, charts, variables, coordinates, discretization levels, or physical units. It must declare:

- (i) the candidate source parameters;
- (ii) the selected persistence parameter;
- (iii) the order convention;
- (iv) normalization and unit conventions;
- (v) tie handling and incomparable cases;
- (vi) intentionally discarded information;
- (vii) the extraction defect introduced by the selection.

Definition 5.29 (Normalization extraction-faithfulness status). A normalization extraction-faithfulness claim has exactly one of the following statuses:

proved, conditional, operational, [Spec], failed, undefined.

Only proved or fully discharged conditional extraction-faithfulness may support a normalization transport theorem.

Proposition 5.30 (Scale or coordinate choice is not a Core parameter by name). *A scale, coordinate, chart index, discretization level, or physical variable is not a Core persistence parameter merely by being chosen during normalization.*

Proof. The Core requires constructible one-parameter persistence or filtered data with a declared order and repaired evaluation. A named scale or coordinate supplies no such object until a one-parameter extraction and constructibility record are provided. □

Proposition 5.31 (Smoothing or resolution does not imply constructibility). *Smoothing, regularization, desingularization, or chart refinement does not by itself imply that the extracted persistence object is constructible on the declared windows.*

Proof. Constructibility is a property of the extracted persistence object in the Core regime. Smoother or more regular source-side data may still yield nonconstructible, infinite-rank, uncontrolled, or improperly parameterized persistence data. Constructibility must be verified after extraction. □

Remark 5.32 (Multi-scale normalization risk). Common risks include collapsing space, time, scale, amplitude, and discretization depth into one parameter, using chart-dependent orderings, or treating a locally convenient scale as globally canonical. These choices are not neutral unless proved neutral for the declared purpose.

AGI-N.5. Normalization defects

Definition 5.33 (Normalization defect). A *normalization defect* is a declared discrepancy

$$\delta^{\text{nor}}$$

between the source datum X , the normalized datum X^{nor} , and the Core-relevant data extracted from them.

Definition 5.34 (Resolution defect). A *resolution defect* is a normalization defect arising specifically from a resolution, desingularization, blow-up, blow-down, or local chart replacement procedure. It is denoted

$$\delta^{\text{res}}.$$

Definition 5.35 (Scale-normalization defect). A *scale-normalization defect* is a discrepancy arising from rescaling, renormalization, unit conversion, or normalization of local magnitudes. It is denoted

$$\delta^{\text{scale}}.$$

Definition 5.36 (Filtration-alignment defect). A *filtration-alignment defect* is a discrepancy arising when a normalized object's filtration is aligned with the Core persistence parameter. It is denoted

$$\delta^{\text{filt-align}}.$$

Definition 5.37 (Normalization extraction defect). A *normalization extraction defect* is a discrepancy arising when extracting Core-readable filtered or persistence data from a normalized object. It is denoted

$$\delta^{\text{nor-ext}}.$$

Definition 5.38 (Reverse-transport defect). A *reverse-transport defect* is a discrepancy arising when attempting to transport a Core-visible statement about X^{nor} back to the original source datum X . It is denoted

$$\delta^{\text{rev}}.$$

Definition 5.39 (Chart-normalization defect). A *chart-normalization defect* is a discrepancy arising from chart selection, chart refinement, coordinate replacement, transition-map mismatch, or local-to-global chart assembly in a normalization route. It is denoted

$$\delta^{\text{chart}}.$$

Definition 5.40 (Smoothing or regularization defect). A *smoothing or regularization defect* is a discrepancy introduced by smoothing, mollification, regularization, denoising, or any analogous operation used before Core-readable extraction. It is denoted

$$\delta^{\text{smooth}}.$$

Definition 5.41 (Normalization truncation defect). A *normalization truncation defect* is a discrepancy introduced by finite-scale, finite-layer, finite-window, finite-rank, or finite-precision truncation inside a normalization route. It is denoted

$$\delta^{\text{nor-trunc}}.$$

Definition 5.42 (Normalized tower-artifact transport defect). A *normalized tower-artifact transport defect* is a discrepancy introduced when a tower comparison artifact, apex policy, terminal mode, or tower-side profile is transported through normalization or reverse-normalization. It is denoted

$$\Delta_{\phi, \text{nor}}.$$

Definition 5.43 (Normalization ledger entry). A *normalization ledger entry* records:

- (i) the source datum X ;
- (ii) the normalized datum X^{nor} ;
- (iii) the normalization operation $\mathfrak{N}$;
- (iv) auxiliary choices;
- (v) the realization target;
- (vi) the extraction route;
- (vii) the defect type;
- (viii) the defect value or bound;
- (ix) the aggregation rule;
- (x) how the defect enters $\Sigma\delta$;
- (xi) artifact references.

Proposition 5.44 (Normalization defects affect Core only through ledger). *Normalization defects affect Core audit only if they are included in the declared δ -ledger.*

Proof. Normalization defects are not Core gate verdicts and not monitored Type IV diagnostics. They affect Core audit only by contributing to the total budget

$$\Sigma\delta.$$

□

Remark 5.45 (Normalization is not budget-free). A normalization procedure may make the object easier to analyze. However, if it changes the Core-relevant data, the change must be measured or bounded and ledgered.

AGI-N.5bis. Defect namespace and double-counting discipline

Definition 5.46 (Primitive normalization discrepancy). A *primitive normalization discrepancy* is an atomic source of mismatch introduced or exposed by normalization before it is aggregated into a ledger entry.

Definition 5.47 (Normalization discrepancy relation). Whenever an AGI-N defect is used together with defects from Appendix M, Appendices H–J, or any problem interface appendix, the relation between the symbols must be declared as exactly one of:

alias, refinement, aggregate, disjoint.

Proposition 5.48 (No double-counting of normalization defects). *The same primitive normalization discrepancy may not be counted twice in the declared ledger.*

Proof. The budget comparison is meaningful only when the ledger aggregation reflects the declared primitive discrepancies. Counting the same discrepancy through two labels without an alias, refinement, or aggregate relation misstates the total defect. Therefore double-counting is disallowed. □

Proposition 5.49 (Undeclared defect relation blocks consumption). *If two defect symbols may refer to overlapping primitive discrepancies and their relation is undeclared, then a certificate may not consume both as independent budget components.*

Proof. Independence, refinement, and aliasing are different ledger semantics. Without declaring which relation holds, the verifier cannot reconstruct the intended aggregation. The affected budget component is therefore not audit-readable. $\square$

Remark 5.50 (Ledger-catalog registration). The symbols

$$\delta^{\text{nor}}, \delta^{\text{res}}, \delta^{\text{scale}}, \delta^{\text{filt-align}}, \delta^{\text{nor-ext}}, \delta^{\text{rev}}, \delta^{\text{chart}}, \delta^{\text{smooth}}, \delta^{\text{nor-trunc}}, \Delta_{\phi, \text{nor}}, \delta^{\text{PDE-scale}}$$

are interface-layer defect namespaces. Before verifier-side consumption, they should be registered in the Appendix M ledger catalog or explicitly mapped to existing catalog entries as aliases, refinements, aggregates, or disjoint components.

AGI-N.6. Normalized repaired evaluation

Definition 5.51 (Normalized repaired evaluation). Let X^{nor} be a Core-readable normalized object with realization F^{nor} . The *normalized repaired evaluation* on (i, W, τ) is

$$\text{Eval}_{i, W, \tau}(F^{\text{nor}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F^{\text{nor}})).$$

Definition 5.52 (Normalization-adjusted budget). If a certificate record uses normalized data, the adjusted budget is

$$\Sigma \delta_{\text{nor}} := \Sigma \delta_{\text{base}} \oplus \delta^{\text{nor}} \oplus \delta^{\text{res}} \oplus \delta^{\text{scale}} \oplus \delta^{\text{filt-align}} \oplus \delta^{\text{nor-ext}} \oplus \delta^{\text{rev}} \oplus \delta^{\text{chart}} \oplus \delta^{\text{smooth}} \oplus \delta^{\text{nor-trunc}} \oplus \Delta_{\phi, \text{nor}},$$

with omitted components set to the identity element of the declared ledger.

Proposition 5.53 (Normalized evidence must pass adjusted budget). *A certificate record relying on normalized data may pass Audit Mode only if*

$$\text{val}_B(\Sigma \delta_{\text{nor}}) < \text{gap}_{\tau}(\mathfrak{B}_{\text{nor}}).$$

Proof. By Chapter 6, all declared deviations used by a certificate must be included in the total ledger. Normalization defects are such deviations. Therefore the normalization-adjusted budget must satisfy the strict pass condition. $\square$

Remark 5.54 (Normalized pass is not source pass). A Core pass for a normalized object does not automatically imply a Core pass for the original source object. A transport theorem or controlled reverse-normalization statement is required.

Remark 5.55 (Evaluation record is still required). A normalization ledger does not replace the repaired evaluation record

$$\text{Eval}_{i, W, \tau}(F^{\text{nor}}).$$

If normalized data are used in a certificate, the repaired evaluation record or its reconstructible artifact path must be declared.

AGI-N.6bis. Normalization comparison modes and gap safety

Definition 5.56 (Normalization comparison). A *normalization comparison* is a comparison between a source datum, a normalized datum, two normalization routes, two extracted profiles, or a normalized tower and its apex profile.

Definition 5.57 (Normalization comparison mode). Every normalization comparison consumed by a certificate must declare exactly one local mode:

source_exact, normalized_exact, normalized_metric, controlled_ledger, advisory_only, unavailable, failed.

Its projection to Core comparison modes is governed as follows:

- (i) normalized_exact projects to Core mode exact only when an actual equality or profile isomorphism of the declared Core-readable normalized profiles is supplied;
- (ii) normalized_metric projects to metric_gap_certified only after a bottleneck bound, scalarization route, and two-sided threshold-gap record are supplied for the windowed pre-threshold profiles;
- (iii) source_exact remains external unless transported to a normalized Core-readable exact comparison;
- (iv) controlled_ledger contributes only through the declared ledger unless it also supplies the required exact or metric record;
- (v) advisory_only and unavailable project to not_compared;
- (vi) failed blocks the invoked comparison route.

Proposition 5.58 (Source exactness is not normalized Core exactness). *An equality, isomorphism, or exact relation in the source domain is not a Core exact comparison of normalized profiles unless the declared normalization and extraction route transports it to an actual profile equality or isomorphism.*

Proof. Source exactness and normalized Core exactness live in different typed regimes. Normalization may change coordinates, scales, charts, filtrations, or extracted persistence. Hence a transport or realization record is required. $\square$

Proposition 5.59 (Normalization closeness is not metric-gap certification). *A small smoothing parameter, local chart error, scale error, discretization error, or numerical residual is not a metric-gap-certified comparison unless it supplies the required bottleneck and threshold-clearance records for the actual windowed pre-threshold profiles.*

Proof. Metric-gap certification is a statement about persistence-profile discrepancy and threshold clearance. A source-side or implementation-side error estimate has the wrong type unless converted into that record. $\square$

Remark 5.60 (Gap is measured after realization and windowing). The protective gap used for threshold safety is the gap of the realized, windowed, pre-threshold profile. It is not a generic source-domain scale margin and is not inherited merely from a normalization parameter.

AGI-N.7. Source-to-normalized transport

Definition 5.61 (Source-to-normalized transport statement). A *source-to-normalized transport statement* is a statement of the form:

$$\text{property of } X \implies \text{property of } X^{\text{nor}},$$

or

$$\text{Core-visible behavior of } X \implies \text{Core-visible behavior of } X^{\text{nor}}.$$

Definition 5.62 (Normalized-to-source transport statement). A *normalized-to-source transport statement* is a statement of the form:

$$\text{property of } X^{\text{nor}} \implies \text{property of } X,$$

or

$$\text{Core-visible behavior of } X^{\text{nor}} \implies \text{Core-visible behavior of } X.$$

Definition 5.63 (Normalization transport theorem). A *normalization transport theorem* is a source-to-normalized or normalized-to-source transport statement supplied with:

- (i) the source datum X ;
- (ii) the normalization operation $\mathfrak{N}$;
- (iii) the normalized datum X^{nor} ;
- (iv) auxiliary choices;
- (v) realization and extraction routes;
- (vi) repaired evaluation records;
- (vii) admissible representative data, if invoked;
- (viii) eligibility data, if Ext^1 is invoked;
- (ix) tower comparison artifacts, if Type IV diagnostics are invoked;
- (x) all normalization, extraction, and reverse-transport defects;
- (xi) ledger entries and artifact references;
- (xii) a proof of the stated transport direction.

Definition 5.64 (Default status of normalization transport). Every source-to-normalized or normalized-to-source transport statement is [Spec] by default unless separately proved under the relevant interface protocol.

Proposition 5.65 (Normalized Core pass does not imply source conclusion). A *Core pass for X^{nor}* does not imply a conclusion about X unless a normalized-to-source transport theorem proves that implication.

Proof. Core pass for X^{nor} certifies Core-visible behavior of the normalized object. A conclusion about X concerns the original source datum. The normalization may discard, alter, approximate, or reparameterize information. Therefore a transport theorem is required. $\square$

Remark 5.66 (Two transport directions). The directions

$$X \Rightarrow X^{\text{nor}}$$

and

$$X^{\text{nor}} \Rightarrow X$$

are distinct. Normalization often makes the forward direction easier than the reverse direction. The reverse direction is usually the one needed for problem-specific conclusions.

AGI-N.7bis. Losslessness and theorem-promotion discipline

Definition 5.67 (Lossless normalization statement). A *lossless normalization statement* asserts that the normalization and extraction route preserves the features required for a declared source-side or Core-side conclusion.

Definition 5.68 (Normalization theorem promotion). *Normalization theorem promotion* is the operation of changing a normalization-assisted implication from [Spec] or candidate status to theorem status. It requires a proved normalization transport theorem and updated claim-register status.

Proposition 5.69 (Normalization is not lossless by declaration). *Declaring a normalization useful, canonical, standard, resolution-like, or regularizing does not establish that it is lossless for the declared purpose.*

Proof. Losslessness is a preservation theorem about the features consumed by the interface. Usefulness or canonical form may motivate the construction, but it does not prove preservation of source conclusions, Core profiles, eligibility data, or tower artifacts. $\square$

Proposition 5.70 (Reverse-transport hypotheses are non-scalar). *Missing hypotheses needed to transport a normalized Core pass back to the source datum cannot be repaired by metric slack, advisory normalization evidence, or a successful normalization pre-gate.*

Proof. Reverse-transport hypotheses determine whether the normalized statement has the intended source-side meaning. They are semantic and logical obligations, not scalar perturbation components. Thus ledger slack cannot discharge them. $\square$

Remark 5.71 (The reverse direction is the dangerous direction). Most problem-specific conclusions require returning from the normalized object to the original source datum. This reverse direction is precisely where information loss, chart dependence, truncation, and model replacement can invalidate an otherwise clean normalized Core pass.

AGI-N.8. Resolution and singularity claims

Definition 5.72 (Singularity proxy). A *singularity proxy* is an interface-level signal suggesting the presence of a singular, unstable, or unresolved region. Examples include:

- (i) blow-up of a numerical or analytic observable;
- (ii) concentration of persistence mass;
- (iii) nonzero residual repaired degree-1 profile

$$\text{Eval}_{1,W,\tau}(F) \neq 0;$$

- (iv) unstable Betti integral behavior;
- (v) anomalous tower defect;
- (vi) failure of normalization pre-gate;
- (vii) local scale collapse;
- (viii) chart incompatibility;
- (ix) failure of finite-layer-to-apex comparison.

Definition 5.73 (Resolution success claim). A *resolution success claim* is a statement asserting that a resolution-assisted normalization has removed, controlled, isolated, or localized a singularity or defect.

Proposition 5.74 (Resolution success is not Core success). A *resolution success claim* does not imply Core pass unless the resulting normalized object satisfies Core gate, diagnostic, artifact, and budget conditions.

Proof. Resolution success is an interface-level or domain-level statement. Core pass requires B-Gate⁺, monitored Type IV cleanliness, required artifacts, and budget admissibility. The former does not imply the latter without further verification. $\square$

Proposition 5.75 (Resolution success is not problem-specific regularity). A *resolution success claim* does not imply problem-specific regularity, smoothness, or non-blow-up unless a separate transport theorem proves that implication.

Proof. Resolution-assisted normalization may alter the object, isolate a defect, or replace local data. A problem-specific regularity statement concerns the original source problem. A transport theorem is required to connect the normalized behavior back to the original problem. $\square$

Remark 5.76 (Singularity proxy is not singularity theorem). A singularity proxy may indicate where to inspect. It is not itself a theorem asserting singularity formation or singularity absence.

AGI-N.9. Normalization and monitored Type IV

Definition 5.77 (Normalized tower). A *normalized tower* is a directed system

$$F_0^{\text{nor}} \longrightarrow F_1^{\text{nor}} \longrightarrow F_2^{\text{nor}} \longrightarrow \cdots$$

obtained from normalized source data and equipped with a declared apex object

$$F_\infty^{\text{nor}}.$$

Definition 5.78 (Normalized tower comparison artifact). A *normalized tower comparison artifact* is a declared comparison

$$\phi_{i,W,\tau}^{\text{nor}} : \text{ColimProfile}_{i,W,\tau}(F_\bullet^{\text{nor}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty^{\text{nor}}).$$

Definition 5.79 (Normalization Type IV proxy). A *Normalization Type IV proxy* is a normalization-side signal suggesting possible monitored Type IV obstruction. Examples include:

- (i) normalization failure in a directed tower;
- (ii) mismatch between finite-stage and limiting normalized objects;

- (iii) persistent defect after resolution;
- (iv) loss of information under normalization;
- (v) emergence of a limiting feature absent at finite normalized stages;
- (vi) failure of normalized apex reconstruction;
- (vii) nonzero normalized tower comparison residue.

Definition 5.80 (Actual normalized Type IV diagnostic). Let

$$\mathcal{M}_{\text{nor}} \subset \{(i, W, \tau) : i \text{ monitored, } W \text{ declared, } \tau \text{ declared}\}$$

be the declared finite monitored scope for normalized tower data. The actual monitored Type IV diagnostic is defined only when the normalized tower comparison artifacts are declared and terminal-tame on every member of $\mathcal{M}_{\text{nor}}$. In that case it is

$$\begin{aligned} \mu_{\text{Collapse}}^{\text{nor}} &:= \sum_{(i, W, \tau) \in \mathcal{M}_{\text{nor}}} \text{tgdim}_W \ker(\phi_{i, W, \tau}^{\text{nor}}), \\ u_{\text{Collapse}}^{\text{nor}} &:= \sum_{(i, W, \tau) \in \mathcal{M}_{\text{nor}}} \text{tgdim}_W \text{coker}(\phi_{i, W, \tau}^{\text{nor}}). \end{aligned}$$

If the artifacts are not invoked, not declared, or not terminal-tame, the diagnostic is undefined/not_invoked; it is not recorded as $(0, 0)$.

Remark 5.81 (Decorated normalized diagnostics). The pair

$$(\mu_{\text{Collapse}}^{\text{nor}}, u_{\text{Collapse}}^{\text{nor}})$$

is the diagnostic pair for the declared normalized tower and its declared artifact. It is not the undecorated diagnostic pair of the original source datum. Passing from the decorated normalized pair to an original-source diagnostic or conclusion is a separate claim requiring artifact coherence and a normalized-to-source transport theorem.

Proposition 5.82 (Normalization Type IV proxy is not Type IV diagnosis). *A Normalization Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0).$$

Proof. Monitored Type IV diagnostics are computed from window-relative terminal-generic dimensions of kernel and cokernel defect profiles of declared tower comparison artifacts. A normalization proxy may suggest related behavior, but it does not compute those defect profiles unless a Core-readable normalized tower and its artifact

$$\phi_{i, W, \tau}^{\text{nor}}$$

are explicitly constructed. □

Remark 5.83 (Correct use of normalization proxies). Normalization proxies may guide where tower diagnostics should be computed. They may not replace

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

AGI-N.9bis. Type IV artifact coherence for normalized towers

Definition 5.84 (Artifact-coherent normalized Type IV record). An *artifact-coherent normalized Type IV record* is a Type IV record in which the normalization route, extraction route, tower objects, apex object, terminal mode, monitored scope, and tower comparison artifacts remain fixed throughout the diagnostic claim.

Proposition 5.85 (Changing normalized artifacts creates a new claim). *Changing the normalization route, extraction route, apex policy, terminal mode, monitored scope, or tower comparison artifact creates a new Type IV claim rather than a continuation of the previous one.*

Proof. The Type IV diagnostic is computed from the declared artifacts and monitored scope. Changing these data changes the object being diagnosed. Therefore the resulting assertion is a new claim requiring its own manifest and ledger records. $\square$

Proposition 5.86 (Undefined normalized Type IV is non-verdict). *If the normalized Type IV diagnostic is not invoked, not declared, or not terminal-tame, it supplies no zero diagnostic and no nonzero diagnostic.*

Proof. The diagnostic is a typed quantity defined only under the declared artifact and terminal-tameness conditions. When those conditions fail or are absent, the diagnostic is undefined. Undefined data cannot be converted into either $(0, 0)$ or $eq(0, 0)$. $\square$

Remark 5.87 (Terminal-tameness is required). A normalized tower may look stable in finite windows without supporting a terminal-generic dimension. The terminal-tameness hypothesis is part of the Type IV diagnostic, not an optional convenience.

AGI-N.10. Normalization certificate records

Definition 5.88 (Normalization certificate record). A *normalization certificate record* is a certificate record whose source data have been processed by a declared resolution-assisted normalization and whose Core-readable data are extracted from the normalized object.

Definition 5.89 (AGI-N manifest entry). An *AGI-N manifest entry* records:

- (i) the source datum X ;
- (ii) the normalization operation $\mathfrak{N}$;
- (iii) the normalization status;
- (iv) the normalized datum X^{nor} ;
- (v) auxiliary normalization choices;
- (vi) the realization target;
- (vii) the Core-readable extraction route;
- (viii) declared windows and thresholds;
- (ix) collapse policy;
- (x) monitored degrees;

- (xi) repaired evaluation records

$$\text{Eval}_{i,w,\tau}(F^{\text{nor}});$$
- (xii) admissible representative data, if invoked;
- (xiii) eligibility data, if Ext^1 is invoked;
- (xiv) normalized tower data, if used;
- (xv) tower comparison artifacts, if Type IV diagnostics are invoked;
- (xvi) normalization defects and ledger entries;
- (xvii) monitored Type IV diagnostics;
- (xviii) Core gate verdicts;
- (xix) artifact references;
- (xx) status of all normalization transport statements.

Proposition 5.90 (Normalization certificate records must satisfy Minimal Manifest). *Every normalization certificate record must satisfy the Minimal Manifest Axiom of Appendix G.*

Proof. Normalization does not exempt a certificate from Core auditability. The Minimal Manifest requires declared objects, windows, thresholds, collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledger policy, diagnostics, tower artifacts when invoked, gate verdicts, artifact references, and final status. Therefore AGI-N records must satisfy those requirements. $\square$

Remark 5.91 (Normalization artifacts). Normalization artifacts may include resolution charts, blow-up data, scale maps, smoothing parameters, local model replacements, filtered objects, persistence extraction scripts, repaired evaluation records, tower comparison artifacts, error estimates, diagnostic tables, and ledger records. They support reconstruction but do not themselves determine Core pass.

AGI-N.11. Normalization pass, Core pass, and problem conclusion

Definition 5.92 (Normalization interface pass). *A normalization interface pass means that the source datum has been successfully normalized into Core-readable or ledger-compatible form under the declared normalization protocol.*

Definition 5.93 (Core pass for normalized data). *A Core pass for normalized data means that the corresponding certificate record passes B-Gate⁺, monitored Type IV diagnostics, required artifact checks, and the budget condition.*

Definition 5.94 (Problem-specific conclusion). *A problem-specific conclusion is a statement about the original source problem, such as regularity, smoothness, non-blow-up, desingularization, equivalence, finiteness, or well-posedness.*

Proposition 5.95 (Core pass for normalized data is not problem conclusion). *A Core pass for normalized data does not imply a problem-specific conclusion unless a separate normalized-to-source transport theorem proves that implication.*

Proof. The Core pass concerns the normalized Core-readable object. A problem-specific conclusion concerns the original problem. The normalization step may be non-invertible, approximate, lossy, or dependent on auxiliary choices. Thus a transport theorem is required. $\square$

Remark 5.96 (Three-layer distinction). The following must remain distinct:

normalization interface pass, Core pass, problem-specific theorem.

Confusing these layers would violate the non-claim discipline of Chapter 7.

AGI-N.12. Relation to Navier–Stokes interface

Remark 5.97 (Use in Navier–Stokes appendices). In the Navier–Stokes appendices, resolution-assisted normalization may be used to prepare velocity, vorticity, energy, pressure, enstrophy, or localized scale profiles for Core-readable persistence extraction. However, it remains a pre-gate interface. It does not prove Navier–Stokes regularity, non-blow-up, uniqueness, or smoothness.

Definition 5.98 (PDE normalization pre-gate). A *PDE normalization pre-gate* is a normalization pre-gate applied to PDE-derived data, such as:

- (i) local energy profiles;
- (ii) vorticity concentration profiles;
- (iii) pressure fields;
- (iv) scale-invariant quantities;
- (v) weak-solution approximants;
- (vi) discretized flow fields;
- (vii) candidate singular neighborhoods;
- (viii) blow-up rescaling windows;
- (ix) local regularity charts.

Definition 5.99 (PDE normalized profile). A *PDE normalized profile* is a normalized PDE-derived datum realized as a filtered or persistence object and evaluated through

$$\text{Eval}_{i,W,\tau}(F_{\text{PDE}}^{\text{nor}}).$$

Proposition 5.100 (PDE normalization pre-gate does not imply PDE regularity). *Passing a PDE normalization pre-gate does not imply PDE regularity.*

Proof. A PDE normalization pre-gate only prepares PDE-derived data for further interface processing or Core-readable evaluation. PDE regularity is a problem-specific conclusion about solutions of a PDE. The former does not imply the latter without a transport theorem. $\square$

Remark 5.101 (PDE profile is not PDE theorem). A repaired evaluation record derived from a PDE normalized profile may be useful evidence inside the AK interface. It is not, by itself, a theorem about Navier–Stokes or any other PDE.

AGI-N.12bis. PDE, discretization, and rescaling hardening

Definition 5.102 (PDE normalization evidence status). A PDE normalization artifact has exactly one of the following statuses:

sample, calibration, bounded_approximation, certified_Core_record, transport_theorem, failed, undefined.

Only certified_Core_record or transport_theorem status may support verifier-side consumption.

Definition 5.103 (PDE rescaling defect). A *PDE rescaling defect* is a discrepancy introduced by blow-up rescaling, parabolic scaling, localization, renormalization, or chart-dependent scale selection in PDE-derived data. It is denoted

$$\delta^{\text{PDE-scale}}.$$

Proposition 5.104 (Numerical normalized profile is not regularity theorem). *A numerical or sampled normalized PDE profile is not a regularity, smoothness, uniqueness, or non-blow-up theorem.*

Proof. A numerical profile has finite computational and modeling scope. PDE regularity and non-blow-up statements are problem-specific analytic conclusions about the source equation. A transport theorem or certified reduction is required. $\square$

Proposition 5.105 (Blow-up rescaling profile is not singularity exclusion). *A normalized profile obtained by blow-up rescaling does not exclude singularity formation in the original source problem unless a normalized-to-source theorem proves that implication.*

Proof. Blow-up rescaling changes scale, coordinates, and often limiting objects. The absence of a detected defect in the rescaled profile need not imply absence of a singularity in the original problem without a reverse transport theorem. $\square$

Proposition 5.106 (Sampled stabilization is not certified regularity). *Observed stabilization across sampled normalized windows is not a certified regularity or non-blow-up claim unless it is upgraded to an artifact-coherent Core stable-band record and transported back to the source problem.*

Proof. Sampled stabilization is finite advisory evidence. A certified Core stable band requires artifact coherence, threshold-gap safety, and declared diagnostics across the band. A source regularity statement additionally requires normalized-to-source transport. $\square$

Remark 5.107 (Discretization and measurement remain external until ledgered). Mesh size, timestep, sampling density, numerical precision, sensor error, or solver residuals are not Core metrics by name. They must be converted into declared profile discrepancies or ledgered defects before being consumed by the verifier.

AGI-N.13. Summary of resolution-assisted normalization interface

Theorem 5.108 (Appendix AGI-N interface package). *Within the Problem Interface layer, the following package is available.*

- (i) *Resolution-assisted normalization is a pre-gate interface, not a Core theorem.*
- (ii) *Normalization source data require declared normalization operations and auxiliary choices.*
- (iii) *A normalized object becomes Core-readable only after a declared filtered or persistence realization.*

(iv) *All Core-relevant normalized profiles are evaluated through*

$$\text{Eval}_{i,W,\tau}(F^{\text{nor}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F^{\text{nor}})).$$

(v) *Normalization pre-gates do not replace Core gates.*

(vi) *Normalization defects affect Core audit only through the declared δ -ledger.*

(vii) *Normalized evidence must pass the normalization-adjusted budget condition*

$$\text{val}_B(\Sigma \delta_{\text{nor}}) < \text{gap}_{\tau}(\mathfrak{B}_{\text{nor}}).$$

(viii) *A Core pass for a normalized object does not imply a conclusion about the original source object without a normalized-to-source transport theorem.*

(ix) *Resolution success does not imply Core success.*

(x) *Resolution success does not imply problem-specific regularity, smoothness, or non-blow-up.*

(xi) *Normalization Type IV proxies do not replace monitored Type IV diagnostics.*

(xii) *Normalized Type IV diagnostics require a declared tower comparison artifact*

$$\phi_{i,W,\tau}^{\text{nor}}.$$

(xiii) *Normalization certificate records must satisfy the Minimal Manifest Axiom.*

(xiv) *PDE normalization pre-gates do not imply PDE regularity.*

(xv) *PDE normalized profiles do not imply PDE theorems without separate transport.*

(xvi) *Normalization sovereignty forbids resolution, smoothing, regularization, and model replacement from overriding verifier-side Core requirements.*

(xvii) *Normalization status transitions require explicit certificates and are not automatic.*

(xviii) *Multi-scale, chart-dependent, or PDE-derived normalized data require declared one-parameter extraction before entering the constructible persistence regime.*

(xix) *Source exactness, normalization closeness, and implementation-level error are not Core exactness or metric-gap certification without declared Core-readable comparison records.*

(xx) *Normalization defects must obey namespace registration and double-counting discipline.*

(xxi) *Normalization is not lossless by declaration, and reverse-transport hypotheses are non-scalar.*

(xxii) *Normalized Type IV diagnostics require artifact coherence, terminal-tameness, and declared finite monitored scope.*

(xxiii) *Numerical PDE profiles, blow-up rescaling profiles, and sampled stabilization do not imply PDE regularity without certified Core records and normalized-to-source transport.*

Proof. Items (i)–(v) follow from the definitions of normalization, pre-gates, Core-readable normalized objects, and repaired evaluation. Item (vi) is Proposition 5.44. Item (vii) is Proposition 5.53. Item (viii) is Proposition 5.65. Items (ix)–(x) are Propositions 5.74 and 5.75. Item (xi) is Proposition 5.82. Item (xii) follows from Definition 5.78. Item (xiii) is Proposition 5.90. Item (xiv) is Proposition 5.100. Item (xv) follows from Remark 5.101. The additional v19.0 hardening clauses follow as follows. Item (xvi) follows from Definition 5.4 and Proposition 5.6. Item (xvii) follows from Definitions 5.14 and 5.15, together with Propositions 5.16 and 5.17. Item (xviii) follows from Definition 5.28 and Propositions 5.30 and 5.31. Item (xix) follows from Definitions 5.56 and 5.57, together with Propositions 5.58 and 5.59. Item (xx) follows from Definitions 5.46 and 5.47, together with Propositions 5.48 and 5.49. Item (xxi) follows from Definition 5.67, Definition 5.68, Proposition 5.69, and Proposition 5.70. Item (xxii) follows from Definition 5.84, Proposition 5.85, and Proposition 5.86. Item (xxiii) follows from Definitions 5.102 and 5.103, together with Propositions 5.104, 5.105, and 5.106. $\square$

AGI-N.14. Explicit exclusions

The following are not asserted in Appendix AGI-N.

- No resolution theorem is proved.
- No desingularization theorem is proved.
- No smoothing theorem is proved.
- No regularization theorem is proved.
- No normalization operation is assumed to be invertible.
- No normalization operation is assumed to be lossless.
- No normalization pre-gate replaces B-Gate⁺.
- No normalization pre-gate replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No normalization pre-gate replaces eligibility.
- No normalization pre-gate replaces monitored Type IV diagnostics.
- No normalization pre-gate replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No normalization pre-gate replaces the budget condition

$$\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

- No normalization Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No resolution success claim is treated as Core pass.

- No Core pass for normalized data is treated as a theorem about the original source datum.
- No PDE normalization pre-gate is treated as Navier–Stokes regularity.
- No PDE normalized profile is treated as Navier–Stokes regularity.
- No singularity proxy is treated as a singularity theorem.
- No advisory normalization artifact supports Core acceptance.
- No global exactness of T_{τ}^{bar} is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift C_{τ} is assumed.

AGI-N.15. Provenance and status

The material in this appendix depends on:

- (i) the interface protocol of Chapter 8;
- (ii) the non-claim discipline of Chapter 7;
- (iii) the repaired barcode-level collapse and constructible persistence framework of Appendix A;
- (iv) the admissible representative discipline of Appendix B;
- (v) the eligible realization regime of Appendix C;
- (vi) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;
- (vii) the manifest schema of Appendix G;
- (viii) the advisory and discretization disciplines of Appendices H and I;
- (ix) the measurement discipline of Appendix J;
- (x) the ledger catalog of Appendix M;
- (xi) the arithmetic interface sovereignty and transport-separation pattern of Appendix MS-A;
- (xii) the Mirror/Transfer and higher-categorical namespace disciplines of Appendices L and N.

Its role is to provide a disciplined pre-gate normalization interface for problem-specific data, especially before PDE-oriented appendices, without allowing normalization to become a hidden proof mechanism.

Status labels for Appendix AGI-N. *Problem-interface definitions:* Definitions 5.8, 5.9, 5.10, 5.11, 5.19, 5.20, 5.23, 5.24, 5.25, 5.33, 5.34, 5.35, 5.36, 5.37, 5.38, 5.43, 5.51, 5.52, 5.61, 5.62, 5.63, 5.64, 5.72, 5.73, 5.77, 5.78, 5.79, 5.80, 5.88, 5.89, 5.92, 5.93, 5.94, 5.98, 5.99.

Problem-interface propositions/theorems: Propositions 5.21, 5.26, 5.44, 5.53, 5.65, 5.74, 5.75, 5.82, 5.90, 5.95, 5.100, Theorem 5.108.

Operational cautions: Remarks 5.1, 5.2, 5.3, 5.12, 5.13, 5.22, 5.27, 5.45, 5.54, 5.55, 5.66, 5.76, 5.83, 5.91, 5.96, 5.97, 5.101.

Additional v19.0 hardening definitions: Definitions 5.4, 5.5, 5.14, 5.15, 5.28, 5.29, 5.39, 5.40, 5.41, 5.42, 5.46, 5.47, 5.56, 5.57, 5.67, 5.68, 5.84, 5.102, 5.103.

Additional v19.0 hardening propositions: Propositions 5.6, 5.16, 5.17, 5.30, 5.31, 5.48, 5.49, 5.58, 5.59, 5.69, 5.70, 5.85, 5.86, 5.104, 5.105, 5.106.

Additional v19.0 hardening remarks: Remarks 5.7, 5.18, 5.32, 5.50, 5.60, 5.71, 5.81, 5.87, 5.107.

Explicit exclusions: Section AGI-N.14.

6 Appendix NS-A: Navier–Stokes Exploration Protocol

NS-A.1. Scope and exploration status

This appendix defines an exploration protocol for Navier–Stokes-related data in the AK framework. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Navier–Stokes exploration is not Navier–Stokes proof.

The protocol may generate candidate windows, candidate filtrations, advisory indicators, persistence profiles, normalized objects, or tower candidates for further Core evaluation. However, none of these exploratory outputs replaces:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \phi_{i,W,\tau}, \quad \text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

All Core-relevant persistence data produced from Navier–Stokes source data must be evaluated through the repaired windowed profile:

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Whenever tower diagnostics are invoked, the required comparison is a declared tower comparison artifact:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Remark 6.1 (Interface status). The material in this appendix is interface-level and exploration-level material. Any implication from observed Navier–Stokes behavior to regularity, smoothness, uniqueness, continuation, non-blow-up, blow-up, or singularity exclusion is [Spec] by default unless separately proved under the theoretical soundness layer and proof-first program.

Remark 6.2 (Relation to Appendix AGI-N). Appendix AGI-N supplies the resolution-assisted normalization interface. The present appendix may use such normalization as a pre-gate for Navier–Stokes-derived data, but normalization remains separate from proof. A normalized PDE profile is still not a PDE theorem.

Remark 6.3 (No exact-collapse restoration). This appendix does not restore the old exact-collapse framework. In particular, it does not assume a global exact collapse functor, a positive-threshold Serre localization, or a global strict filtered lift. All Core-relevant NS-derived evaluations must be expressed through

$$\text{Eval}_{i,W,\tau}$$

and barcode-level threshold deletion

$$T_{\tau}^{\text{bar}}.$$

NS-A.2. Navier–Stokes source data

Definition 6.4 (Navier–Stokes source datum). A *Navier–Stokes source datum* is a mathematical, numerical, or observational object associated with a Navier–Stokes problem. Examples include:

(i) a velocity field $u(x, t)$;

(ii) a pressure field $p(x, t)$;

(iii) vorticity

$$\omega = \nabla \times u;$$

(iv) local energy density;

(v) enstrophy density;

(vi) strain tensor or deformation tensor data;

(vii) weak-solution approximants;

(viii) Leray–Hopf type data;

(ix) numerical simulation snapshots;

(x) discretized flow fields;

(xi) scale-localized flow profiles;

(xii) candidate singular neighborhoods;

(xiii) blow-up rescaling profiles;

(xiv) localized parabolic cylinder data.

Definition 6.5 (Exploration input). An *exploration input* is a Navier–Stokes source datum together with:

(i) a declared spatial domain;

(ii) a declared time interval;

(iii) boundary or decay conditions, if used;

(iv) viscosity parameter;

- (v) sampling or discretization scale, if applicable;
- (vi) normalization policy, if applicable;
- (vii) observable family;
- (viii) window, scale, and threshold policies;
- (ix) artifact references.

Definition 6.6 (NS source status). A Navier–Stokes source datum has one of the following statuses:

- (i) *raw*: source data are declared but not yet processed;
- (ii) *observable-extracted*: PDE observables have been extracted;
- (iii) *normalized*: resolution-assisted normalization has been applied;
- (iv) *Core-readable*: filtered or persistence realization is declared;
- (v) *tower-readable*: a Core-readable tower and tower comparison artifact are declared;
- (vi) *advisory*: used only for exploration;
- (vii) *failed*: required preprocessing, realization, constructibility, artifact, or budget conditions fail.

Remark 6.7 (Source data are not Core data). A velocity field, vorticity field, pressure field, simulation output, or weak-solution approximant is not automatically a Core object. It becomes Core-readable only after a declared filtered or persistence realization route is provided.

Remark 6.8 (Source status is not proof status). The status of source data records where they sit in the exploration pipeline. It does not assert a Navier–Stokes theorem, nor does it determine Core pass.

NS-A.3. PDE observables

Definition 6.9 (PDE observable). A *PDE observable* is a scalar, vector, tensor, or derived quantity extracted from Navier–Stokes source data for exploration, normalization, realization, or advisory analysis. Typical examples include:

- (i) $|u|$, the velocity magnitude;
- (ii) $|\omega|$, the vorticity magnitude;
- (iii) $|\nabla u|$, the strain or gradient magnitude;
- (iv) local kinetic energy density;
- (v) local enstrophy density;
- (vi) pressure magnitude or pressure oscillation;
- (vii) scale-invariant local energy quantities;
- (viii) local intermittency indicators;
- (ix) parabolic cylinder averages;

- (x) blow-up candidate scores;
- (xi) localized dissipation proxies;
- (xii) spatial or temporal concentration profiles.

Definition 6.10 (Observable family). An *observable family* is a declared collection

$$\mathcal{O}_{\text{NS}} = \{O_\lambda\}_{\lambda \in \Lambda}$$

of PDE observables used for exploration, filtration construction, normalization, advisory analysis, or Core-readable realization.

Definition 6.11 (Observable status). An observable has one of the following statuses:

- (i) *advisory*: used only for exploration or prioritization;
- (ii) *pre-gate*: used to select or normalize data before Core evaluation;
- (iii) *realization*: used to construct filtered or persistence objects;
- (iv) *ledgered*: used with declared error contribution;
- (v) *verifier-side*: used directly in a Core-readable certificate record.

Definition 6.12 (Observable extraction defect). An *observable extraction defect* is a declared discrepancy

$$\delta^{\text{obs}}$$

arising from computing, approximating, interpolating, sampling, smoothing, or otherwise extracting an observable from Navier–Stokes source data.

Remark 6.13 (Observable status must be declared). The same observable may be advisory in one run and realization-relevant in another. Its status must be declared in the certificate or exploration record.

Remark 6.14 (Observable is not theorem). An observable may correlate with regularity, concentration, or instability. Such correlation is not a proof of a Navier–Stokes conclusion unless a separate theoretical transport theorem is supplied.

NS-A.4. Filtered realization of Navier–Stokes data

Definition 6.15 (Navier–Stokes filtered realization hypothesis). A *Navier–Stokes filtered realization hypothesis* is a declared map or procedure

$$\mathfrak{R}_{\text{NS}} : \mathcal{D}_{\text{NS}} \longrightarrow \text{FiltCh}((\cdot)k) \quad \text{or} \quad \mathfrak{R}_{\text{NS}} : \mathcal{D}_{\text{NS}} \longrightarrow \text{Pers}_k^{\text{cons}},$$

where $\mathcal{D}_{\text{NS}}$ is a declared class of Navier–Stokes source data, normalized data, or observables.

Definition 6.16 (PDE persistence extraction). A *PDE persistence extraction* is a procedure that maps PDE observables to constructible persistence data. Examples include:

- (i) sublevel-set persistence of an observable;
- (ii) superlevel-set persistence of a concentration field;

- (iii) cubical persistence from discretized flow data;
- (iv) time-windowed persistence profiles;
- (v) scale-parameter persistence from localized quantities;
- (vi) persistence of thresholded vorticity or enstrophy regions;
- (vii) persistence of normalized blow-up rescaling profiles;
- (viii) persistence of parabolic cylinder aggregate fields.

Definition 6.17 (Core-readable Navier–Stokes object). A Navier–Stokes source datum is *Core-readable* if the realization produces:

$$F_{\text{NS}} \in \text{FiltCh}(()k) \quad \text{or} \quad P_{\text{NS}} \in \text{Pers}_k^{\text{cons}}$$

such that, for declared i, W, τ , the repaired evaluation

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F_{\text{NS}}))$$

or the direct persistence expression

$$T_{\tau}^{\text{bar}}(W_W P_{\text{NS}})$$

is defined in the constructible monitored regime.

Definition 6.18 (NS realization defect). An *NS realization defect* is a declared discrepancy

$$\delta^{\text{NS-real}}$$

arising from mapping PDE source data or observables into filtered or persistence data.

Proposition 6.19 (No Core use without PDE realization). *Navier–Stokes source data cannot support Core certification unless they are realized as Core-readable filtered or persistence data, or as verifier-side certificate data.*

Proof. The Core verifier evaluates repaired persistence profiles, admissible representatives, eligible realized objects, diagnostics, ledgers, tower artifacts, and gate verdicts. Raw PDE fields, numerical observables, or simulation outputs are not automatically such objects. Therefore a declared realization is required. $\square$

Remark 6.20 (Purpose-faithfulness). A PDE realization must be purpose-faithful. If the exploration concerns vorticity concentration, then the realization must preserve the relevant concentration structure. If it concerns energy localization, then the realization must preserve the relevant energy-localization structure. Purpose-faithfulness does not mean that every PDE property is preserved.

NS-A.5. Window, scale, and threshold policies

Definition 6.21 (Navier–Stokes window). A *Navier–Stokes window* is a declared right-open interval

$$W = [a, a')$$

in a chosen exploration parameter, such as:

- (i) physical time;
- (ii) spatial scale;

- (iii) parabolic scale;
- (iv) observable magnitude;
- (v) filtration parameter;
- (vi) simulation step;
- (vii) normalized concentration level;
- (viii) blow-up rescaling depth.

Definition 6.22 (Scale policy). A *scale policy* specifies how spatial, temporal, and parabolic scales are selected and related. It may include:

$$r, \quad r^2, \quad t_0 - r^2 < t < t_0, \quad B_r(x_0),$$

or other declared local-scale structures.

Definition 6.23 (Threshold policy for NS exploration). A *threshold policy for NS exploration* specifies the collapse threshold τ , threshold sweep, stable band candidate, or threshold-selection rule used for repaired persistence evaluation.

Definition 6.24 (NS evaluation policy). An *NS evaluation policy* declares:

- (i) monitored degrees i ;
- (ii) windows W ;
- (iii) thresholds τ ;
- (iv) observable family;
- (v) realization route;
- (vi) whether the resulting profile is advisory, pre-gate, or verifier-side;
- (vii) ledger components associated with the evaluation.

Remark 6.25 (Scale is not proof). Scale selection may focus attention on candidate singular or regular regions. It does not prove regularity or blow-up. Scale policies are exploration and realization policies, not PDE theorems.

Remark 6.26 (Threshold stability is not PDE regularity). A stable band in repaired persistence data may be useful for Core analysis. It is not, by itself, Navier–Stokes regularity. A transport theorem is required to convert Core-visible stability into a PDE conclusion.

NS-A.6. Advisory indicators

Definition 6.27 (Navier–Stokes advisory indicator). A *Navier–Stokes advisory indicator* is an exploratory quantity used to rank, flag, or prioritize regions for further Core evaluation. Examples include:

- (i) vorticity concentration score;
- (ii) enstrophy growth score;

- (iii) local energy concentration score;
- (iv) scale-invariant warning score;
- (v) spectral instability indicator;
- (vi) Betti integral of a thresholded observable;
- (vii) persistence mass concentration;
- (viii) normalization Type IV proxy;
- (ix) candidate blow-up score Φ_{NS} ;
- (x) finite-resolution instability score;
- (xi) parabolic cylinder anomaly score.

Definition 6.28 (Navier–Stokes potential). A *Navier–Stokes potential* is an advisory navigation quantity

$$\Phi_{\text{NS}} : \mathcal{X}_{\text{NS}} \longrightarrow \mathbb{R} \cup \{\infty\}$$

on a declared exploration space $\mathcal{X}_{\text{NS}}$, used to prioritize candidate regions, scales, windows, or normalization targets.

Definition 6.29 (NS advisory-to-budget conversion). An *NS advisory-to-budget conversion* is a declared rule assigning a ledger component

$$\delta^{\text{NS-adv}}$$

to an advisory NS indicator when that indicator is used in a certificate-relevant way.

Proposition 6.30 (NS advisory indicators do not determine Core verdicts). *No Navier–Stokes advisory indicator determines Core pass or Core reject.*

Proof. By Appendix H and Chapter 6, advisory quantities may guide inspection but do not determine Core verdicts. Core verdicts are determined by repaired evaluation records, gate verdicts, monitored diagnostics, required artifacts, and budget comparison. $\square$

Remark 6.31 (Correct use of Φ_{NS}). The potential

$$\Phi_{\text{NS}}$$

may identify promising or dangerous regions. It is not a proof of regularity, blow-up, or Core admissibility.

NS-A.7. Exploration pre-gates

Definition 6.32 (Navier–Stokes exploration pre-gate). A *Navier–Stokes exploration pre-gate* is a condition used to decide whether a candidate region or dataset should be submitted to Core-readable realization. Examples include:

- (i) sufficient data resolution;
- (ii) bounded measurement noise;
- (iii) successful normalization;

- (iv) admissible scale policy;
- (v) constructibility of extracted persistence;
- (vi) acceptable discretization error;
- (vii) nonpathological sampling pattern;
- (viii) declared boundary condition compatibility;
- (ix) availability of repaired evaluation records;
- (x) availability of tower comparison artifacts when tower diagnostics are invoked.

Definition 6.33 (NS pre-gate record). *An NS pre-gate record records:*

- (i) the candidate region, scale, or dataset;
- (ii) the pre-gate condition;
- (iii) pass/fail/not-applicable status;
- (iv) the reason for failure, if any;
- (v) artifact references supporting the decision.

Proposition 6.34 (NS pre-gates do not replace Core gates). *A Navier–Stokes exploration pre-gate may select candidates for Core evaluation, but it does not replace B-Gate⁺, monitored Type IV diagnostics, required tower comparison artifacts, or the budget condition.*

Proof. Pre-gates are interface-level filters. Core gates are verifier-side conditions. Passing a pre-gate does not imply passing the Core. $\square$

Remark 6.35 (Pre-gate failure). Failure of an exploration pre-gate means that the candidate is not suitable for the declared exploration route. It is not a proof of singularity, non-regularity, or Core rejection.

NS-A.8. Discretization and measurement discipline

Definition 6.36 (NS discretization error). *An NS discretization error is a declared error bound*

$$\delta^{\text{NS-disc}}$$

arising from grid resolution, time stepping, mesh approximation, numerical truncation, or discrete persistence computation.

Definition 6.37 (NS measurement error). *An NS measurement error is a declared error bound*

$$\delta^{\text{NS-meas}}$$

arising from observing, sampling, interpolating, or estimating Navier–Stokes source data or observables.

Definition 6.38 (NS algorithmic error). *An NS algorithmic error is a declared error bound*

$$\delta^{\text{NS-alg}}$$

arising from numerical solvers, persistence algorithms, integration schemes, interpolation routines, or implementation choices.

Definition 6.39 (NS exploration ledger). The *NS exploration ledger* aggregates exploration-level errors:

$$\Sigma\delta_{\text{NS}} := \delta^{\text{NS-disc}} \oplus \delta^{\text{NS-meas}} \oplus \delta^{\text{NS-alg}} \oplus \delta^{\text{NS-real}} \oplus \delta^{\text{NS-adv}} \oplus \delta^{\text{NS-param}} \oplus \delta^{\text{obs}} \oplus \delta^{\text{nor}} \oplus \delta^{\text{filt-align}},$$

with omitted components set to the identity element of the declared ledger. If an exploration record introduces additional registered NS-interface defects, the aggregation must be extended by declaration rather than consumed silently.

Proposition 6.40 (Exploratory evidence must be ledgered before Core use). *If numerical or measured Navier–Stokes evidence is used in a Core certificate record, its discretization, measurement, normalization, realization, observable-extraction, and algorithmic errors must be included in the declared δ -ledger.*

Proof. By Appendices I, J, AGI-N, and M, all approximation, measurement, normalization, realization, extraction, and implementation discrepancies used by a certificate must enter the budget ledger. Therefore exploratory PDE evidence must be ledgered before Core use. $\square$

Remark 6.41 (Simulation is not proof). A numerical simulation may generate valuable exploratory evidence. It is not a proof of Navier–Stokes regularity or non-regularity. If simulation-derived data are used in a certificate, their errors must be declared and ledgered.

NS-A.9. Exploration-to-Core re-entry

Definition 6.42 (Exploration-to-Core re-entry). *Exploration-to-Core re-entry* is the process by which exploratory Navier–Stokes data are converted into a Core certificate record. It requires:

- (i) a Core-readable realization;
- (ii) declared windows;
- (iii) threshold policy;
- (iv) monitored degrees;
- (v) repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}});$$

- (vi) admissible representative data

$$C_{\tau,W}F_{\text{NS}}$$

if representatives are invoked;

- (vii) eligibility data, if Ext^1 is invoked;
- (viii) tower comparison artifacts, if Type IV diagnostics are invoked;
- (ix) monitored Type IV diagnostics;
- (x) ledgered errors;
- (xi) gate verdicts;
- (xii) artifact references.

Definition 6.43 (NS Core candidate). An *NS Core candidate* is a realized Navier–Stokes-derived object prepared for Core evaluation but not yet certified.

Definition 6.44 (NS Core certificate record). An *NS Core certificate record* is a certificate record obtained from an NS Core candidate after all required repaired evaluation records, admissible representative records when invoked, eligibility records when invoked, tower artifacts when invoked, Core gates, diagnostics, ledgers, and artifact references have been supplied.

Proposition 6.45 (Exploration output is not certificate). *An exploration output is not a Core certificate unless it satisfies the Minimal Manifest Axiom and all Core verifier conditions.*

Proof. Exploration output may contain candidate windows, indicators, plots, persistence summaries, normalized objects, or candidate towers. A Core certificate additionally requires declared objects, windows, threshold and collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledger policy, diagnostics, tower artifacts when invoked, gate verdicts, artifact references, and final status. Thus exploration output is not a certificate by itself. $\square$

Remark 6.46 (Re-entry is the critical boundary). The boundary between exploration and proof is crossed only at Core re-entry. Before re-entry, the objects are advisory or candidate data. After re-entry, they are evaluated by the ordinary Core verifier. Even after Core pass, a PDE conclusion still requires a separate theoretical transport theorem.

NS-A.10. NS Type IV proxies

Definition 6.47 (NS-derived tower). An *NS-derived tower* is a directed system

$$F_0^{\text{NS}} \longrightarrow F_1^{\text{NS}} \longrightarrow F_2^{\text{NS}} \longrightarrow \dots$$

obtained from Navier–Stokes-derived data, such as grid refinements, scale refinements, time-window refinements, normalized blow-up profiles, or finite-resolution approximants.

Definition 6.48 (NS tower comparison artifact). An *NS tower comparison artifact* is a declared comparison

$$\phi_{i,W,\tau}^{\text{NS}} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}^{\text{NS}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}^{\text{NS}}).$$

Definition 6.49 (NS Type IV proxy). An *NS Type IV proxy* is an exploratory signal suggesting possible monitored generic Type IV obstruction in a realized Navier–Stokes-derived tower. Examples include:

- (i) finite-scale collapse followed by limiting reappearance;
- (ii) discrepancy between finite-resolution and continuum-limit persistence;
- (iii) anomalous tower behavior under grid refinement;
- (iv) persistence features appearing only at a limiting scale;
- (v) normalization failure in a limiting regime;
- (vi) mismatch between local finite windows and global limiting behavior;
- (vii) candidate defect visible only after parabolic rescaling.

Definition 6.50 (Actual NS Type IV diagnostic). Let

$$\mathcal{M}_{\text{NS}} \subset \{(i, W, \tau)\}$$

be the declared finite monitored scope for an NS-derived tower. The *actual NS Type IV diagnostic* is defined only when, for every $(i, W, \tau) \in \mathcal{M}_{\text{NS}}$, a declared tower comparison artifact

$$\phi_{i,W,\tau}^{\text{NS}} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}^{\text{NS}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}^{\text{NS}})$$

is supplied and the associated kernel and cokernel defect profiles are terminal-tame on the declared window. In that case,

$$\mu_{\text{Collapse}}^{\text{NS}} := \sum_{(i,W,\tau) \in \mathcal{M}_{\text{NS}}} \text{tgdim}_W \ker(\phi_{i,W,\tau}^{\text{NS}}),$$

and

$$u_{\text{Collapse}}^{\text{NS}} := \sum_{(i,W,\tau) \in \mathcal{M}_{\text{NS}}} \text{tgdim}_W \text{coker}(\phi_{i,W,\tau}^{\text{NS}}).$$

If the monitored scope, tower artifact, apex object, terminal mode, or terminal-tameness proof is missing, the diagnostic is *undefined*; it is not recorded as $(0, 0)$.

Proposition 6.51 (NS Type IV proxy is not Type IV diagnosis). *An NS Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0).$$

Proof. Monitored Type IV diagnostics are computed from window-relative terminal-generic dimensions tgdim_W of kernel and cokernel defect profiles of declared tower comparison artifacts. An NS proxy may suggest where such diagnostics should be computed, but it is not the diagnostic itself. $\square$

Remark 6.52 (Correct use of NS Type IV proxies). NS Type IV proxies may guide tower construction, refinement, or rechecking. They may not replace the computation of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

NS-A.11. Exploration artifacts

Definition 6.53 (NS exploration artifact). *An NS exploration artifact* is any artifact supporting an exploration record or Core re-entry record. Examples include:

- (i) PDE source data;
- (ii) velocity or vorticity fields;
- (iii) simulation files;
- (iv) grid or mesh specifications;
- (v) time-step records;
- (vi) observable definitions;
- (vii) normalization artifacts;
- (viii) persistence extraction scripts;
- (ix) repaired evaluation records;

- (x) barcode files;
- (xi) Betti integral tables;
- (xii) advisory indicator logs;
- (xiii) discretization and measurement error estimates;
- (xiv) NS tower comparison artifacts;
- (xv) diagnostic tables;
- (xvi) certificate manifests.

Definition 6.54 (NS exploration manifest entry). *An NS exploration manifest entry records:*

- (i) the Navier–Stokes source datum;
- (ii) the source status;
- (iii) the observable family;
- (iv) the realization hypothesis;
- (v) window, scale, and threshold policies;
- (vi) advisory indicators;
- (vii) pre-gate results;
- (viii) normalization policy, if used;
- (ix) discretization, measurement, realization, observable-extraction, and algorithmic errors;
- (x) repaired evaluation records, if available;
- (xi) tower comparison artifacts, if available;
- (xii) artifact references;
- (xiii) re-entry status.

Proposition 6.55 (Artifacts are required for re-entry audit). *An NS exploration output cannot be used for Core re-entry unless the required artifact references are supplied.*

Proof. Core re-entry requires audit-readable reconstruction. Without artifacts, an auditor cannot reconstruct the source data, observables, realization route, repaired evaluation records, error estimates, diagnostics, tower artifacts, or ledger entries. Thus artifact references are required. $\square$

NS-A.12. Exploration pass, Core pass, and PDE conclusion

Definition 6.56 (NS exploration pass). An *NS exploration pass* means that a candidate datum has passed the declared exploration pre-gates and is suitable for Core re-entry.

Definition 6.57 (Core pass for NS-derived data). A *Core pass for NS-derived data* means that the corresponding NS Core certificate record passes B-Gate⁺, monitored Type IV diagnostics, required artifact checks, and the budget condition.

Definition 6.58 (Navier–Stokes conclusion). A *Navier–Stokes conclusion* is a problem-specific statement about existence, smoothness, regularity, uniqueness, continuation, blow-up, non-blow-up, or singularity exclusion of Navier–Stokes solutions.

Definition 6.59 (NS theoretical transport theorem). An *NS theoretical transport theorem* is a theorem proving that a declared Core-visible condition or Core certificate behavior implies a declared Navier–Stokes conclusion. It must specify:

- (i) the PDE class;
- (ii) the domain and boundary conditions;
- (iii) the solution class;
- (iv) the realization route;
- (v) the Core-visible condition used;
- (vi) all approximation, normalization, and extraction assumptions;
- (vii) the exact Navier–Stokes conclusion;
- (viii) proof references or formal proof artifacts.

Proposition 6.60 (Core pass is not NS conclusion). A *Core pass for NS-derived data does not imply a Navier–Stokes conclusion unless a separate theoretical transport theorem proves that implication.*

Proof. A Core pass certifies Core-visible persistence, diagnostic, artifact, and budget conditions. A Navier–Stokes conclusion concerns PDE solutions. A theoretical transport theorem is required to connect them. $\square$

Remark 6.61 (Three-layer distinction). The following must remain distinct:

NS exploration pass, Core pass, Navier–Stokes theorem.

Confusing these layers would violate the non-claim discipline of Chapter 7.

NS-A.12bis. v19 hardening: exploration sovereignty, re-entry modes, and diagnostic coherence

Definition 6.62 (NS exploration sovereignty). *NS exploration sovereignty* is the rule that exploratory PDE data, advisory indicators, numerical simulations, normalized profiles, finite-resolution observations, or potential values cannot override Core verifier requirements. Every theorem-relevant use must re-enter through a Core-readable realization, repaired evaluation records, declared artifacts, a ledger, and the Minimal Manifest Axiom.

Proposition 6.63 (No exploration sovereignty override). *No NS exploration output can function as verifier-side evidence unless it has been converted into a declared Core-readable object, a verifier-side certificate record, or a ledgered component.*

Proof. Exploration produces candidate data and priorities. Core verification uses typed records: repaired evaluations, representatives, eligibility records, tower artifacts, diagnostics, ledgers, and manifests. Since these are different layers, no exploration output overrides the Core layer. $\square$

Definition 6.64 (NS re-entry mode). An NS exploration artifact has one of the following re-entry modes:

- (i) *advisory-only*;
- (ii) *pre-gate*;
- (iii) *realization-candidate*;
- (iv) *ledgered-support*;
- (v) *verifier-side-record*;
- (vi) *failed*;
- (vii) *undefined*.

Proposition 6.65 (Re-entry mode is not automatic). *An advisory or pre-gate NS artifact does not become a verifier-side record merely because it is numerically stable, visually convincing, or repeatedly observed.*

Proof. Verifier-side status requires declared realization, artifacts, and ledger semantics. Repeated observation may support exploration, but it does not supply those records. $\square$

Definition 6.66 (NS one-parameter extraction). An *NS one-parameter extraction* is a declared procedure selecting one Core persistence parameter from PDE variables such as physical time, parabolic scale, spatial scale, observable magnitude, rescaling depth, grid resolution, or normalized concentration level.

Definition 6.67 (NS parameter-extraction defect). An *NS parameter-extraction defect* is a declared discrepancy

$$\delta^{\text{NS-param}}$$

arising from selecting, collapsing, reparameterizing, discretizing, or comparing PDE-side scales as a one-parameter persistence variable.

Proposition 6.68 (PDE scale is not automatically a Core parameter). *Physical time, parabolic scale, spatial radius, grid resolution, and observable magnitude are not automatically the Core persistence parameter.*

Proof. A Core persistence parameter must be selected by a declared one-parameter extraction and must yield constructible persistence data on the declared windows. PDE variables may be candidates for this role, but the identification is not automatic. $\square$

Definition 6.69 (NS diagnostic coherence record). An *NS diagnostic coherence record* records the monitored scope, tower artifact, apex object, terminal mode, terminal-tameness proof, and provenance of each Type IV diagnostic used for NS-derived data.

Proposition 6.70 (Changing diagnostic artifacts creates a new NS claim). *Changing the monitored scope, apex policy, terminal mode, tower comparison artifact, realization route, or extraction route of an NS diagnostic creates a new diagnostic claim.*

Proof. The diagnostic is defined from a specific artifact family and terminal-generic profile data. Changing those data changes the mathematical object being diagnosed. $\square$

Definition 6.71 (NS sampled stability). *NS sampled stability* means stability observed over a finite sample of grids, scales, windows, simulations, or thresholds.

Definition 6.72 (NS certified Core stable band). An *NS certified Core stable band* is an artifact-coherent stable band, in the sense of Appendix D, for which the relevant NS-derived repaired diagnostics are certified across the declared band.

Proposition 6.73 (Sampled NS stability is not certified band stability). *NS sampled stability does not imply an NS certified Core stable band without a band-level artifact-coherent proof and the required two-sided threshold-gap records.*

Proof. A finite sample may miss threshold-adjacent or limiting changes. Certified band stability is a uniform artifact-coherent statement, not a sampled observation. $\square$

Remark 6.74 (Exploration remains proposer-side until re-entry). NS exploration, including simulation, visualization, advisory scoring, and candidate-window discovery, remains proposer-side until the data are promoted into an audit-readable Core record.

Remark 6.75 (NS ledger namespace registration). The symbols

$$\delta^{\text{NS-disc}}, \quad \delta^{\text{NS-meas}}, \quad \delta^{\text{NS-alg}}, \quad \delta^{\text{NS-real}}, \quad \delta^{\text{NS-adv}}, \quad \delta^{\text{obs}}, \quad \delta^{\text{NS-param}}$$

must be registered in the ledger catalog before being consumed in a certificate. If any symbol aliases, refines, aggregates, or is disjoint from a normalization defect of Appendix AGI-N, that relation must be declared to prevent double counting.

NS-A.13. Summary of Navier–Stokes exploration protocol

Theorem 6.76 (Appendix NS-A exploration package). *Within the Problem Interface layer, the following package is available.*

- (i) *Navier–Stokes source data are not Core objects by default.*
- (ii) *PDE observables must have declared status: advisory, pre-gate, realization, ledgered, or verifier-side.*
- (iii) *Core use requires a declared filtered or persistence realization.*
- (iv) *Core-readable NS-derived objects are evaluated through*

$$\text{Eval}_{i,w,\tau}(F_{\text{NS}}) = T_{\tau}^{\text{bar}}(\mathbf{W}_w \mathbf{P}_i(F_{\text{NS}})).$$
- (v) *Window, scale, threshold, and evaluation policies must be declared.*
- (vi) *NS advisory indicators do not determine Core verdicts.*
- (vii) *NS exploration pre-gates do not replace Core gates.*
- (viii) *Numerical, measured, normalized, realized, observable-extracted, advisory-conversion, parameter-extraction, or algorithmic errors must be ledgered before Core use.*
- (ix) *Exploration-to-Core re-entry requires the Minimal Manifest data and ordinary Core verifier conditions.*

(x) NS Type IV proxies do not replace monitored Type IV diagnostics.

(xi) NS tower diagnostics require declared tower comparison artifacts

$$\phi_{i,W,\tau}^{\text{NS}}.$$

(xii) Exploration artifacts are required for audit-readable Core re-entry.

(xiii) A Core pass for NS-derived data does not imply a Navier–Stokes conclusion without a separate theoretical transport theorem.

(xiv) NS exploration sovereignty prevents advisory, numerical, or pre-gate artifacts from overriding Core verifier requirements.

(xv) NS re-entry modes distinguish advisory-only material from verifier-side records.

(xvi) PDE-side scale variables require a declared one-parameter extraction before they become Core persistence parameters.

(xvii) NS Type IV diagnostics require artifact-coherent diagnostic records; changing the artifact data creates a new claim.

(xviii) Sampled NS stability is not certified Core stable-band evidence without Appendix D band-level coherence.

Proof. Items (i)–(iii) follow from the definitions of NS source data, observables, and filtered realization. Item (iv) follows from the definition of Core-readable NS object. Item (v) follows from the window, scale, threshold, and evaluation policy definitions. Item (vi) is Proposition 6.30. Item (vii) is Proposition 6.34. Item (viii) is Proposition 6.40. Item (ix) follows from the exploration-to-Core re-entry definition and Proposition 6.45. Item (x) is Proposition 6.51. Item (xi) follows from Definition 6.48. Item (xii) is Proposition 6.55. Item (xiii) is Proposition 6.60. Items (xiv)–(xviii) follow from Definition 6.62, Proposition 6.63, Definition 6.64, Proposition 6.65, Definition 6.66, Proposition 6.68, Definition 6.69, Proposition 6.70, and Proposition 6.73. $\square$

NS-A.14. Explicit exclusions

The following are not asserted in Appendix NS-A.

- No Navier–Stokes existence theorem is proved.
- No Navier–Stokes uniqueness theorem is proved.
- No Navier–Stokes regularity theorem is proved.
- No Navier–Stokes smoothness theorem is proved.
- No non-blow-up theorem is proved.
- No blow-up theorem is proved.
- No singularity exclusion theorem is proved.
- No numerical simulation is treated as proof.
- No PDE observable is treated as a Core object without realization.

- No advisory indicator is treated as a Core verdict.
- No Φ_{NS} value is treated as a proof.
- No exploration pre-gate replaces B-Gate⁺.
- No exploration pre-gate replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No exploration pre-gate replaces eligibility.
- No exploration pre-gate replaces monitored Type IV diagnostics.
- No exploration pre-gate replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No NS Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No Core pass for NS-derived data is treated as a Navier–Stokes theorem.
- No normalization success is treated as PDE regularity.
- No discretized zero diagnostic is treated as continuum zero diagnostic without diagnostic convergence.
- No exploration artifact supports Core acceptance by itself.
- No global exactness of T_{τ}^{bar} is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift C_{τ} is assumed.

NS-A.15. Provenance and status

The material in this appendix depends on:

- (i) the interface protocol of Chapter 8;
- (ii) the non-claim discipline of Chapter 7;
- (iii) the repaired barcode-level collapse and constructible persistence framework of Appendix A;
- (iv) the admissible representative discipline of Appendix B;
- (v) the eligible realization regime of Appendix C;
- (vi) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;
- (vii) the manifest schema of Appendix G;
- (viii) the advisory-invariant discipline of Appendix H;
- (ix) the grid-to-continuum and discretization discipline of Appendix I;

- (x) the measurement-level stability discipline of Appendix J;
- (xi) the ledger catalog of Appendix M;
- (xii) the normalization interface of Appendix AGI-N.

Its role is to define a disciplined exploration protocol for Navier–Stokes-derived data without allowing exploratory evidence to become proof.

Status labels for Appendix NS-A. *Problem-interface definitions:* Definitions [6.4](#), [6.5](#), [6.6](#), [6.9](#), [6.10](#), [6.11](#), [6.12](#), [6.15](#), [6.16](#), [6.17](#), [6.18](#), [6.21](#), [6.22](#), [6.23](#), [6.24](#), [6.27](#), [6.28](#), [6.29](#), [6.32](#), [6.33](#), [6.36](#), [6.37](#), [6.38](#), [6.39](#), [6.42](#), [6.43](#), [6.44](#), [6.47](#), [6.48](#), [6.49](#), [6.50](#), [6.53](#), [6.54](#), [6.56](#), [6.57](#), [6.58](#), [6.59](#).

Problem-interface propositions/theorems: Propositions [6.19](#), [6.30](#), [6.34](#), [6.40](#), [6.45](#), [6.51](#), [6.55](#), [6.60](#), Theorem [6.76](#).

Operational cautions: Remarks [6.1](#), [6.2](#), [6.3](#), [6.7](#), [6.8](#), [6.13](#), [6.14](#), [6.20](#), [6.25](#), [6.26](#), [6.31](#), [6.35](#), [6.41](#), [6.46](#), [6.52](#), [6.61](#).

Additional v19.0 hardening definitions: Definitions [6.62](#), [6.64](#), [6.66](#), [6.67](#), [6.69](#), [6.71](#), [6.72](#).

Additional v19.0 hardening propositions: Propositions [6.63](#), [6.65](#), [6.68](#), [6.70](#), [6.73](#).

Additional v19.0 hardening remarks: Remarks [6.74](#), [6.75](#).

Explicit exclusions: Section NS-A.14.

7 Appendix NS-B: Navier–Stokes Theoretical Soundness Layer

NS-B.1. Scope and theoretical status

This appendix defines the theoretical soundness layer for Navier–Stokes interfaces in the AK framework. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A Core pass for NS-derived data is not a PDE theorem without a proved soundness bridge.

The purpose of this appendix is to state what must be supplied before exploration outputs, normalized objects, discretized data, measured observables, tower diagnostics, or Core certificates may be interpreted as evidence for a Navier–Stokes statement.

All Core-relevant NS-derived persistence data remain governed by the repaired windowed evaluation:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Whenever Type IV diagnostics are invoked, the relevant comparison must be a declared tower comparison artifact:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

Remark 7.1 (Interface status). All implications from Core-visible data to Navier–Stokes regularity, smoothness, uniqueness, continuation, non-blow-up, blow-up, or singularity exclusion are [Spec] by default. They become theorem-level statements only after the required soundness bridge and transport theorem are proved.

Remark 7.2 (Relation to Appendix NS-A). Appendix NS-A defined the exploration protocol. The present appendix does not repeat exploration. It determines which additional theoretical conditions are needed for exploration-derived or Core-derived data to have PDE meaning.

Remark 7.3 (No exact-collapse restoration). This appendix does not restore the old exact-collapse framework. In particular, it does not assume a global exact collapse functor, a positive-threshold Serre localization, or a global strict filtered lift. All soundness claims must be formulated relative to repaired evaluation, admissible representatives, declared tower artifacts, and ledgered defects.

NS-B.2. Navier–Stokes continuum target

Definition 7.4 (Navier–Stokes continuum target). A *Navier–Stokes continuum target* is a declared PDE object

$$(u, p)$$

on a declared spacetime domain

$$\Omega \times I,$$

where u is a velocity field and p is a pressure field satisfying the Navier–Stokes equations in a declared sense.

Definition 7.5 (Solution class). A *solution class* is the declared analytic class in which the continuum target is interpreted. Examples include:

- (i) smooth classical solutions;
- (ii) strong solutions;
- (iii) mild solutions;
- (iv) weak solutions;
- (v) Leray–Hopf weak solutions;
- (vi) local-in-time regular solutions;
- (vii) discretized approximants with a declared continuum limit.

Definition 7.6 (PDE target statement). A *PDE target statement* is the Navier–Stokes conclusion one intends to support or prove. Examples include:

- (i) regularity on a spacetime cylinder;
- (ii) smooth continuation beyond a time T ;
- (iii) exclusion of blow-up on a region;
- (iv) boundedness of a critical norm;
- (v) vanishing or control of a singular set;
- (vi) uniqueness in a declared solution class;
- (vii) stability under a declared approximation or normalization regime.

Definition 7.7 (PDE target record). A *PDE target record* contains:

- (i) the equation and domain;
- (ii) the boundary or decay conditions;
- (iii) the solution class;
- (iv) the intended PDE target statement;
- (v) the local or global region to which the statement applies;
- (vi) the analytic criteria intended to support the conclusion;
- (vii) artifact references.

Remark 7.8 (PDE target must be declared). Without a declared solution class and target statement, it is impossible to determine what theoretical soundness would mean. A Core certificate alone does not determine the PDE theorem being claimed.

NS-B.3. Continuum realization hypothesis

Definition 7.9 (Continuum realization hypothesis). A *continuum realization hypothesis* is a declared map or procedure

$$\mathfrak{R}_{\text{NS}}^{\text{cont}} : (u, p) \longrightarrow F_{\text{NS}}$$

or

$$\mathfrak{R}_{\text{NS}}^{\text{cont}} : (u, p) \longrightarrow P_{\text{NS}},$$

where

$$F_{\text{NS}} \in \text{FiltCh}(()k), \quad P_{\text{NS}} \in \text{Pers}_k^{\text{cons}},$$

and the realized object is compatible with the repaired AK Core protocol.

Definition 7.10 (Purpose-faithful PDE realization). A continuum realization is *purpose-faithful* for a PDE target statement if the Core-visible features it preserves are sufficient for the intended PDE soundness bridge. For example:

- (i) a vorticity-based realization must preserve the relevant vorticity concentration behavior;
- (ii) an energy-based realization must preserve the relevant local energy behavior;
- (iii) a scale-critical realization must preserve the relevant scaling-sensitive quantities;
- (iv) a singular-set realization must preserve the relevant local defect structure;
- (v) a pressure-sensitive realization must preserve the pressure behavior required by the target statement.

Definition 7.11 (Core-visible NS feature). A *Core-visible NS feature* is a repaired persistence, representative, diagnostic, artifact, or ledger feature of a realized NS object, such as:

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}}), \quad C_{\tau,W}F_{\text{NS}}, \quad \phi_{i,W,\tau}^{\text{NS}}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta.$$

Proposition 7.12 (No PDE interpretation without purpose-faithfulness). A *Core certificate for NS-derived data cannot be interpreted as evidence for a PDE target statement unless the realization is purpose-faithful for that target.*

Proof. A Core certificate concerns the realized persistence or filtered object. A PDE target statement concerns the original continuum solution. If the realization does not preserve the PDE features relevant to the target, no implication from Core behavior to the PDE target is justified. $\square$

Remark 7.13 (Purpose-faithful is target-relative). A realization may be purpose-faithful for detecting vorticity concentration but not for proving pressure regularity. It may be suitable for local exploration but not for global continuation. Thus purpose-faithfulness must be stated relative to the target theorem.

NS-B.4. Grid-to-continuum soundness

Definition 7.14 (NS discretized target). An *NS discretized target* is a grid, mesh, numerical field, simulation output, or finite-dimensional approximation

$$(u_h, p_h)$$

intended to approximate a continuum target

$$(u, p).$$

Definition 7.15 (NS grid-to-continuum soundness). A discretized Navier–Stokes realization satisfies *grid-to-continuum soundness* if:

- (i) the discretized targets (u_h, p_h) converge to a declared continuum target (u, p) in a stated topology or weak sense;
- (ii) the persistence realizations of (u_h, p_h) converge to the persistence realization of (u, p) in the Core-relevant sense;
- (iii) the repaired evaluations converge or are controlled:

$$\text{Eval}_{i,W,\tau}(F_{\text{NS},h}) \longrightarrow \text{Eval}_{i,W,\tau}(F_{\text{NS}});$$

- (iv) the discretization errors are bounded by declared ledger components

$$\delta_h^{\text{NS-disc}};$$

- (v) any diagnostic transfer from discretized to continuum data is justified.

Definition 7.16 (NS diagnostic convergence). *NS diagnostic convergence* holds if the monitored Type IV diagnostics computed from discretized NS towers converge to, or eventually agree with, the diagnostics of the declared continuum tower:

$$(\mu_{\text{Collapse}}^{(h)}, u_{\text{Collapse}}^{(h)}) \longrightarrow (\mu_{\text{Collapse}}, u_{\text{Collapse}})$$

in the declared diagnostic sense, with both sides computed from declared tower comparison artifacts.

Definition 7.17 (Continuum tower comparison artifact). A *continuum tower comparison artifact* is a declared comparison

$$\phi_{i,W,\tau}^{\text{cont}} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}^{\text{cont}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}^{\text{cont}})$$

associated to a continuum NS-derived tower.

Definition 7.18 (Discretized tower comparison artifact). A *discretized tower comparison artifact* is a declared comparison

$$\phi_{i,W,\tau}^{(h)} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}^{(h)}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}^{(h)})$$

associated to a discretized NS-derived tower at grid or resolution scale h .

Proposition 7.19 (Discretized zero diagnostics do not imply continuum zero diagnostics). *At a fixed grid scale h , the condition*

$$(\mu_{\text{Collapse}}^{(h)}, u_{\text{Collapse}}^{(h)}) = (0, 0)$$

does not imply

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

for the continuum target without NS diagnostic convergence.

Proof. A finite grid may fail to resolve limiting generic kernel or cokernel defects. Without a convergence theorem connecting discretized defect profiles to continuum defect profiles through declared tower comparison artifacts, no implication from discretized diagnostics to continuum diagnostics is valid. $\square$

Remark 7.20 (Relation to Appendix I). Appendix I established the general grid-to-continuum discipline. This section specializes that discipline to Navier–Stokes-derived data.

NS-B.5. Measurement and observable soundness

Definition 7.21 (Observable soundness). A PDE observable $O(u, p)$ is *sound for a target statement* if control of the observable in the declared sense is sufficient, through a proved analytic theorem, to support the target statement.

Definition 7.22 (Measurement soundness). A measured observable

$$\hat{O}$$

is *measurement-sound* if:

- (i) it approximates the ideal observable $O(u, p)$;
- (ii) the measurement error is bounded by a declared ledger component

$$\delta^{\text{NS-meas}};$$

- (iii) the error bound is small enough relative to the safety margin;
- (iv) the measurement artifacts are available for audit.

Definition 7.23 (Observable-to-regularity bridge). An *observable-to-regularity bridge* is a theorem or declared [Spec] statement of the form:

$$\text{control of selected PDE observables} \implies \text{regularity or continuation in the declared solution class.}$$

Definition 7.24 (Persistence-observable bridge). A *persistence-observable bridge* is a theorem or declared [Spec] statement connecting repaired persistence behavior,

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}}),$$

to control of a PDE observable relevant to the target statement.

Proposition 7.25 (Measured observable control is not PDE regularity by default). *Measured control of a PDE observable does not imply PDE regularity unless an observable-to-regularity bridge is proved and measurement error is ledgered.*

Proof. Measured control concerns approximate observable values. PDE regularity is an analytic property of the continuum solution. A theorem connecting the observable to regularity is required, and measurement errors must be accounted for. $\square$

Proposition 7.26 (Persistence control is not observable control by default). *Control of*

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}})$$

does not imply control of a PDE observable unless a persistence-observable bridge is supplied.

Proof. Repaired persistence behavior is Core-visible topological or filtered data. A PDE observable is an analytic quantity of the continuum solution. The two are not identical by definition. A bridge theorem is required to transport control from one to the other. $\square$

Remark 7.27 (Betti integral caution). A Betti integral or persistence summary derived from a PDE observable is a measurement-level or Core-readable quantity only after realization. It is not a regularity criterion unless a separate observable-to-regularity bridge proves that role.

NS-B.6. Energy, vorticity, pressure, and scale compatibility

Definition 7.28 (Energy-compatible realization). A realization is *energy-compatible* if the filtered or persistence structure preserves the local energy behavior relevant to the declared PDE target statement.

Definition 7.29 (Vorticity-compatible realization). A realization is *vorticity-compatible* if the filtered or persistence structure preserves the vorticity concentration behavior relevant to the declared PDE target statement.

Definition 7.30 (Pressure-compatible realization). A realization is *pressure-compatible* if the filtered or persistence structure preserves the pressure behavior relevant to the declared PDE target statement.

Definition 7.31 (Scale-compatible realization). A realization is *scale-compatible* if it respects the scaling relation required by the Navier–Stokes analysis being used, such as parabolic scaling or critical-norm scaling.

Definition 7.32 (Criticality-compatible realization). A realization is *criticality-compatible* if the Core-visible data preserve the distinction between subcritical, critical, and supercritical behavior relevant to the declared analytic criterion.

Proposition 7.33 (Scale compatibility is not automatic). *A filtered or persistence realization of NS data is not scale-compatible by default.*

Proof. Persistence parameters, spatial scales, temporal windows, and collapse thresholds may transform differently under Navier–Stokes scaling. Without an explicit scale policy and proof of compatibility, the realization need not preserve scale-critical structure. $\square$

Remark 7.34 (Compatibility is target-dependent). Energy compatibility, vorticity compatibility, pressure compatibility, scale compatibility, and criticality compatibility are not universal. Each must be stated relative to a target theorem and a chosen realization.

NS-B.7. The NS soundness bridge

Definition 7.35 (NS soundness bridge). An *NS soundness bridge* is a theorem or declared [Spec] statement connecting Core-visible behavior of a realized NS object to a PDE target statement. It has the schematic form:

Core certificate behavior of $F_{\text{NS}} \implies$ PDE target statement for (u, p) .

Definition 7.36 (Admissible NS soundness bridge). An NS soundness bridge is *admissible* only if it declares:

- (i) the solution class;
- (ii) the PDE target statement;
- (iii) the realization hypothesis;
- (iv) purpose-faithfulness of the realization;
- (v) repaired evaluation records used;
- (vi) admissible representative and eligibility data, if Ext^1 is invoked;
- (vii) declared tower comparison artifacts, if Type IV diagnostics are used;
- (viii) grid-to-continuum soundness, if discretized data are used;
- (ix) measurement soundness, if measured observables are used;
- (x) normalization transport, if normalized data are used;
- (xi) analytic criteria connecting the realized features to the PDE target;
- (xii) all ledgered defects;
- (xiii) the final logical implication.

Definition 7.37 (Bridge status). An NS soundness bridge has one of the following statuses:

- (i) *proved*: the implication is established under the declared hypotheses;
- (ii) *conditional*: the implication is established assuming explicitly listed external theorems or hypotheses;
- (iii) *speculative*: the implication is proposed but not proved;
- (iv) *failed*: the implication is known not to follow under the declared hypotheses;
- (v) *undeclared*: no bridge has been supplied.

Remark 7.38 (Bridge status vocabulary alignment). In cross-appendix comparisons, the bridge-status token *speculative* is aligned with [Spec], and *undeclared* is aligned with *undefined*. These alignments are vocabulary identifications only; they do not promote a bridge to theorem-level status.

Proposition 7.39 (No NS conclusion without soundness bridge). *No Navier–Stokes conclusion follows from a Core pass for NS-derived data without an admissible NS soundness bridge.*

Proof. A Core pass concerns the realized persistence or filtered object. A Navier–Stokes conclusion concerns the continuum PDE solution. The soundness bridge is exactly the theorem needed to connect these two levels. Without it, the implication is not established. $\square$

Remark 7.40 (Bridge is the bottleneck). The main theoretical difficulty is not producing exploratory indicators. It is proving that Core-visible behavior of the realized object implies a PDE target statement. This is the soundness bottleneck.

NS-B.8. Eligible regime and Ext^1 use

Definition 7.41 (NS eligible regime). A Navier–Stokes interface is in the *NS eligible regime* on $(F; W, \tau)$ if:

- (i) the NS realization produces an admissible representative

$$C_{\tau, W}F;$$

- (ii) for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau, W}F) \cong \text{Eval}_{i, W, \tau}(F);$$

- (iii) the realization

$$\mathcal{R}(C_{\tau, W}F)$$

is defined in $D^b(k\text{-mod})$;

- (iv) one has

$$\mathcal{R}(C_{\tau, W}F) \in D^{[-1, 0]}(k\text{-mod});$$

- (v) the edge-realization condition holds:

$$H^{-1}(\mathcal{R}(C_{\tau, W}F)) \cong H_1^{\tau, W}(F).$$

Proposition 7.42 (NS Ext^1 -use requires eligibility). A Navier–Stokes interface may invoke the Core one-way bridge

$$\text{Eval}_{1, W, \tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0$$

only in the NS eligible regime.

Proof. The Core one-way bridge is proved only in the eligible regime of Chapter 4 and Appendix C. Navier–Stokes origin does not weaken or replace those hypotheses. $\square$

Remark 7.43 (PDE cohomology is not Core Ext^1). Any PDE-derived cohomological obstruction, weak-solution defect, or analytic obstruction is not automatically the Core group

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k).$$

A precise realization and eligibility proof are required.

NS-B.9. Normalization soundness

Definition 7.44 (NS normalization soundness). A resolution-assisted normalization used in a Navier–Stokes interface is *sound* if:

- (i) the normalization operation is defined on the declared source data;
- (ii) all auxiliary choices are declared;
- (iii) the normalization defects are ledgered;
- (iv) the normalized object is Core-readable;
- (v) the normalized repaired evaluation

$$\text{Eval}_{i, W, \tau}(F_{\text{NS}}^{\text{nor}})$$

is declared when used;

- (vi) a normalized-to-source transport statement is supplied when conclusions about the original PDE solution are intended.

Definition 7.45 (Normalized-to-source NS bridge). A *normalized-to-source NS bridge* is a theorem or declared [Spec] statement of the form:

Core-visible behavior of normalized NS data $\implies$ PDE conclusion for the original NS target.

Proposition 7.46 (Normalization soundness does not imply PDE regularity). *Even sound normalization does not imply Navier–Stokes regularity unless a normalized-to-source regularity bridge is proved.*

Proof. Sound normalization prepares a normalized Core-readable object with ledgered defects. Regularity is a statement about the original continuum solution. A theorem transferring normalized behavior back to the original solution is required. $\square$

Remark 7.47 (Relation to Appendix AGI-N). Appendix AGI-N gives the general normalization interface. This section states the additional soundness requirement when normalization is used for Navier–Stokes conclusions.

NS-B.10. Type IV soundness for NS towers

Definition 7.48 (NS continuum tower). An *NS continuum tower* is a directed tower of Core-readable objects derived from continuum Navier–Stokes data, such as scale towers, time-window towers, local-cylinder towers, normalized refinement towers, or blow-up rescaling towers.

Definition 7.49 (NS Type IV soundness). An NS tower satisfies *Type IV soundness* if the monitored Type IV diagnostics of the realized tower correspond, through a declared soundness bridge, to the limiting PDE behavior being studied.

Definition 7.50 (Type IV-to-PDE bridge). A *Type IV-to-PDE bridge* is a theorem or declared [Spec] statement of the form:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0) \implies \text{declared PDE target statement.}$$

Proposition 7.51 (Type IV cleanliness is not PDE regularity by default). *The condition*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

for an NS-derived tower does not imply PDE regularity unless Type IV soundness and an appropriate regularity bridge are proved.

Proof. Type IV cleanliness is a Core diagnostic statement about declared tower comparison artifacts. PDE regularity is an analytic statement about Navier–Stokes solutions. The former implies the latter only through a theorem connecting the diagnostic to the PDE target. $\square$

Remark 7.52 (Type IV may be useful but not sufficient). Type IV diagnostics may be valuable for detecting limiting mismatch between finite-stage and limiting structures. However, their PDE meaning must be established separately.

NS-B.11. Theoretical soundness record

Definition 7.53 (NS theoretical soundness record). An *NS theoretical soundness record* is a manifest-level record containing:

- (i) the continuum target (u, p) ;
- (ii) the solution class;
- (iii) the PDE target statement;
- (iv) the realization hypothesis;
- (v) purpose-faithfulness evidence;
- (vi) repaired evaluation records used;
- (vii) representative and eligibility data, if Ext^1 is invoked;
- (viii) tower comparison artifacts, if Type IV diagnostics are used;
- (ix) grid-to-continuum soundness data, if applicable;
- (x) measurement soundness data, if applicable;
- (xi) normalization soundness data, if applicable;
- (xii) Type IV soundness data, if tower diagnostics are used;
- (xiii) the declared NS soundness bridge;
- (xiv) bridge status;
- (xv) ledger entries;
- (xvi) artifact references;
- (xvii) status labels for all problem-specific implications.

Proposition 7.54 (Soundness record is required for PDE interpretation). A *Core certificate for NS-derived data cannot be interpreted as a Navier–Stokes theorem without an NS theoretical soundness record*.

Proof. The soundness record supplies the continuum target, solution class, realization, purpose-faithfulness, error accounting, and bridge theorem needed to interpret Core behavior as a PDE statement. Without it, the interpretation is not audit-readable. $\square$

Remark 7.55 (Soundness record is not proof by itself). A soundness record may contain [Spec] statements. Only those bridge components that are actually proved can support theorem-level conclusions.

NS-B.12. Core pass, soundness pass, and PDE theorem

Definition 7.56 (NS soundness pass). An *NS soundness pass* means that the theoretical soundness record supplies all required hypotheses and proved bridge statements needed to interpret a Core certificate as a PDE conclusion.

Definition 7.57 (NS theorem-level pass). An *NS theorem-level pass* means:

- (i) the NS Core certificate passes;
- (ii) the NS theoretical soundness record passes;
- (iii) the relevant NS soundness bridge is proved;
- (iv) the target PDE statement follows in the declared solution class.

Definition 7.58 (NS theorem-level failure). An *NS theorem-level failure* occurs if:

- (i) the Core certificate fails;
- (ii) the soundness record is missing;
- (iii) the bridge is speculative, failed, or undeclared;
- (iv) the bridge hypotheses are not satisfied;
- (v) ledgered defects exceed the declared budget;
- (vi) required artifacts are missing.

Proposition 7.59 (Core pass plus unproved soundness bridge is not theorem-level pass). A *Core pass together with an unproved [Spec] soundness bridge does not constitute an NS theorem-level pass*.

Proof. A theorem-level pass requires a proved bridge. A [Spec] bridge is a specification or conjectural interface statement. Therefore it cannot supply theorem-level implication. $\square$

Remark 7.60 (Layer separation). The following must remain distinct:

exploration pass, Core pass, soundness pass, PDE theorem.

This appendix concerns the third layer.

NS-B.12bis. v19 hardening: soundness sovereignty and bridge promotion

Definition 7.61 (NS soundness sovereignty). *NS soundness sovereignty* is the rule that PDE interpretation is governed by proved soundness bridges, not by Core pass, exploration success, numerical convergence, or terminology.

Proposition 7.62 (No soundness sovereignty override). *No Core-visible NS feature can be interpreted as a PDE theorem unless the declared soundness bridge proves the corresponding implication in the declared solution class*.

Proof. Core-visible features and PDE theorems live in different semantic layers. The soundness bridge is the required logical map between them. $\square$

Definition 7.63 (NS bridge promotion record). An *NS bridge promotion record* records a proposed promotion of a bridge from [Spec] or conditional status to theorem-level status, including the hypotheses, proof artifacts, external analytic theorems, and verification of all ledgered defects.

Proposition 7.64 (Bridge names do not promote bridge status). *Calling a statement a regularity bridge, diagnostic bridge, Type IV-to-PDE bridge, or observable bridge does not promote it beyond [Spec] status.*

Proof. Status depends on proof, hypotheses, and artifact records, not on nomenclature. $\square$

Definition 7.65 (NS diagnostic convergence mode). An NS diagnostic comparison between discretized and continuum towers has one of the following modes:

- (i) *not-compared*;
- (ii) *sampled*;
- (iii) *bounded-approximation*;
- (iv) *eventual-agreement*;
- (v) *proved-convergence*;
- (vi) *failed*;
- (vii) *undefined*.

Proposition 7.66 (Sampled diagnostic agreement is not diagnostic convergence). *Agreement of Type IV diagnostics at finitely many grid scales is not NS diagnostic convergence unless an eventual-agreement or convergence theorem is supplied.*

Proof. Finite agreement does not control unresolved limiting kernel or cokernel defect profiles. Diagnostic convergence is a tower-level statement involving declared artifacts. $\square$

Definition 7.67 (NS compatibility burden). The *NS compatibility burden* is the obligation to prove, relative to the target theorem, that energy, vorticity, pressure, scale, and criticality information survive the realization route used in the Core certificate.

Proposition 7.68 (Compatibility is non-scalar). *Missing energy, vorticity, pressure, scale, or criticality compatibility cannot be repaired by a small scalar ledger margin.*

Proof. Compatibility is a typed semantic condition connecting PDE quantities to realized Core-visible features. A scalar budget can bound declared discrepancies, but it cannot supply a missing semantic theorem. $\square$

Definition 7.69 (NS theorem-promotion schema). An *NS theorem-promotion schema* consists of a Core certificate, a soundness record, a proved admissible bridge, all required continuum/grid/measurement/nor-malization soundness clauses, and a final derivation of the PDE target statement.

Proposition 7.70 (Core pass plus incomplete soundness is non-theorem). *A Core pass together with incomplete, conditional-with-unmet-hypotheses, speculative, failed, or undeclared soundness data is not a theorem-level NS result.*

Proof. Theorem-level status requires a complete proved implication from the Core certificate to the PDE target statement. Incomplete soundness leaves the implication unproved. $\square$

Remark 7.71 (Soundness bridge remains the bottleneck). The v19 interface can discipline exploration and certification, but it does not remove the analytic burden of proving the soundness bridge.

Remark 7.72 (NS-B ledger namespace registration). Defects used by NS-B, including discretization, measurement, realization, normalization, observable, diagnostic-convergence, and bridge-related defects, must be registered in Appendix M or explicitly related to the AGI-N and NS-A namespaces by alias, refinement, aggregation, or disjointness declarations.

NS-B.13. Summary of theoretical soundness layer

Theorem 7.73 (Appendix NS-B theoretical soundness package). *Within the Problem Interface layer, the following package is available.*

- (i) *A Navier–Stokes continuum target, solution class, and target statement must be declared before PDE interpretation.*
- (ii) *A continuum realization must be purpose-faithful for the declared PDE target.*
- (iii) *Core-visible NS features must be expressed through repaired evaluation, admissible representatives when invoked, declared tower artifacts when used, diagnostics, and ledger records.*
- (iv) *Discretized data require grid-to-continuum soundness.*
- (v) *Discretized zero diagnostics do not imply continuum zero diagnostics without diagnostic convergence.*
- (vi) *Measured observable control does not imply PDE regularity without an observable-to-regularity bridge and ledgered measurement error.*
- (vii) *Persistence control does not imply observable control without a persistence-observable bridge.*
- (viii) *Energy, vorticity, pressure, scale, and criticality compatibility are target-dependent and must be declared.*
- (ix) *No Navier–Stokes conclusion follows from Core pass without an admissible NS soundness bridge.*
- (x) *The Core Ext¹-clause may be invoked only in the NS eligible regime.*
- (xi) *Sound normalization does not imply PDE regularity without a normalized-to-source regularity bridge.*
- (xii) *Type IV cleanliness is not PDE regularity by default.*
- (xiii) *An NS theoretical soundness record is required for PDE interpretation.*
- (xiv) *Core pass plus unproved [Spec] soundness bridge is not theorem-level pass.*
- (xv) *NS soundness sovereignty prevents Core pass, exploration success, and diagnostic terminology from becoming PDE interpretation without a proved bridge.*
- (xvi) *Bridge-promotion requires an explicit promotion record and proof artifacts.*
- (xvii) *Sampled diagnostic agreement is not diagnostic convergence.*
- (xviii) *Compatibility with energy, vorticity, pressure, scale, and criticality is a non-scalar obligation.*
- (xix) *Theorem-level promotion requires the full NS theorem-promotion schema.*

Proof. Items (i)–(ii) follow from the definitions of continuum target and purpose-faithful realization. Item (iii) follows from the definition of Core-visible NS feature. Items (iv)–(v) follow from grid-to-continuum soundness and Proposition 7.19. Item (vi) is Proposition 7.25. Item (vii) is Proposition 7.26. Item (viii) follows from the compatibility definitions. Item (ix) is Proposition 7.39. Item (x) is Proposition 7.42. Item (xi) is Proposition 7.46. Item (xii) is Proposition 7.51. Item (xiii) is Proposition 7.54. Item (xiv) is Proposition 7.59. Items (xv)–(xix) follow from Definition 7.61, Proposition 7.62, Definition 7.63, Proposition 7.64, Definition 7.65, Proposition 7.66, Definition 7.67, Proposition 7.68, Definition 7.69, and Proposition 7.70. $\square$

NS-B.14. Explicit exclusions

The following are not asserted in Appendix NS-B.

- No Navier–Stokes regularity theorem is proved.
- No smoothness theorem is proved.
- No uniqueness theorem is proved.
- No existence theorem is proved.
- No continuation theorem is proved.
- No non-blow-up theorem is proved.
- No blow-up theorem is proved.
- No singularity exclusion theorem is proved.
- No Core pass is interpreted as a PDE theorem without a proved soundness bridge.
- No discretized diagnostic is treated as continuum diagnostic without diagnostic convergence.
- No numerical convergence is assumed without a grid-to-continuum statement.
- No measured observable is treated as exact without measurement ledger.
- No vorticity, energy, pressure, scale, or criticality observable is treated as a regularity criterion without an analytic bridge.
- No repaired persistence control is treated as observable control without a persistence-observable bridge.
- No normalization success is treated as regularity.
- No Type IV cleanliness is treated as PDE regularity by default.
- No PDE-derived cohomological obstruction is automatically identified with Core Ext¹.
- No [Spec] soundness bridge is treated as a theorem.
- No exploration artifact supports theorem-level PDE conclusion by itself.
- No global exactness of T_τ^{bar} is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift C_τ is assumed.

NS-B.15. Provenance and status

The material in this appendix depends on:

- (i) the Navier–Stokes exploration protocol of Appendix NS-A;
- (ii) the normalization interface of Appendix AGI-N;
- (iii) the repaired barcode-level collapse and constructible persistence framework of Appendix A;
- (iv) the admissible representative discipline of Appendix B;
- (v) the eligible realization regime of Appendix C;
- (vi) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;
- (vii) the manifest schema of Appendix G;
- (viii) the grid-to-continuum discipline of Appendix I;
- (ix) the measurement-level stability discipline of Appendix J;
- (x) the ledger catalog of Appendix M;
- (xi) the interface protocol of Chapter 8;
- (xii) the non-claim discipline of Chapter 7.

Its role is to state the theoretical conditions required before NS-derived Core certificates may be interpreted as PDE-relevant statements.

Status labels for Appendix NS-B. *Problem-interface definitions:* Definitions [7.4](#), [7.5](#), [7.6](#), [7.7](#), [7.9](#), [7.10](#), [7.11](#), [7.14](#), [7.15](#), [7.16](#), [7.17](#), [7.18](#), [7.21](#), [7.22](#), [7.23](#), [7.24](#), [7.28](#), [7.29](#), [7.30](#), [7.31](#), [7.32](#), [7.35](#), [7.36](#), [7.37](#), [7.41](#), [7.44](#), [7.45](#), [7.48](#), [7.49](#), [7.50](#), [7.53](#), [7.56](#), [7.57](#), [7.58](#).

Problem-interface propositions/theorems: Propositions [7.12](#), [7.19](#), [7.25](#), [7.26](#), [7.33](#), [7.39](#), [7.42](#), [7.46](#), [7.51](#), [7.54](#), [7.59](#), Theorem [7.73](#).

Operational cautions: Remarks [7.1](#), [7.2](#), [7.3](#), [7.8](#), [7.13](#), [7.20](#), [7.27](#), [7.34](#), [7.40](#), [7.43](#), [7.47](#), [7.52](#), [7.55](#), [7.60](#).

Additional v19.0 hardening definitions: Definitions [7.61](#), [7.63](#), [7.65](#), [7.67](#), [7.69](#).

Additional v19.0 hardening propositions: Propositions [7.62](#), [7.64](#), [7.66](#), [7.68](#), [7.70](#).

Additional v19.0 hardening remarks: Remarks [7.71](#), [7.72](#), [7.38](#).

Explicit exclusions: Section NS-B.14.

8 Appendix NS-C: Navier–Stokes Proof-First Program

NS-C.1. Scope and proof-first status

This appendix defines a proof-first program for Navier–Stokes interfaces in the AK framework. It belongs to the Problem Interface layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A proof-first program is a proof architecture, not a completed proof.

The purpose of this appendix is to state, in an auditable way, what must be proved before an AK-based Navier–Stokes interface can support a theorem-level Navier–Stokes conclusion.

All Core-side proof obligations must be expressed through the repaired Core vocabulary:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)),$$

admissible representatives

$$C_{\tau,W}F,$$

declared tower comparison artifacts

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}),$$

and the audit budget condition

$$\text{val}_B(\Sigma\delta) < \text{gap}_{\tau}(\mathfrak{B}).$$

Remark 8.1 (Proof-first status). This appendix does not prove Navier–Stokes regularity, smoothness, uniqueness, continuation, non-blow-up, blow-up, or singularity exclusion. It defines the proof obligations required before such claims may be made.

Remark 8.2 (Relation to Appendices NS-A and NS-B). Appendix NS-A defines exploration. Appendix NS-B defines theoretical soundness. The present appendix reverses the priority: rather than beginning from exploratory outputs, it begins from the target theorem and lists the required bridges, diagnostics, Core certificates, and proof objects.

Remark 8.3 (No exact-collapse restoration). This appendix does not restore the old exact-collapse framework. No global exact collapse functor, positive-threshold Serre localization, or global strict filtered lift C_{τ} is assumed. All proof-first obligations must be stated in terms of repaired evaluation, admissible representatives, declared tower artifacts, and ledgered defects.

NS-C.2. Proof-first principle

Definition 8.4 (Proof-first principle). The *proof-first principle* states that an AK-based Navier–Stokes program must begin by declaring:

- (i) the target PDE theorem;
- (ii) the solution class;
- (iii) the PDE assumptions;
- (iv) the Core-readable realization route;
- (v) the repaired evaluation records required;

- (vi) the admissible representative data required, if any;
- (vii) the declared tower comparison artifacts required, if any;
- (viii) the soundness bridge;
- (ix) the required AK Core certificate;
- (x) all external analytic lemmas;
- (xi) all ledgered defects;
- (xii) all failure conditions;
- (xiii) the proof object schema.

Exploration may support this program, but it may not determine the theorem.

Definition 8.5 (Proof-first record). *A proof-first record is a manifest-level record that lists:*

- (i) the target theorem;
- (ii) required analytic hypotheses;
- (iii) required realization bridge;
- (iv) required repaired evaluation records;
- (v) required Core certificates;
- (vi) required soundness bridges;
- (vii) required transport statements;
- (viii) required tower comparison artifacts;
- (ix) required ledger conditions;
- (x) unresolved gaps;
- (xi) status labels.

Proposition 8.6 (Exploration cannot replace proof-first obligations). *No exploration output, advisory indicator, numerical result, normalization success, Type IV proxy, or Core candidate replaces the proof obligations declared in a proof-first record.*

Proof. Exploration outputs may identify candidates or suggest structures. Proof obligations are logical requirements for theorem-level implication. Candidate generation does not discharge those requirements. $\square$

Remark 8.7 (Why proof-first is required). For a Clay-level PDE problem, exploratory agreement is insufficient. The architecture must specify the theorem, hypotheses, analytic bridges, Core-side certificate conditions, and proof-object requirements before any theorem-level conclusion is claimed.

NS-C.3. Target theorem templates

Definition 8.8 (Navier–Stokes target theorem template). A *Navier–Stokes target theorem template* is a formal statement of the intended PDE conclusion. It must specify:

- (i) the spatial dimension;
- (ii) the domain;
- (iii) the boundary or decay conditions;
- (iv) the viscosity parameter;
- (v) the initial data class;
- (vi) the solution class;
- (vii) the time interval;
- (viii) the exact conclusion to be proved.

Definition 8.9 (Local regularity target). A *local regularity target* is a theorem template asserting regularity or smoothness of a solution on a declared spacetime cylinder

$$Q_r(x_0, t_0) = B_r(x_0) \times (t_0 - r^2, t_0).$$

Definition 8.10 (Continuation target). A *continuation target* is a theorem template asserting that a solution can be continued beyond a time T under declared conditions.

Definition 8.11 (Non-blow-up target). A *non-blow-up target* is a theorem template asserting that no singularity forms in a declared region or time interval.

Definition 8.12 (Global regularity target). A *global regularity target* is a theorem template asserting global smoothness or regularity for all time in a declared solution class.

Definition 8.13 (Clay-level target). A *Clay-level target* is a theorem template matching the global existence and smoothness problem for the three-dimensional incompressible Navier–Stokes equations in a standard Clay-compatible formulation.

Remark 8.14 (Global targets are strongest). A global regularity target is substantially stronger than a local regularity target. The proof-first program should generally treat local, continuation, non-blow-up, global, and Clay-level targets as distinct stages.

NS-C.4. Core certificate target

Definition 8.15 (NS Core certificate target). An *NS Core certificate target* is the Core-side certificate statement required by the proof program. It has the schematic form:

$$C_{\text{NS}} \models B\text{-Gate}^+ \wedge (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0) \wedge (\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B})),$$

for a declared Core-readable Navier–Stokes object.

Definition 8.16 (NS Core object of proof). The *NS Core object of proof* is the filtered or persistence object

$$F_{\text{NS}} \quad \text{or} \quad P_{\text{NS}}$$

obtained from the continuum Navier–Stokes target through the declared realization route and used in the Core certificate target.

Definition 8.17 (NS repaired evaluation proof target). An *NS repaired evaluation proof target* is a required verifier-side statement of the form:

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}}) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F_{\text{NS}})),$$

together with the required vanishing or comparison condition in monitored degrees.

Definition 8.18 (NS tower proof target). An *NS tower proof target* is a required verifier-side tower statement involving a declared artifact

$$\phi_{i,W,\tau}^{\text{NS}} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}^{\text{NS}}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}^{\text{NS}})$$

and the diagnostic condition

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0).$$

Definition 8.19 (Core certificate sufficiency claim). A *Core certificate sufficiency claim* is a statement of the form:

$$C_{\text{NS}} \text{ passes Core} \implies \text{PDE target theorem.}$$

Proposition 8.20 (Core certificate sufficiency is [Spec] by default). *Every Core certificate sufficiency claim for Navier–Stokes is [Spec] by default unless proved by an admissible NS soundness bridge.*

Proof. A Core certificate is a statement about repaired persistence, admissible representatives, diagnostics, artifacts, and budget. A Navier–Stokes theorem is a statement about PDE solutions. The implication requires an analytic soundness bridge. Without that bridge, it remains [Spec]. $\square$

Remark 8.21 (Core pass is necessary only if declared). The proof program may declare Core pass as a sufficient condition, a necessary condition, or an auxiliary certificate. These logical roles must not be confused.

NS-C.5. Required bridge lemmas

Definition 8.22 (Bridge lemma). A *bridge lemma* is a theorem connecting one layer of the proof program to another. For Navier–Stokes, bridge lemmas may connect:

$$\text{PDE observables} \leftrightarrow \text{persistence realization} \leftrightarrow \text{Core certificate} \leftrightarrow \text{PDE target conclusion.}$$

Definition 8.23 (Realization bridge). A *realization bridge* proves that the chosen PDE-to-persistence realization is purpose-faithful for the target theorem.

Definition 8.24 (Evaluation bridge). An *evaluation bridge* proves that the repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}})$$

capture the PDE-relevant features required by the proof route.

Definition 8.25 (Collapse bridge). A *collapse bridge* proves that the barcode-level after-threshold Core data preserve or detect the PDE-relevant obstruction or regularity feature needed by the proof.

Definition 8.26 (Diagnostic bridge). A *diagnostic bridge* proves that monitored Type IV cleanliness or failure, computed from the declared artifact

$$\phi_{i,W,\tau}^{\text{NS}},$$

corresponds to the intended PDE limiting behavior.

Definition 8.27 (Regularity bridge). A *regularity bridge* proves that the verified Core-visible condition implies the declared Navier–Stokes regularity, continuation, or non-blow-up target.

Definition 8.28 (Compactness bridge). A *compactness bridge* proves that limiting behavior of realized or normalized objects is compatible with the required continuum PDE compactness, convergence, or weak-to-strong passage.

Definition 8.29 (Global assembly bridge). A *global assembly bridge* proves that local or windowed Core certificates assemble into the global PDE conclusion required by a global or Clay-level target.

Remark 8.30 (Bridge lemmas are the main mathematical burden). The AK Core can certify its own internal statements. The hard PDE work lies in proving the bridge lemmas that connect Core-visible statements to Navier–Stokes regularity.

NS-C.6. Required analytic hypotheses

Definition 8.31 (Analytic hypothesis set). An *analytic hypothesis set* is the collection of PDE-side assumptions required by the proof-first program. It may include:

- (i) energy inequality;
- (ii) local energy inequality;
- (iii) weak compactness;
- (iv) pressure decomposition;
- (v) vorticity equation control;
- (vi) scale-critical norm bounds;
- (vii) interpolation inequalities;
- (viii) ε -regularity criterion;
- (ix) backward uniqueness;
- (x) continuation criterion;
- (xi) uniqueness or weak-strong uniqueness assumptions;
- (xii) local-to-global assembly assumptions.

Definition 8.32 (External analytic theorem). An *external analytic theorem* is a known or separately proved PDE theorem used by the proof-first program. It must be cited, stated, or imported with its hypotheses.

Definition 8.33 (Analytic gap). An *analytic gap* is a missing or unproved PDE-side statement required to complete the proof-first program.

Definition 8.34 (Core-side gap). A *Core-side gap* is a missing or unverified AK-side requirement, such as missing repaired evaluation records, missing admissible representatives, missing eligibility, missing tower artifacts, failed diagnostics, or failed budget comparison.

Definition 8.35 (Bridge gap). A *bridge gap* is a missing or unproved implication between PDE-side data and Core-side data, or between Core-side pass and a PDE target conclusion.

Proposition 8.36 (Analytic gaps prevent theorem-level pass). *If the proof-first program contains an unresolved analytic gap that is necessary for the target theorem, then theorem-level pass is not obtained.*

Proof. A theorem-level pass requires all required implications to be proved. An unresolved necessary analytic gap means at least one implication is missing. Therefore the theorem-level conclusion does not follow. $\square$

Proposition 8.37 (Bridge gaps prevent theorem-level pass). *If the proof-first program contains an unresolved bridge gap necessary for the target theorem, then theorem-level pass is not obtained.*

Proof. A bridge gap means that one of the required cross-layer implications is missing. Since theorem-level status requires a complete chain from PDE assumptions through Core certification to the PDE conclusion, the target theorem does not follow. $\square$

Remark 8.38 (Gaps should be explicit). A proof-first record should expose analytic, Core-side, and bridge gaps rather than hide them. A visible gap is preferable to an unstated theorem-level claim.

NS-C.7. Continuum, discretization, and proof status

Definition 8.39 (Continuum proof mode). A proof-first program is in *continuum proof mode* if all Core-readable objects and diagnostics are derived from continuum Navier–Stokes data through a proved continuum realization hypothesis.

Definition 8.40 (Discretized proof-support mode). A proof-first program is in *discretized proof-support mode* if discretized or numerical data support exploration, conjecture formation, candidate selection, or empirical checking, but theorem-level claims still require independent continuum soundness.

Definition 8.41 (Discretized theorem mode). A proof-first program is in *discretized theorem mode* only if grid-to-continuum convergence, repaired evaluation convergence, diagnostic convergence, measurement soundness, and all analytic transport bridges are proved sufficiently to support theorem-level claims.

Definition 8.42 (Hybrid proof-support mode). A proof-first program is in *hybrid proof-support mode* if continuum arguments provide the main theorem route while discretized or numerical artifacts provide auxiliary verification, debugging, or candidate selection.

Proposition 8.43 (Discretized proof-support mode is not theorem mode). *Discretized proof-support mode does not yield theorem-level Navier–Stokes conclusions.*

Proof. Discretized proof-support mode allows numerical or finite-resolution data to support exploration. Theorem-level conclusions require continuum statements and proved grid-to-continuum and soundness bridges. Therefore proof-support mode is not theorem mode. $\square$

Remark 8.44 (Preferred theorem route). The preferred route for a theorem-level claim is continuum proof mode. Discretized data may remain valuable for exploration, but the proof should ultimately close at the continuum level or include a fully proved grid-to-continuum bridge.

NS-C.8. Failure modes

Definition 8.45 (Proof-first failure mode). A *proof-first failure mode* is a declared way in which the Navier–Stokes proof-first program may fail to establish its target theorem.

Definition 8.46 (Realization failure). *Realization failure* occurs when the PDE source data cannot be realized as a purpose-faithful Core-readable object.

Definition 8.47 (Evaluation failure). *Evaluation failure* occurs when repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}})$$

are missing, nonconstructible, unstable beyond budget, or fail the required Core-side condition.

Definition 8.48 (Eligibility failure). *Eligibility failure* occurs when the realized object does not satisfy the eligible regime required for the Core Ext¹-clause.

Definition 8.49 (Tower artifact failure). *Tower artifact failure* occurs when Type IV diagnostics are invoked but the required tower comparison artifact

$$\phi_{i,W,\tau}^{\text{NS}}$$

is missing, ill-defined, or not audit-readable.

Definition 8.50 (Diagnostic failure). *Diagnostic failure* occurs when the monitored Type IV diagnostics fail:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0).$$

Definition 8.51 (Budget failure). *Budget failure* occurs when the declared total budget fails the strict inequality:

$$\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$

Definition 8.52 (Soundness failure). *Soundness failure* occurs when the Core certificate passes internally but no proved soundness bridge connects it to the PDE target theorem.

Definition 8.53 (Transport failure). *Transport failure* occurs when a normalized, discretized, localized, or transformed object passes a relevant check, but the required implication back to the original continuum PDE target is missing or false.

Proposition 8.54 (Soundness failure blocks PDE conclusion). *If soundness failure occurs, then no Navier–Stokes theorem-level conclusion follows, even if the Core certificate passes.*

Proof. Soundness failure means that the implication from Core behavior to the PDE target theorem is not proved. Therefore the Core pass remains internal to the AK certificate and does not yield the PDE theorem. $\square$

Remark 8.55 (Failure classification is constructive). A failure mode is not merely negative. It identifies which proof obligation must be repaired: realization, repaired evaluation, eligibility, tower artifact, diagnostics, budget, soundness, or transport.

NS-C.9. Proof object requirements

Definition 8.56 (NS proof object). An *NS proof object* is a structured record intended to support a theorem-level Navier–Stokes conclusion through the AK interface. It must include both Core certificate data and PDE-side soundness data.

Definition 8.57 (NS proof object schema). An NS proof object must contain:

- (i) target theorem template;
- (ii) solution class;
- (iii) continuum source data;
- (iv) realization bridge;
- (v) Core-readable object;
- (vi) declared windows, thresholds, and monitored degrees;
- (vii) repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F_{\text{NS}});$$
- (viii) admissible representative data

$$C_{\tau,W}F_{\text{NS}}$$
 if representatives are invoked;
- (ix) eligibility data, if Ext^1 is invoked;
- (x) tower comparison artifacts

$$\phi_{i,W,\tau}^{\text{NS}}$$
 if Type IV diagnostics are invoked;
- (xi) Core certificate target;
- (xii) monitored Type IV diagnostics;
- (xiii) budget ledger;
- (xiv) Core gate verdicts;
- (xv) regularity bridge;
- (xvi) compactness bridge, if needed;
- (xvii) global assembly bridge, if needed;
- (xviii) normalization or discretization transport, if used;
- (xix) artifact references;
- (xx) status labels for every bridge and claim.

Definition 8.58 (Theorem-level NS proof object). An NS proof object is *theorem-level* if every required bridge, transport statement, diagnostic, ledger condition, Core gate condition, artifact requirement, and PDE-side analytic condition is proved or verified under the declared hypotheses.

Definition 8.59 (Proof object status). An NS proof object has one of the following statuses:

- (i) *schema-only*: the required fields are declared but not filled;
- (ii) *partial*: some proof fields are filled and some gaps remain;
- (iii) *Core-complete*: Core fields are complete but PDE-side soundness is incomplete;
- (iv) *soundness-complete*: soundness bridges are proved but Core fields remain incomplete;
- (v) *theorem-level*: all required components are complete and proved;
- (vi) *failed*: at least one required component fails.

Proposition 8.60 (Incomplete NS proof object is not theorem-level). *An incomplete NS proof object is not theorem-level.*

Proof. A theorem-level proof object requires all necessary logical components. If any required bridge, diagnostic, ledger, artifact, or analytic component is missing, the proof object cannot establish the target theorem. $\square$

Remark 8.61 (Status labels are mandatory). Every component of an NS proof object should be labeled as Core theorem, interface theorem, operational policy, [Spec], search artifact, or non-claim. This prevents conjectural bridges from being read as proved theorems.

NS-C.10. Minimal theorem route

Definition 8.62 (Minimal theorem route). A *minimal theorem route* is the shortest chain of proved implications from the target theorem template assumptions to the PDE conclusion through AK Core certification. It has the schematic form:

PDE assumptions $\Rightarrow$ purpose-faithful Core realization $\Rightarrow$ repaired Core certificate pass $\Rightarrow$ proved soundness bridge $\Rightarrow$ PDE conclusion

Definition 8.63 (Core route segment). A *Core route segment* is the part of a minimal theorem route that proves the required repaired Core certificate:

$$\text{Eval}_{I,W,\tau}(F_{\text{NS}}) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}),$$

together with any required representative, eligibility, and artifact conditions.

Definition 8.64 (PDE route segment). A *PDE route segment* is the part of a minimal theorem route that proves the PDE-side analytic implication from the Core certificate or Core-visible condition to the target PDE conclusion.

Definition 8.65 (Route redundancy). A proof route has *redundancy* if it includes advisory indicators, numerical evidence, alternative normalizations, categorical comparisons, or search artifacts not logically required for the theorem-level implication.

Proposition 8.66 (Minimal route should separate proof from support). *A minimal theorem route should separate logically necessary proof components from supportive exploration artifacts.*

Proof. Theorem-level validity depends only on necessary proved implications. Exploration artifacts may motivate or support discovery but should not be mixed with the proof chain unless they are converted into verified proof components. $\square$

Remark 8.67 (Usefulness of redundant support). Redundant support may improve confidence, debugging, and discovery. It should be recorded as advisory or supporting material, not as part of the minimal proof chain.

NS-C.11. Candidate proof program

Definition 8.68 (Candidate AK–NS proof program). A *candidate AK–NS proof program* consists of the following tasks:

- (i) choose a target theorem template;
- (ii) specify the solution class and PDE assumptions;
- (iii) define a purpose-faithful realization from PDE data to Core-readable persistence or filtered data;
- (iv) prove constructibility and repaired evaluation compatibility;
- (v) establish the relevant Core certificate target;
- (vi) prove admissible representative and eligibility conditions when Ext^1 is invoked;
- (vii) construct required tower comparison artifacts when Type IV diagnostics are invoked;
- (viii) prove diagnostic soundness for monitored Type IV;
- (ix) prove budget admissibility under all approximation, normalization, and measurement defects;
- (x) prove a regularity, continuation, or non-blow-up bridge;
- (xi) assemble the NS proof object;
- (xii) classify all remaining gaps.

Definition 8.69 (Program status). A candidate AK–NS proof program has one of the following statuses:

- (i) *exploratory*: dominated by search or advisory evidence;
- (ii) *interface-specified*: target, realization, and bridge requirements are stated;
- (iii) *partially proved*: some bridge lemmas or Core components are proved;
- (iv) *Core-certified*: Core certificate is established but PDE soundness is incomplete;
- (v) *soundness-certified*: PDE soundness bridge is proved but Core certificate is incomplete or separate;
- (vi) *theorem-level*: all required components are proved and assembled.

Proposition 8.70 (Core-certified is not theorem-level by default). A *Core-certified AK–NS proof program* is not theorem-level unless the PDE soundness bridge is also proved.

Proof. Core certification establishes AK-internal certificate conditions. Theorem-level Navier–Stokes conclusion requires an additional bridge from those conditions to PDE regularity or the declared target statement. $\square$

Remark 8.71 (Program status should be honest). A proof program should report its status conservatively. Calling an exploratory or Core-certified program theorem-level without a proved soundness bridge violates the non-claim discipline.

NS-C.12. Relation to Clay-level formulation

Proposition 8.72 (Clay-level target requires global bridge). *A Clay-level target cannot be obtained from local or windowed Core certificates unless a global regularity bridge assembles the local certificates into the global PDE conclusion.*

Proof. Local or windowed certificates concern restricted spacetime or parameter domains. A Clay-level target is global in time and space under the declared formulation. A theorem assembling local certificate behavior into global existence and smoothness is therefore required. $\square$

Definition 8.73 (Clay-level proof obligation). *A Clay-level proof obligation is a required component for upgrading a local, continuation, or windowed AK–NS proof route to a Clay-level global result. It includes:*

- (i) global-in-time solution class compatibility;
- (ii) global local-to-global assembly;
- (iii) control of all possible singularity scenarios;
- (iv) compatibility with the standard Clay formulation;
- (v) proof that the AK-side certificate route covers every required PDE case;
- (vi) elimination of all analytic, Core-side, and bridge gaps.

Remark 8.74 (No Clay claim). This appendix does not claim to solve the Clay Navier–Stokes problem. It states what would be needed for an AK-based proof program to reach that level.

NS-C.13. Closure of the Navier–Stokes appendices

Definition 8.75 (NS appendix closure). *The NS appendix closure is the requirement that every Navier–Stokes-facing claim remain classified into one of the following layers:*

exploration, Core certificate, soundness bridge, proof-first theorem route, PDE theorem.

Proposition 8.76 (NS appendices close without proving Navier–Stokes). *Appendices NS-A–NS-C close the Navier–Stokes interface layer, but they do not prove a Navier–Stokes theorem.*

Proof. Appendix NS-A defines exploration. Appendix NS-B defines theoretical soundness. Appendix NS-C defines proof-first architecture and proof obligations. None of these appendices supplies a completed soundness bridge or a theorem-level proof object establishing a PDE target conclusion. Therefore the NS layer is structurally closed but not theorem-complete. $\square$

Remark 8.77 (Meaning of closure). Closure here means that the interface architecture is internally disciplined: exploration cannot masquerade as proof, Core pass cannot masquerade as PDE theorem, and proof-first obligations cannot be discharged by terminology. It does not mean that the Navier–Stokes problem is solved.

NS-C.13bis. v19 hardening: proof-object sovereignty and typed theorem routing

Definition 8.78 (NS proof-object sovereignty). *NS proof-object sovereignty* is the rule that theorem-level status is determined by a complete proof object, not by exploration success, Core certification alone, partial bridge diagrams, or terminology.

Proposition 8.79 (No proof-object sovereignty override). *No advisory, numerical, Core-certified, or partially bridged NS program is theorem-level unless the theorem-level proof object is complete.*

Proof. Theorem-level status requires all required Core, soundness, transport, analytic, artifact, and ledger components. Partial completion does not discharge the missing components. $\square$

Definition 8.80 (Typed NS route edge). A *typed NS route edge* is a declared implication in the proof route whose source layer, target layer, hypotheses, defects, artifacts, and status are recorded.

Definition 8.81 (NS route dependency graph). An *NS route dependency graph* is the directed graph of typed route edges required for a theorem-level proof object. It must distinguish PDE-side analytic implications, Core-side certificate implications, soundness bridges, normalization transports, discretization transports, and global assembly bridges.

Proposition 8.82 (Untyped route edges are non-verdict). *An untyped or unstated implication in an NS proof route cannot contribute to theorem-level status.*

Proof. Without source layer, target layer, hypotheses, and status, the implication is not audit-readable and cannot be checked as part of a proof object. $\square$

Definition 8.83 (NS proof evidence mode). An NS evidence item has one of the following modes:

- (i) *theorem*;
- (ii) *conditional-theorem*;
- (iii) *Core-certificate*;
- (iv) *interface-theorem*;
- (v) *Spec*;
- (vi) *advisory*;
- (vii) *search-artifact*;
- (viii) *failed*;
- (ix) *undefined*.

Proposition 8.84 (Evidence modes do not collapse). *A [Spec], advisory, search, or Core-certificate evidence item does not become a PDE theorem merely by appearing in a proof-first record.*

Proof. The evidence mode records the logical status of the item. The proof-first record aggregates obligations; it does not promote their status. $\square$

Definition 8.85 (NS typed budget failure). A *typed NS budget failure* occurs when the scalarized declared budget fails

$$\text{val}_B(\Sigma\delta_{\text{NS}}) < \text{gap}_\tau(\mathfrak{B}_{\text{NS}}),$$

or when a required non-scalar obligation is missing.

Proposition 8.86 (Scalar margin does not repair bridge gaps). *A small scalar budget margin does not repair a missing realization bridge, soundness bridge, compactness bridge, global assembly bridge, or Clay-level bridge.*

Proof. Bridge gaps are typed logical gaps. Scalar ledger bounds control declared discrepancies but do not supply missing implications. $\square$

Definition 8.87 (NS release-level closure). *NS release-level closure* means that NS-A, NS-B, and NS-C close the released Navier–Stokes interface stack in this revision by classifying exploration, Core certification, soundness, proof-first routing, and non-claims. It does not assert theorem-completeness.

Proposition 8.88 (Release-level closure is not theorem-completeness). *NS release-level closure does not prove any Navier–Stokes theorem and does not eliminate analytic, Core-side, or bridge gaps.*

Proof. Closure is a governance and interface-architecture property. Theorem-completeness requires a completed proof object and proved bridges. $\square$

Remark 8.89 (Proof-first records must expose gaps). A proof-first record is stronger when it explicitly exposes unresolved gaps, because it prevents architectural completeness from being misread as mathematical proof.

Remark 8.90 (NS-C budget notation). Whenever a budget comparison is consumed as a theorem-route condition, the typed form

$$\text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B})$$

should be used, with the protection family and all non-scalar obligations declared.

NS-C.14. Summary of proof-first program

Theorem 8.91 (Appendix NS-C proof-first package). *Within the Problem Interface layer, the following package is available.*

- (i) *A proof-first program begins from a declared target theorem, not from exploratory indicators.*
- (ii) *A Core certificate target must be distinguished from a PDE theorem target.*
- (iii) *Core certificate sufficiency for Navier–Stokes is [Spec] by default.*
- (iv) *Core-side proof obligations must be expressed through*

$$\text{Eval}_{i,w,\tau}, \quad C_{\tau,w}F, \quad \phi_{i,w,\tau}, \quad \text{val}_B(\Sigma\delta) < \text{gap}_\tau(\mathfrak{B}).$$
- (v) *The main proof obligations are realization, evaluation, collapse, diagnostic, compactness, global assembly, and regularity bridges.*
- (vi) *Analytic gaps and bridge gaps prevent theorem-level pass.*
- (vii) *Discretized proof-support mode is not theorem mode.*
- (viii) *Failure modes must be classified as realization, evaluation, eligibility, tower artifact, diagnostic, budget, soundness, or transport failures.*
- (ix) *An NS proof object must contain both Core certificate data and PDE-side soundness data.*

- (x) *Incomplete NS proof objects are not theorem-level.*
- (xi) *A minimal theorem route must separate proof components from supportive exploration artifacts.*
- (xii) *Core-certified status is not theorem-level by default.*
- (xiii) *Clay-level targets require a global regularity bridge and are not claimed here.*
- (xiv) *Appendices NS-A–NS-C close the Navier–Stokes interface layer without proving Navier–Stokes.*
- (xv) *Theorem-level status is governed by proof-object sovereignty.*
- (xvi) *The proof route must be represented by typed route edges and an audit-readable dependency graph.*
- (xvii) *Evidence modes do not collapse: [Spec], advisory, search, and Core-certificate modes remain distinct.*
- (xviii) *Typed budget failure includes scalar budget failure and missing non-scalar obligations.*
- (xix) *Release-level NS closure is not theorem-completeness.*

Proof. Items (i)–(ii) follow from the proof-first and target theorem definitions. Item (iii) is Proposition 8.20. Item (iv) follows from the repaired Core vocabulary requirements in Sections NS-C.1 and NS-C.4. Item (v) follows from the bridge lemma definitions. Item (vi) follows from Propositions 8.36 and 8.37. Item (vii) is Proposition 8.43. Item (viii) follows from the failure mode definitions. Items (ix)–(x) follow from the NS proof object schema and Proposition 8.60. Item (xi) is Proposition 8.66. Item (xii) is Proposition 8.70. Item (xiii) is Proposition 8.72. Item (xiv) is Proposition 8.76. Items (xv)–(xix) follow from Definition 8.78, Proposition 8.79, Definition 8.80, Definition 8.81, Proposition 8.82, Definition 8.83, Proposition 8.84, Definition 8.85, Proposition 8.86, Definition 8.87, and Proposition 8.88. $\square$

NS-C.15. Explicit exclusions

The following are not asserted in Appendix NS-C.

- No Navier–Stokes regularity theorem is proved.
- No global smoothness theorem is proved.
- No Clay Millennium problem is solved.
- No existence theorem is proved.
- No uniqueness theorem is proved.
- No continuation theorem is proved.
- No non-blow-up theorem is proved.
- No blow-up theorem is proved.
- No singularity exclusion theorem is proved.
- No Core certificate is asserted to imply PDE regularity without a proved soundness bridge.
- No exploratory indicator is treated as proof.

- No numerical evidence is treated as proof.
- No discretized proof-support mode is treated as theorem mode.
- No [Spec] bridge is treated as a theorem.
- No local certificate is treated as global regularity without a global bridge.
- No incomplete proof object is treated as theorem-level.
- No analytic gap is hidden or discharged by terminology.
- No bridge gap is hidden or discharged by terminology.
- No Core-side gap is hidden or discharged by terminology.
- No Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$
- No Type IV diagnostic is invoked without a declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No admissible representative

$$C_{\tau,W}F$$

is replaced by informal filtered-lift terminology.

- No repaired evaluation record

$$\text{Eval}_{i,W,\tau}(F)$$

is replaced by old collapse notation.

- No AK terminology alone is used to claim a Navier–Stokes theorem.
- No global exactness of T_{τ}^{bar} is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift C_{τ} is assumed.

NS-C.16. Provenance and status

The material in this appendix depends on:

- (i) the Navier–Stokes exploration protocol of Appendix NS-A;
- (ii) the theoretical soundness layer of Appendix NS-B;
- (iii) the normalization interface of Appendix AGI-N;
- (iv) the repaired barcode-level collapse and constructible persistence framework of Appendix A;
- (v) the admissible representative discipline of Appendix B;
- (vi) the eligible realization regime of Appendix C;

- (vii) the monitored Type IV diagnostic calculus and tower comparison artifact discipline of Appendix D;
- (viii) the manifest requirements of Appendix G;
- (ix) the ledger catalog of Appendix M;
- (x) the non-claim discipline of Chapter 7;
- (xi) the interface protocol of Chapter 8.

Its role is to define a proof-first architecture for a possible AK-based Navier–Stokes program without asserting that the program has been completed.

Status labels for Appendix NS-C. *Problem-interface definitions:* Definitions [8.4](#), [8.5](#), [8.8](#), [8.9](#), [8.10](#), [8.11](#), [8.12](#), [8.13](#), [8.15](#), [8.16](#), [8.17](#), [8.18](#), [8.19](#), [8.22](#), [8.23](#), [8.24](#), [8.25](#), [8.26](#), [8.27](#), [8.28](#), [8.29](#), [8.31](#), [8.32](#), [8.33](#), [8.34](#), [8.35](#), [8.39](#), [8.40](#), [8.41](#), [8.42](#), [8.45](#), [8.46](#), [8.47](#), [8.48](#), [8.49](#), [8.50](#), [8.51](#), [8.52](#), [8.53](#), [8.56](#), [8.57](#), [8.58](#), [8.59](#), [8.62](#), [8.63](#), [8.64](#), [8.65](#), [8.68](#), [8.69](#), [8.73](#), [8.75](#).

Problem-interface propositions/theorems: Propositions [8.6](#), [8.20](#), [8.36](#), [8.37](#), [8.43](#), [8.54](#), [8.60](#), [8.66](#), [8.70](#), [8.72](#), [8.76](#), Theorem [8.91](#).

Operational cautions: Remarks [8.1](#), [8.2](#), [8.3](#), [8.7](#), [8.14](#), [8.21](#), [8.30](#), [8.38](#), [8.44](#), [8.55](#), [8.61](#), [8.67](#), [8.71](#), [8.74](#), [8.77](#).

Additional v19.0 hardening definitions: Definitions [8.78](#), [8.80](#), [8.81](#), [8.83](#), [8.85](#), [8.87](#).

Additional v19.0 hardening propositions: Propositions [8.79](#), [8.82](#), [8.84](#), [8.86](#), [8.88](#).

Additional v19.0 hardening remarks: Remarks [8.89](#), [8.90](#).

Explicit exclusions: Section NS-C.15.

NS layer closure: Definition [8.75](#), Definition [8.87](#), Propositions [8.76](#), [8.88](#), Theorem [8.91](#).